

算法题讲义



by 一休老师

1 课程介绍

1.1 408 题型介绍

	选择题 40 题	第 41 题	第 42~47 题
分值	40×2	13~15 分	55~57 分
推荐时间	60~70 分钟	20 分钟	6×15 分钟

408 考试时间非常紧张，很多同学反映题目做不完。算法大题往往是耗时最多的部分。因此，对于大多数同学而言，算法题的目标应该是“在尽可能短的时间内，拿到尽可能多的分数”。请牢记：**时间就是分数！**

1.2 本课程使用指南

本课程主要面向算法基础相对薄弱的同学，内容讲解细致，旨在帮助大家稳妥地拿下算法题的大部分分数。建议在学习完数据结构基础课后再开始本课程的学习。

- **如果你是跨考或基础薄弱的同学：**

1. 先观看每一章的视频课程（包括直播和录播），理解核心思想。
2. 独立完成配套的练习题，动手实践。
3. 复看一遍本讲义，进行知识点的巩固和回顾。

- **如果你是基础较好的同学：**

1. 可以直接阅读讲义，快速掌握考点。
2. 对有疑问或掌握不牢固的部分，再去观看对应的视频讲解。

部分带 * 的小节表示可以跳过，部分带 ** 的小节表示仅供参考、意义不大、建议跳过。

焚诀

如果你觉得算法题拿70%的分数就挺好了，或者复习时间不够，可以如此学习：

- 第二、三、章可以只需要学习暴力解部分（直播的第1、2节课都可以先只学前半段），树和图尽可能学习。
- ‘*’ 部分全部跳过。

1.3 审题

审题是算法题的灵魂。

—— 一位一直不休息的考研前辈

审题是解对算法题的第一步，也是最重要的一步。一个条件的遗漏或误解，可能导致整个解题方向的错误。

1.3.1 题目类型

考研算法题的考查范围通常集中在以下四种数据结构上，我们将在后续章节中逐一攻克：

- 第二章：顺序表（数组）
- 第三章：链表
- 第四章：树
- 第五章：图

第六章是一些必要的补充知识，大家可以当字典查阅。

1.3.2 复杂度要求

正确解读题目对时间/空间复杂度的要求，是写出符合标准答案的关键。考试中的复杂度一般指 **最坏时间/空间复杂度**，只有当分析和快速排序有关的相关算法时使用**平均复杂度**

题目描述	时间复杂度要求	空间复杂度要求	备注
"时间上尽可能高效"	尽可能低	无要求	可以使用辅助空间来换取时间
"时间空间上都尽可能高效"	尽可能低	尽可能低	可以使用辅助空间来换取时间
"尽可能高效"	尽可能低	尽可能低	可以使用辅助空间来换取时间
"空间复杂度为 $O(1)$ 且时间上尽可能高效"	尽可能低	必须为 $O(1)$	严格限制空间，通常要求原地修改
无明确描述	能实现即可	能实现即可	常见于树和图，重点考察逻辑实现

对时间/空间复杂度有要求——顺序表、链表

这类题目通常存在多种解法（如暴力解、优化解），需要我们给出最优解。

例：【2011年真题】（15分）一个长度为 L （ $L \geq 1$ ）的升序序列 S ，处在第 $\lfloor L/2 \rfloor$ 个位置的数称为 S 的中位数。例如，若序列 $S_1 = (11, 13, 15, 17, 19)$ ，则 S_1 的中位数是15，两个序列的中位数是含它们所有元素的升序序列的中位数。例如，若 $S_2 = (2, 4, 6, 8, 20)$ ，则 S_1 和 S_2 的中位数是11。现在有两个等长升序序列 A 和 B ，试设计一个**在时间和空间两方面都尽可能高效的**算法，找出两个序列 A 和 B 的中位数。**分析：**题目明确要求“时间和空间都高效”，暗示我们暴力解法（如合并数组后找中位数）虽然能做，但可能不是最优解。我们需要寻找更优的方法。

无复杂度要求——树、图

这类题目通常没有明确的复杂度限制，意味着解法比较固定，只要能正确实现逻辑即可，而不考虑其他解法。

例：【2014年真题】（13分）二叉树的带权路径长度（WPL）是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T ，采用二叉链表存储，结点结构如下

left	weight	right
------	--------	-------

其中叶结点的weight域保存该结点的非负权值。设root为指向 T 的根结点的指针，请设计求 T 的WPL的算法 **分析：**题目没有提及任何复杂度要求，并且最后也没有要求分析复杂度。这强烈暗示我们，只需要通过遍历（如先序、后序或层序）等常规方法正确计算出WPL即可。

1.3.3 挖掘易被忽略的信息

括号内的条件

出题人通常会把一些重要的条件放在括号内，这些条件可能会限制我们做这道题的方法，一定要划记下来。

例：【2025年真题】（13 分）有两个长度均为 n 的一维整型数组 $A[n]$ 、 $res[n]$ ，计算 $A[i]$ 与 $A[j]$ ($0 \leq i \leq j \leq n-1$) 乘积的最大值，并将其保存到 $res[i]$ 中。若 $A[] = \{1, 4, -9, 6\}$ ，则得到 $res[] = \{6, 24, 81, 36\}$ 。现给定数组 A ，请设计时间空间上尽可能高效的算法 $CalMulMax$ ，求 res 中各元素的值。函数原型为：void $CalMulMax(int A[], int res[], int n)$ 。

分析：该题最重要的条件就是 ($0 \leq i \leq j \leq n-1$)，在考场上很多同学看到但忽视了这个条件，他们有一种惯性思维，碰到不会的顺序表的题就默认使用快速排序，但因为这个条件存在，我们不能随意改变数组 A 的分布，也就不能使用快速排序。

隐含的信息

除了明确文字，题目中还包含大量隐含信息，需要我们主动挖掘。

- **数据范围与特性：**
 - "均为正整数" -> 无需考虑 0 和负数。
 - "递增/有序" -> 暗示折半查找、双指针等高效算法。
 - "元素各不相同" -> 无需处理重复元素。
- **专业术语：**
 - "稳定的排序" -> 不能使用快排、堆排、希尔排序。
 - "原地算法" -> 空间复杂度要求 $O(1)$ 。
 - "要求用递归/非递归" -> 直接限定了算法的实现结构。

1.4 常见问题

? Q1: 考试中能否使用 C++ 的 STL (标准模板库)? 比如 sort()?

💡 A: 除了特别简单的数据结构(如栈、队列), 其他一般情况下 **不允许**。408 考试的核心目的之一就是考察你对基础数据结构和算法的实现。使用 STL 会掩盖这些实现细节。你应该默认排序、折半查找等过程都需要自己手动实现。有一个简单的判断方法, 如果这类不重要的数据结构或过程小于五行, 那直接使用没问题, 否则需要写清楚。**常见可直接使用的部分大家可以阅读第六章。**

? Q2: 需要写 main 函数和 #include 头文件吗?

💡 A: **不需要**。答题时只需要专注于实现题目要求的那个核心函数即可, 评卷老师只看你的核心算法逻辑。

? Q3: 算法的“基本设计思想”应该怎么写?

💡 A: 用清晰、简洁的自然语言, 分步骤描述你的算法逻辑。把它想象成你在向一个不懂编程的同学解释你的思路。例如: “1. 定义两个指针 i 和 j, 分别指向两个数组的起始位置。2. 循环比较两个指针所指元素的大小...”。清晰的思路是高分代码的基础。

? Q4: 如果想不出最优解, 只写了暴力解法能得分吗?

💡 A: **能, 而且推荐这样写!** 一个正确的暴力解法, 通常能拿到该题一半以上的分数。这远比一个错误的优化解或直接放弃要好得多。在考场上, 可以先写出暴力解确保基础分, 然后在此基础上思考如何优化。

2 顺序表

我们从一道经典的真题开始本章的学习。

【2011 年真题，15 分】 一个长度为 L ($L \geq 1$) 的升序序列 S ，处在第 $\lfloor L/2 \rfloor$ 个位置的数称为 S 的中位数。例如，若序列 $S_1 = (11, 13, 15, 17, 19)$ ，则 S_1 的中位数是 15。两个序列的中位数是含它们所有元素的升序序列的中位数。例如，若 $S_2 = (2, 4, 6, 8, 20)$ ，则 S_1 和 S_2 的中位数是 11。现在有两个等长升序序列 A 和 B ，试设计一个在时间和空间两方面都尽可能高效的算法，找出两个序列 A 和 B 的中位数。要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用 C、C++ 或 Java 语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。这道题完美地引出了我们本章将要探讨的核心主题：

- **如何从暴力解法出发？**（比如，将两个数组合并成一个大数组，再用快速排序找中位数）
- **如何利用“有序”这一特性进行优化？**（比如，使用多指针法进行归并，或者更进一步，利用折半查找的思想）
- **如何在多种解法中权衡时间与空间复杂度？** 带着这些问题，让我们正式进入顺序表的世界，系统学习解决这类问题的通用方法和技巧。

2.1 思考流程

在面对顺序表（或链表）的算法题时，推荐大家遵循以下思考流程，稳扎稳打，确保得分。

推荐流程：构思暴力解 -> 思考能否优化 -> 构思优化解

- **暴力解法：**最容易想到的、最直观的做法。通常思维难度小，但时间或空间复杂度较高，难以拿到满分。
- **优化解法：**在暴力解的基础上，利用题目给出的所有“隐藏”条件（如有序性、数据范围等），降低算法的复杂度。通常思维难度较大，但得分会更高。

以一道 15 分的算法大题为例，两种策略的投入产出比如下：

	所花时间	得分
暴力解法	5 分钟	11~12 分
优化解法	10~20 分钟	13~15 分



直奔最优解
20分钟0分×

暴力后优化
5分钟10分√

核心策略：先求暴力解，再求优化解。在考场上，孤注一掷地追求满分解法，如果失败可能导致一分不得。因此，无论如何，都应先确保能想出正确的暴力解法，拿到基础分，再尝试优化。

2.2 暴力解法

练习：请在 6 分钟内想出以下例题的直观做法（不考虑复杂度）

- **例题 2-1**：已知升序数组 A 含有 n 个整数，求是否存在值为 x 的元素，存在返回数组下标，不存在则返回 -1。例：A={1, 2, 3, 7, 8, 11, 15}, $n=7$, $x=8$, 则返回数组下标 4。
- **例题 2-2**：已知两个整数升序数组 A 和 B，元素个数分别为 n 和 m ，请将它们合并成一个新的升序数组 C。例：A={1, 2, 7}, B={3, 4, 5, 10}, 则 C={1, 2, 3, 4, 5, 7, 10}。
- **例题 2-3**：已知数组 A 中有 n 个非负元素，每个元素值都小于 n ，其中只有一个数重复出现过，请输出这个数。例： $n=6$, A={1, 2, 5, 2, 4, 3}, 则 2 重复出现，输出 2。
- **例题 2-4**：已知数组 A 含有 n 个整数 (n 为奇数)，求数组 A 的中位数。例： $n=5$, A={2, 5, 9, 3, 8}, 排序后为 {2, 3, 5, 8, 9}, 中位数为 5。
- **例题 2-5**：已知数组 A 含有 n 个整数，将小于 k 的元素放在数组左侧，大于等于 k 的元素放在右侧。例： $n=7$, A={4, 5, 9, 3, 7, 10, 1}, $k=6$, 处理后可能为 {1, 3, 4, 5, 9, 7, 10}。
- **例题 2-6**：已知数组 A 含有 n 个整数，要求另一个数组 res, $res[i]$ 表示长度为 $i+1$ 的所有连续子序列中，元素和的最大值。

例： $n=4$, A={1, -2, 3, 5}, 则 $res=\{5, 8, 6, 7\}$ 。

	连续子序列	元素和最大值
$i=0$	{1}、{-2}、{3}、{5}	{5}→5
$i=1$	{1, -2}、{-2, 3}、{3, 5}	{3, 5}→8
$i=2$	{1, -2, 3}、{-2, 3, 5}	{-2, 3, 5}→6
$i=3$	{1, -2, 3, 5}	{1, -2, 3, 5}→7

暴力解法，顾名思义，就是采用最直接、最不加思考优化的方法来解决。它不追求效率，只求得正确答案，因此常常是我们解题的“保底”方案或安全网。在顺序表的题目中，暴力解法通常体现为以下两种基本思想：**枚举法**和**排序法**。

2.2.1 枚举法

请大家回忆一下《复仇者联盟3》中，奇异博士在使用“超能力”看到 14,000,605 个不同的未来，发现只有 1 次可以赢。这种不辞辛劳、把所有可能性都尝试一遍来寻找答案的方法，就是我们今天要学习的一种最基础、也最花费时间的算法——**枚举法**。为了找到问题的答案，**枚举法**要求我们系统地、一个不漏地尝试所有可能的解，直到找到正确的解，或者试完所有可能之后发现无解。



我推进了时间 浏览未来的可能性
去看即将到来的这一战所有可能出现的结果
一千四百万六百零五种
我们赢了几种
一种

枚举法，即“暴力穷举”，是解决问题时最简单、最直接的思路。它的核心思想朴素而强大：**逐一考察所有可能的情况**，然后从中找出满足题目条件的解。在实现上，考试中对应着一层或多层 `for` 循环。虽然“暴力”，但一个好的枚举算法也需要清晰的逻辑：

1. **确定枚举对象和范围**：首先要明确我们要“枚举”的是什么？是数组中的一个元素、还是一个数对？然后确定其变化的范围，确保不漏掉任何一种可能，同时也要避免不必要的重复。
2. **进行条件判断**：在枚举的过程中，对每一种可能的情况，根据题目的要求进行判断。如果符合条件，就执行相应的操作（如计数、更新最值或直接返回）。

例题 2-1：对升序数组 A，求是否存在值为 x 的元素，存在返回 1，不存在返回 -1。

- **暴力解**：从头到尾遍历数组 A，逐个比较 `A[i]` 与 `x`。

```
int findElement(int A[], int n, int x)
{
    for (int i = 0; i < n; i++)
        if (A[i] == x) return i;    // 命中
    return -1;                      // 未找到，返回-1
}
```

以上就是枚举法，他的核心就是：确定枚举对象和范围→进行条件判断。复杂度较高，但思考难度很小。

2.2.2 快速排序

如果题目给的是一个无序数组，而有序能让问题变得简单，那么“先排序、再处理”就是一种常见的暴力解题策略。在需要不稳定的排序算法时，快速排序是首选，需要大家能熟练默写（注意参数）。

快速排序

快速排序 (Quick Sort) 是一种分治思想的体现。

1. **划分 (Partition)**: 在待排序区间内任选一个元素（一般是第一个元素）作为 **基准 (pivot)**。通过交换，将所有小于等于基准的元素移动到其左侧，大于等于基准的元素移动到其右侧。操作完成后，基准元素就到达了它的最终位置。
2. **递归 (Recurse)**: 对基准左右两个新的子区间，分别重复执行第一步，直到子区间长度小于等于 1。考试中，快速排序的时间复杂度按 **平均情况** $O(n \log n)$ 计算，空间复杂度为递归栈的深度，平均为 $O(\log n)$ 。

注意：快速排序有很多写法，如交换式和挖坑式，不同写法中间过程不一样但只要能实现排序即可，本代码适用于算法大题，方便记忆。

```
// 快速排序
void Qsort(int A[], int L, int R)
{
    if (L >= R) return; // 区间≤1, 返回
    int i = L, j = R;
    int pivot = A[L]; // 选首元素为基准

    // 不断交换左边比 pivot 大的数和右边比 pivot 小的数
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--; // 从右找第一个 < pivot 的数
        while (i < j && A[i] <= pivot) i++; // 从左找第一个 > pivot 的数
        if (i < j) swap(A[i], A[j]);
    }

    // 将基准值 A[L] 与 A[i] 交换
    swap(A[L], A[i]);

    // 递归处理左右两个子区间
    Qsort(A, L, i - 1);
    Qsort(A, i + 1, R);
}
```

例题 2-4：求含 n 个数 (n 为奇数) 的数组 A 的中位数。

- **暴力解**：先对数组 A 进行快速排序，使其变为升序数组。中位数就是排序后位于最中间的那个元素。

```
// 需要先默写上面的 Qsort 函数...
int findMedian(int A[], int n)
{
    // 调用快速排序
    Qsort(A, 0, n - 1);
    // 对于奇数长度的数组，中位数下标是 n/2
    return A[n / 2];
}
```

以上就是快速排序，考试时只需要默写快排过程即可，请注意调用时的下标。

注意事项:

💡 快速排序不是万能的，有如下情况下不能使用：

- 题目明示或暗示不能改变顺序表元素的次序。
- 要采用稳定的排序算法。

💡 如果题目需要得到降序，有两种方法：

- 方法1：修改快速排序代码（适合基础好的同学）。
- 方法2：快速排序代码不变，主过程段新增 for 循环将数组逆置。for (int i=0; i < n/2; i++) swap(A[i], A[n-i-1]);

💡 正常情况下快速排序代码可以直接默写，但如果数组的类型不是 int 就需要修改。

2.3 判断能否优化

写出暴力解后，下一步就是思考优化的可能性。从以下两个角度自问，往往能发现突破口：

1. **“条件浪费”**：题目中是否给了一些我没用上的重要条件？
 - 例如：数组是**“升序”**的，但我却从头到尾线性扫描，浪费了有序性。
 - 例如：元素值范围是**“0 到 $n-1$ ”**，但我却用了排序，浪费了数据范围的特性。
2. **“目标超额”**：我为了解决问题，是不是做了很多额外的不必要工作？
 - 例如：题目只要求**“找出最小值”**，我却把整个数组都排序了。
 - 例如：题目只要求**“把小于 k 的放左边，大于等于 k 的放右边”**，我却把左右两边的元素内部都排好序了。

最好的算法，就是把题目的所有条件都榨干，并且不做多余的操作。

思考：下面几种暴力解分别符合哪个标准

- 例题 2-1：对升序数组 A ，求是否存在值为 x 的元素，存在返回 1，不存在返回 -1。暴力解：枚举
- 例题 2-2：将两个升序数组 A 和 B 合并成一个新的升序数组 C 。暴力解：快速排序
- 例题 2-3：数组 A 中有 n 个非负元素，每个元素绝对值 $< n$ ，其中只有一个数重复出现过，请输出这个数。暴力解：枚举
- 例题 2-4：求含 n 个数（ n 为奇数）的数组 A 的中位数。暴力解：快速排序
- 例题 2-5：已知数组 A 含有 n 个整数，现在要将小于 k 的元素放在数组左侧，大于等于 k 的元素放在右侧，如何实现这个功能。暴力解：快速排序
- 例题 2-6：已知数组 A 含有 n 个整数，现在要求另一个数组 res ， $res[i]$ 表示长度为 $i+1$ 的连续子序列中元素和的最大值，求 res 数组。暴力解：枚举

2.4 优化解

针对前文分析，我们介绍几种核心的优化思想。

2.4.1 折半查找

核心要素：

- **使用条件：**有序的顺序表（数组）。
- **核心思想：**通过不断与区间中点比较，每次都将在查找范围 **缩小一半**。
- **复杂度：**时间 $O(\log n)$ ，空间 $O(1)$ 。

当题目出现“有序数组”（且是顺序表，不是链表）时，你的脑海里应该立刻响起“折半查找”的警报，真题中频繁考察。

两种经典模板

对比项	模板一：『等值判断』模板 (优先使用)	模板二：『区间收敛』模板
适用场景	查找一个确切的值，通常适合元素不重复。	查找满足条件的第一个或最后一个元素，元素可以重复。
查找成功	$A[mid] == x$ 判断。	区间缩小为只有一个元素，需额外判断。
查找失败	区间中没有元素	区间缩小为只有一个元素，需额外判断。
循环条件	$while (L \leq R)$ (区间没有元素时退出循环)	$while (L < R)$ (区间收敛为一个元素时退出循环)
边界更新逻辑	$R = mid - 1$ 或 $L = mid + 1$ (直接排除 mid)	$R = mid$ 或 $L = mid + 1$ (可能不能排除 mid)

模板一：『等值判断』

这是最基础、最常用的模板，其核心是在循环内部通过 $if (A[mid] == x)$ 直接判断是否找到了目标值。

```
// 折半查找
int B_Search(int A[], int L, int R, int x)
{
    int mid;
    while (L <= R)           // 区间非空则继续
    {
        mid = (R + L) / 2;    // 求中点
```

```

    if (A[mid] == x)
        return mid;           // 查找成功
    else
        if (x < A[mid])
            R = mid - 1;       // x 在左半区间, L ~ mid-1
        else L = mid + 1;     // x 在右半区间, mid+1 ~ R
    }
    return -1;                // 查找失败
}

```

模板二：『区间收敛』

这个模板通过不断**收敛（逼近）**区间来锁定一个边界，专门用于解决**边界查找**问题。

```

// 折半查找第一个等于 x 元素的下标
int B_Search(int A[], int L, int R, int x)
{
    int mid;
    while (L < R)               // 收敛至单元素
    {
        mid = (R + L) / 2;
        if (A[mid] >= x) R = mid; // x 在 L ~ mid
        else L = mid + 1;        // x 在 mid+1 ~ R
    }
    return L;                  // 需额外判断是否等于 x
}

```

注意

- **循环条件与边界更新的匹配**：L ≤ R 和 L < R 分别对应着不同的区间定义和边界更新逻辑，必须配套使用，否则容易导致死循环或漏解。
- **边界更新的细节**：在模板二中，如果将 R = mid 误写为 R = mid - 1，若 A[mid] 正好是你要找的那个边界值时，就会把它错误地排除掉。
- **学会灵活应用**：很多时候折半查找的代码并不能直接默写使用，而是需要根据题目要求来修改，例如题目中给的是降序序列。

例题 2-1：对升序数组 A，求是否存在值为 x 的元素，存在返回 1，不存在返回 -1。

暴力解：枚举

优化解：折半查找

解析：关键字“升序数组”，可使用折半查找优化。本题只是查找一个确切的值，使用最直观的模板一即可。

```

// 折半查找模板一
int B_Search(int A[], int L, int R, int x)
{

```

```

int mid;
while (L <= R)           // 区间非空则继续
{
    mid = (R + L) / 2;    // 求中点
    if (A[mid] == x)
        return mid;      // 查找成功
    else
        if (x < A[mid])
            R = mid - 1;  // x 在左半区间, L ~ mid-1
        else L = mid + 1; // x 在右半区间, mid+1 ~ R
}
return -1;               // 查找失败
}

// 在升序数组 A 中判断是否存在 x
int ans(int A[], int n, int x)
{
    // 直接调用折半查找函数
    if (B_Search(A, 0, n - 1, x) == -1)
        return -1;
    else return 1;
}

```

而如果改一下题目，就适合模板二了：**在一个升序数组 A 中，查找第一个等于 x 的元素的位置，如果存在则返回下标，如果不存在则返回 -1。**（数组中可能存在多个 x）

解析：此题可能有重复元素，因此适用于模板二。因为数组中可能存在多个 x（例如 [1, 2, 3, 3, 3, 4, 4]，查找 3），如果使用模板一，当 A[mid] == x 时直接返回，可能会返回中间的那个 3，而不是第一个。我们需要找到满足 A[i] == x 的最小的 i。

这正是模板二的应用场景。我们使用模板二找到这个位置 pos 后，还需要额外进行一次判断：检查 A[pos] 是否真的等于 x。如果等于，pos 就是答案；如果不等（或者 pos 超出数组范围），则说明数组中不存在 x。

```

// 折半模板二
int B_Search(int A[], int L, int R, int x)
{
    int mid;
    while (L < R)
    {
        mid = (L + R) / 2;
        if (A[mid] >= x) R = mid;
        else L = mid + 1;
    }
    return L;
}

// 在升序数组 A 中查找第一个等于 x 的元素的位置
int ans(int A[], int n, int x)
{
    int pos = B_Search(A, 0, n - 1, x);
    if (A[pos] == x) return pos;    // 查找成功
    else return -1;                // 查找失败
}

```



2.4.2 散列表

使用条件：数据范围有限

- **使用条件：**数据范围有限，可以用数组下标直接表示数据的值。
- **核心思想：**用空间换时间。创建一个辅助数组 `count[]`，`count[i]` 用于记录数值 `i` 出现的相关信息（如次数）。
- **复杂度：**时间 $O(n)$ ，空间 $O(M)$ ，其中 M 是数据范围。

当题目明示或者通过分析确定“元素值为 0 到 $n-1$ ”、“元素绝对值小于 n ”等等，此时我们要马上考虑使用散列表。

例题 2-3：数组 A 中有 n 个非负元素，每个元素绝对值 $< n$ ，其中只有一个数重复出现过，请输出这个数。

如： $n=6$ ， $A=\{1, 2, 5, 2, 4, 3\}$

暴力解：枚举

优化解：散列表

解析：关键信息“绝对值 $< n$ ”，可使用散列表

```
void Hash(int A[], int n)
{
    // 辅助数组充当散列表，大小为 n，下标 0 ~ n-1
    int count[n] = {0}; // 需要初始化
    for (int i = 0; i < n; i++)
    {
        // A[i] 的值在 0 到 n-1 之间，可以直接用作 count 数组的下标
        if (count[A[i]] > 0)
            输出 A[i]; // 说明 A[i] 之前出现过
        count[A[i]]++; // 标记 A[i] 出现
    }
}
```

2018 年 41 题

(13分) 给定一个含 n ($n \geq 1$) 个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如，数组 $\{-5, 3, 2, 3\}$ 中未出现的最小正整数是 1；数组 $\{1, 2, 3\}$ 中未出现的最小正整数是 4。

解析：可以证明最小未出现的正整数 t 一定 $\leq n+1$ （因为如果 $t > n+1$ ，则 $1 \sim t-1$ 都会出现一次，但只有 n 个整数，所以 $t-1 \leq n$ ，即 $t \leq n+1$ ）。因此虽然题目中没有明确给出取值范围，但是通过我们的分析，只有 $0 \sim n+1$ 这个范围的整数，所以可以使用散列表法。

```
int ans(int A[], int n)
{
```

```

int count[n+2] = {0};           // 0~n+1
for (int i=0; i<n; i++)         // 扫描数组A
    if (A[i]>0 && A[i]<=n+1)     // 当0<A[i]<=n+1时才修改
        count[A[i]]++;
for (int j=1; j<=n+1; j++)      // 扫描count数组
    if (count[j]==0)            // j未出现过，是最小未出现正整数
        return j;              // 输出j
}

```

注意

- 使用散列表时一定要考虑 $A[i]$ 的大小是否会超出 count 下标的范围。
- 新建辅助数组（散列表）需要初始化。
- 当题目限制元素的范围时可以考虑使用散列表，条件可以是直接给出也可能是暗示。

2.4.3 预处理

- **使用条件**：需要频繁计算区间和、积、最值等。
- **核心思想**：类似散列表，以空间换时间。提前计算并存储一些结果，供后续快速查询。

考试中常见的预处理分为两类：求区间之和（前缀和）、求区间最值。

前缀和

如果需要多次查询不同区间的和，可以先用 $O(n)$ 的时间预处理出前缀和数组 `sum`，其中 `sum[i]` 表示原数组前 i 个元素的和（下标 $0 \sim$ 下标 $i-1$ ）。之后查询任意区间 $[L, R]$ 的和，只需要 $O(1)$ 的时间。

如果数组 A 下标从 0 开始存放数据，

$$sum[i] = \sum_{k=0}^{i-1} A[k]$$

， $A[L \sim R]$ 之间所有元素之和 = `sum[R+1] - sum[L]`。

只需要提前处理一次 `sum` 数组，之后就能在 $O(1)$ 的时间快速计算元素和。

例题 2-6：已知数组 A 含有 n 个整数，现在要求另一个数组 `res`，`res[i]` 表示长度为 $i+1$ 的连续子序列中元素和的最大值，求 `res` 数组，函数原型为 `void ans(int A[], int n)`。

例： $n=4$ ， $A=\{1, -2, 3, 5\}$

	连续子序列	元素和最大值
$i=0$	$\{1\}$ 、 $\{-2\}$ 、 $\{3\}$ 、 $\{5\}$	$\{5\} \rightarrow 5$
$i=1$	$\{1, -2\}$ 、 $\{-2, 3\}$ 、 $\{3, 5\}$	$\{3, 5\} \rightarrow 8$
$i=2$	$\{1, -2, 3\}$ 、 $\{-2, 3, 5\}$	$\{-2, 3, 5\} \rightarrow 6$
$i=3$	$\{1, -2, 3, 5\}$	$\{1, -2, 3, 5\} \rightarrow 7$

则处理之后的 `res` 数组为 $\{5, 8, 6, 7\}$

暴力解：枚举，时间复杂度 $O(n^3)$

```
void ans(int A[], int n)
{
    // 创建将res数组初值设为最小负数
    int res[n] = {0};
    for (int i = 0; i < n; i++)
        res[i] = INT_MIN;
    // 枚举每种A[i~j]的连续子序列
```

```

for (int i = 0; i < n; i++)
    for (int j = i; j < n; j++)
    {
        int temp = 0;
        for (int k = i; k <= j; k++)
            temp += A[k];
        if (temp > res[j-i])
            res[j-i] = temp;
    }
    输出res数组
}

```

优化解：预处理，时间复杂度 $O(n^2)$

解析：在枚举的过程中我们发现需要大量重复计算，可以预处理前缀和数组，然后每次计算子序列和的时间复杂度就变成了 $O(1)$

```

void ans(int A[], int n)
{
    // 创建并初始化前缀和数组
    int res[n];
    for (int i = 0; i < n; i++)
        res[i] = INT_MIN;
    // 创建并初始化前缀和数组
    int sum[n + 1] = {0};
    for (int i = 1; i <= n; i++)
        sum[i] = sum[i-1] + A[i-1];
    // 枚举每种A[i~j]的连续子序列
    for (int i = 0; i < n; i++)
        for (int j = i; j < n; j++)
        {
            int temp = sum[j+1] - sum[i];
            if (temp > res[j-i])
                res[j-i] = temp;
        }
    输出res数组
}

```

当然本题还有其他写法，甚至有时间复杂度为 $O(n)$ 的方法，但是考研可能用不到，所以此处跳过。

区间最值

如果要频繁求出数组某一区间内的最小值或最大值，也可以通过预处理来实现，比如说设置数组 B ， $B[i]$ 表示 i 下标及之前元素的最值，或者是 i 下标及之后的最值，根据题意来决定。

例题 2-7：已知数组 A 含有 n 个正整数，现在要求另一个数组 res ，计算 $A[i]$ 与 $A[j]$ ($0 \leq i \leq j \leq n-1$) 乘积的最大值，并将其保存到 $res[j]$ 中。求 res 数组，函数原型为 `void ans(int A[], int n)`。

若 $A[] = \{3, 1, 2, 7, 6\}$ ，则得到 $res[] = \{9, 3, 6, 49, 42\}$ 。

暴力解：枚举，时间复杂度为 $O(n^2)$

```
void ans(int A[], int n)
{
    int res[n];
    for (int j = 0; j < n; j++)
    {
        res[j] = INT_MIN;
        for (int i = 0; i <= j; i++)
            if (A[i] * A[j] > res[j]) res[j] = A[i] * A[j];
    }
    输出res数组
}
```

优化解：预处理，时间复杂度为 $O(n)$

解析：要快速确定 $A[i]$ 与 $A[j]$ 的最大乘积，我们可以预处理，使用数组 B 提前维护， $B[j]$ 表示 $A[0 \sim j]$ 之间的最大值， $res[j] = B[j] \times A[j]$ 。

```
void ans(int A[], int n)
{
    int res[n], B[n];
    B[0] = A[0];
    for (int i = 1; i < n; i++)
        B[i] = max(B[i-1], A[i]);
    for (int j = 0; j < n; j++)
        res[j] = B[j] * A[j];
    输出res数组
}
```

通过预处理每个位置之前的最大值，我们只需 $O(1)$ 的时间即可确定当前位置的最值，使得整个算法的时间复杂度降为 $O(n)$ 。

当然，细心的同学可能会发现，其实本题的预处理并不需要数组 B ，使用一个变量 B ，然后把两次循环合并即可。

```
void ans(int A[], int n)
{
```

```
int res[n];
int B = A[0];
for (int j = 0; j < n; j++)
{
    B = max(B, A[j]);
    res[j] = B * A[j];
}
输出res数组
}
```

2.4.4 多指针法

适用条件：线性表（顺序表、链表），主要是**有序线性表**。

核心思想：在一个或多个线性表上设置两个或多个下标变量（指针），根据**特定规则**移动它们，协同完成任务。“**规则**”是**多指针法的灵魂**。它指的是根据题目要求和数据特性（如顺序性）制定出的指针移动策略。算法的每一步都依据这个规则来判断：应该移动哪个指针？移动后执行什么操作？一个设计精良的规则能确保在指针的移动过程中，我们总能做出局部最优决策，并最终导向全局最优解，同时避免了对无效状态的搜索。

多指针法可适用于以下两种场景：

- **多个线性表：**每个线性表设置一个指针，如合并多个有序线性表。
- **单个线性表：**在一个线性表中设置两个指针，例如一前一后或一快一慢，用于查找或调整元素顺序。

经典应用场景包括合并多个有序线性表、寻找第 k 个元素、归并排序等。**注意：不可机械套用模板，应深入理解后灵活应用。**

多指针法

对比枚举法，多指针法的精髓在于其高效的“剪枝”能力，而这种能力完全来自于指针移动的“**规则**”。这个规则通常基于数组的有序性等已知条件。例如，在合并两个升序数组时，规则是“取较小值，移动其指针”。这个简单的规则保证了我们每次选择的都是当前所有未合并元素中的最小值，从而无需比较所有未合并的元素。我们之所以敢于只移动一个指针，是因为我们确信被“跳过”的那个数组中的其他元素（在该指针之后的元素）必然比当前选中的元素更大，不可能成为下一个合并的元素。这种基于规则的确定性移动，让我们一次性地排除了大量的比较，实现了从 $O((m+n)^2)$ 到 $O(m+n)$ 的飞跃。可以说，**找到并证明这个“规则”的正确性，是使用多指针法解决问题的关键。**

多指针法考虑三个要点：**指针起始位置、移动规则、停止处理。**

类型1：多个有序顺序表

这是多指针法最经典的应用场景。**通过在每个有序表上设置一个指针（可能在头或尾），并根据特定规则移动它们，可以高效地解决多种问题。**我们从以下题目中感受：

例题 2-2：给定升序数组 A、B，长度分别为 n 、 m ，合并成一个新的升序数组 C 并输出。

例如：A = {1, 2, 7}，B = {3, 4, 5}。

枚举法：最暴力的枚举是，为了确定新数组 C 的下一个元素，我们需要比较 A 和 B 中还剩下的所有元素，然后找出最小的。确定第二个元素时，又要重复这个过程。这显然是低效的，时间复杂度为 $O((m+n)^2)$ 。

多指针法：在枚举法的基础上，发现升序条件没有用到，数组 A、B、C 都会是升序数组，因此数组 C 的下一个元素一定是数组 A 中最小元素及数组 B 中最小元素其中较小的那个。最小元素在升序数组中是最左侧元素，所以设置两个指针分别指向 A、B 现存最小元素，每次比较指针所指元素并移动即可，时间复杂度 $O(m+n)$ 。

- **分析：**数组 C 需要是升序，元素是从小到大存放，下一个元素一定是数组 A 中最小元素及数组 B 中最小元素其中较小的那个。所以每个数组的指针都应该分别指向他们还未处理的最小元素（即最左侧的下标）。
- **起始位置：**数组 A、B、C 的指针都是从左侧开始向右移动。
- **规则：**比较数组 A、B 对应指针所指的元素，将较小的元素复制到结果数组中，然后将该较小元素所在的数组的指针、最终数组的指针各移动一位。不断重复这个过程。这个规则保证了结果数组始终是升序的，因为我们每次都选取了当前所有待选元素中的最小值。
- **停止处理：**当其中一个数组的指针移出数组（即遍历完该数组的所有元素）时，退出循环。此时，另一个数组中可能还有剩余的元素，由于原数组是有序的，这些剩余元素必然都大于等于已合并的所有元素。因此，只需将这个数组的剩余部分依次复制到结果数组的末尾即可。

```
void Merge(int A[], int n, int B[], int m)
{
    int C[n + m];
    int i = 0, j = 0, k = 0;    // i, j, k 分别是 A, B, C 的指针

    // 直到有一个数组遍历完退出循环
    while (i < n && j < m)
        if (A[i] < B[j])
            C[k++] = A[i++];
        else
            C[k++] = B[j++];

    // 复制 A 中剩余的元素
    while (i < n)
        C[k++] = A[i++];

    // 复制 B 中剩余的元素
    while (j < m)
        C[k++] = B[j++];

    从左到右输出数组 C 中的元素
}
```

例题 2-8 改编：给定升序数组 A、B，数组 A 中都是负整数，数组 B 中都是非负整数，将所有元素平方后按降序排序，存入新数组 C 并输出。

例如： $A = \{-4, -2\}$ $B = \{0, 1, 3\} \rightarrow C = \{16, 9, 4, 1, 0\}$ 。

暴力解：将所有元素平方后存入数组 C，对数组 C 快速排序，时间复杂度 $O((m+n)^2)$ 。

优化解：多指针法，时间复杂度 $O(m+n)$ 。

- **分析：**目标数组 C 需要是降序，元素应从大到小存放。C 的下一个元素必然是 A 和 B 当前剩余元素平方后的较大者。
 - 对于 A（负数升序），其绝对值是降序的，所以平方后的值也是**降序**（ $(-4)^2 > (-2)^2$ ）。最大值在 A[0]。
 - 对于 B（非负数升序），其绝对值是升序的，所以平方后的值是**升序**（ $0^2 < 1^2 < 3^2$ ）。最大值在 B 的末尾。
- **起始位置：**数组 A、C 的指针都是从左侧开始向右移动，数组 B 的指针是从右向左移动。
- **规则：**比较 $|A[i]|$ 和 $|B[j]|$ ，将绝对值较大者的平方值放入 $C[k]$ ，然后移动被选中的指针（i 向右或 j 向左）和结果指针 k。
- **停止处理：**当其中一个指针（i 或 j）越界时，主循环停止。此时，将另一个数组中所有剩余元素的平方值依次复制到 C 的末尾。

```
void mergeAndSquare(int A[], int n, int B[], int m)
{
    int C[n + m];
    int i = 0; // A 的指针，从左向右
    int j = m - 1; // B 的指针，从右向左
    int k = 0; // C 的指针，从左向右

    // 直到有一个数组遍历完退出循环
    while (i < n && j >= 0)
    {
        if (abs(A[i]) > abs(B[j]))
        {
            C[k++] = A[i] * A[i];
            i++;
        }
        else
        {
            C[k++] = B[j] * B[j];
            j--;
        }
    }

    // 复制 A 中剩余元素的平方
    while (i < n)
    {
        C[k++] = A[i] * A[i];
        i++;
    }

    // 复制 B 中剩余元素的平方
    while (j >= 0)
    {
        C[k++] = B[j] * B[j];
        j--;
    }

    // 输出 C...
}
```

类型2：单个有序顺序表

在处理单个有序顺序表时，我们常常可以根据题目的特点，将这个单一的表看作是从某个“切点”分成的两个逻辑上的有序子序列。指针则分别置于这些逻辑子序列的端点，然后像处理多个有序表一样进行操作。首尾双指针（相向而行）就是这种思想最常见的体现，首尾的指针最终会在中间相遇。

例题 2-8：升序数组 A 中包含 n 个整数，将其元素平方后按降序排序，存入新数组 B 并输出。

例如：A = {-4, -2, 0, 1, 3} → B = {16, 9, 4, 1, 0}。

暴力解：平方后快速排序，时间复杂度 $O(n\log n)$ 。

优化解：多指针法，时间复杂度 $O(n)$ 。

思路分析：对于升序序列，平方后的最大值一定产生于原数组的“两端”（绝对值最大的地方）。因此，可以用两个指针 i, j 分别指向 A 的头和尾，比较 $|A[i]|$ 和 $|A[j]|$ ，将较大的平方值从后往前放入结果数组 B。

- **分析：**对于一个包含正负数的升序序列，其平方后的最大值一定产生于原数组的“两端”（绝对值最大的地方）。这相当于将原数组以 0 为“切点”，分成了两个逻辑序列：一个负数序列（绝对值降序）和一个正数序列（绝对值升序）。我们可以设置两个指针，一个在数组头，一个在数组尾，它们分别指向这两个逻辑序列中绝对值最大的元素。
- **起始位置：**头指针 i 指向 $A[0]$ ，尾指针 j 指向 $A[n-1]$ 。结果数组的指针 k 应指向 B 的首位 $B[0]$ 。
- **规则：**比较 $|A[i]|$ 和 $|A[j]|$ ，将绝对值较大者的平方值放入 $B[k]$ ，然后移动相应的指针（i 向右或 j 向左）和结果指针 k（向右）。
- **停止处理：**当头尾指针相遇或交错（ $i > j$ ）时，循环结束。

有的同学会疑惑为什么这里的判断条件只是 $i \leq j$ ，而不需要考虑正负发生变化时的情况，这里可以证明：当有正负发生变化时，那剩下的所有元素正负都相同，也就是说剩下的元素都是单调的，那在单调的情况下继续比较头尾一定能够保证只有一端移动。所以这种情况下仍然是可以继续执行直到 $i < j$ 才结束，并且没有元素剩下。

```
void squareAndSort(int A[], int n)
{
    int B[n];
    int i = 0, j = n - 1;    // 头尾指针
    int k = 0;               // B 指针，从后往前

    // 直到指针相遇退出循环
    while (i <= j)
    {
        if (abs(A[i]) > abs(A[j]))
        {
            B[k++] = A[i] * A[i];
            i++;
        }
        else
        {
            B[k++] = A[j] * A[j];
            j--;
        }
    }
}
```

```
        i++;  
    }  
    else  
    {  
        B[k++] = A[j] * A[j];  
        j--;  
    }  
}  
// 输出 B...  
}
```

* 类型3：单个无序顺序表

无序情况下的多指针法稍复杂，考察频率较低。学有余力的同学可以试着学习本节，掌握单个无序顺序表的处理即可，该方法常用于实现**划分 (Partition)** 操作。

关于**划分 (Partition)** 操作，请读者回忆快速排序的过程，本节的方法都是基于快速排序的思路。

例题 2-9：给定数组 A 含有 n 个互异整数，以 $A[0]$ 为基准，将小于 $A[0]$ 的元素放左侧，大于 $A[0]$ 的元素放右侧。

例如： $A = \{2, 5, 3, -1, -2\} \rightarrow$ 可能结果为 $\{-2, -1, 2, 3, 5\}$ 或 $\{-1, -2, 2, 3, 5\}$ 或 $\{-2, -1, 2, 5, 3\}$ 或 $\{-1, -2, 2, 5, 3\}$ 。

- **暴力解：**快速排序，时间复杂度 $O(n \log n)$ 。
- **优化解：**多指针法，时间复杂度 $O(n)$ 。

思路分析：使用快速排序的单次划分思想，两个指针分别从数组两端向中间移动，分别找到大于和小于基准的元素进行交换，直到相遇后再将基准元素放入正确位置。

```
void ans(int A[], int n)
{
    int i = 0, j = n - 1;           // i, j 为左右指针
    int pivot = A[0];
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--;
        while (i < j && A[i] <= pivot) i++;
        if (i < j) swap(A[i], A[j]); // A[j] < pivot < A[i], 交换
    }
    swap(A[0], A[i]);               // 基准归位
}
```

例题 2-10：给定数组 A 含有 n 个互异整数，将较小的一半元素放左侧，较大的一半放右侧。

例如： $A = \{2, 5, -1, -2\} \rightarrow$ 可能结果为 $\{-2, -1, 2, 5\}$ 或 $\{-1, -2, 2, 5\}$ 或 $\{-2, -1, 5, 2\}$ 或 $\{-1, -2, 5, 2\}$ 。

- **暴力解：**快速排序，时间复杂度 $O(n \log n)$ 。
- **优化解：**多指针法，时间复杂度 $O(n)$ 。

思路分析：在上一道例题的划分基础上改进，上一题会划分成左区间、 x 、右区间， x 所在下标是 i 。但是本题要求两个区间各占一半，若 $i == \lfloor n/2 \rfloor$ ，则说明正好平分区间，如果小于则说明右边的区间过大应该继续处理右边的区间，而如果大于则说明左边的区间过大应该继续处理左边的区间。

```
void ans(int A[], int n)
{

```

```

int L = 0, R = n - 1;           // L和R是当前处理区间的边界
while (true)
{
    int i = L, j = R;
    int pivot = A[L];
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--;
        while (i < j && A[i] <= pivot) i++;
        if (i < j) swap(A[i], A[j]);
    }
    swap(A[L], A[i]);
    if (i == n / 2) return;
    if (i < n / 2) L = i + 1;    // 继续处理右区间
    else R = i - 1;             // 继续处理左区间
}
}

```

例题 2-9 (按 $A[0]$ 划分) 的解法就是快排的单趟 Partition。

例题 2-10 (划分为大小两半) 的解法, 则是在 Partition 思想的基础上, 不断调整, 直到基准 pivot 的最终位置恰好是 $n/2$ 。这被称为 **快速选择 (Quick Select)** 算法。

✏ 快速排序 (Quick Sort) vs. 快速选择 (Quick Select)

- **目标不同**: 快排的目标是**整个数组有序**; 快选的目标是找到**第 k 小的元素**。
- **递归方式不同**:
 - 快排: 划分后, **递归处理左右两个子区间**。 $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$
 - 快选: 划分后, 根据划分点的位置, **只递归处理其中一个子区间**。 $T(n) = T(n/2) + O(n) \rightarrow O(n)$

这就是为什么“找中位数”比“完整排序”更快的原因。

2.5 章节练习

2.5.1 习题

【* 习题 2-1】 (13 分) 已知升序数组 A 含有 n 个整数, 求是否存在值为 x 的元素, 存在返回数组下标, 不存在则返回 -1。设计一个在时间上尽可能高效的算法实现该功能。例: $A=\{1, 2, 3, 7, 8, 11, 15\}$, $n=7$, $x=8$, 则返回数组下标为 4。(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

【* 习题 2-2】 (13 分) 已知两个整数升序数组 A 和 B, 他们的元素数分别为 n 和 m, 请将他们合并成一个新的升序数组 C, 并输出 C 数组。设计一个尽可能高效的算法实现该功能。
例: $A = \{1, 2, 7\}$, $n = 3$, $B = \{3, 4, 5, 10\}$, $m = 4$, 则 $C = \{1, 2, 3, 4, 5, 7, 10\}$ 。(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

【* 习题 2-3】 (15 分) 已知数组 A 中有 n 个非负元素, 每个元素绝对值 $< n$, 其中只有一个数重复出现过, 请输出这个数, 否则输出-1。设计一个时间上尽可能高效的算法实现该功能。
例: $n = 6$, $A = \{1, 2, 5, 2, 4, 3\}$, 则 2 重复出现, 输出 2。 (1) 给出算法的基本设计思想。
(2) 根据设计思想, 采用 C、C++或 Java 语言描述算法, 关键之处给出注释。 (3) 说明你所设计算法的时间复杂度和空间复杂度。

【习题 2-4】（15 分）已知整数数组 A 含有 n 个不相等的整数（ n 为奇数，且数组 A 中每个元素的绝对值 $\leq n$ ）。设计一个时间上尽可能高效的算法，求数组 A 的中位数。例： $n=5$ ， $A=\{-3, 0, 4, -2, 3\}$ ，则中位数为 0。（1）给出算法的基本设计思想。（2）根据设计思想，采用 C、C++ 或 Java 语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。

【习题 2-5】 (15 分) 已知数组 A 含有 n 个整数, 现在要将小于 k 的元素放在数组右侧, 大于等于 k 的元素放在左侧, 设计一个时间上尽可能高效的算法实现这个功能。例: $n = 7$, $A = \{4, 5, 9, 3, 7, 10, 1\}$, $k = 6$, 则处理之后的数组可以为 $\{10, 7, 9, 3, 5, 4, 1\}$, 其中 10、7、9 比 6 大, 在数组左侧; 而 3、5、4、1 小于等于 6, 在数组右侧。(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

【习题 2-6】 (13 分) 有一个数组 A 由 n 个互不相同的正整数组成 (n 为奇数), 该序列中的数先减后增, 中位数是最小值, 输入一个整数 key 。请设计一个时间和空间上尽可能高效的算法, 判断 key 是否在该数组中, 如果是则返回数组下标 (下标从 0 开始), 否则返回 -1。例如对应 $A = \{10, 7, 5, 2, 6, 13, 15\}$, $key = 5$ 时返回下标 2, 若 $key = 8$ 则返回 -1。 (1) 给出算法的基本设计思想。 (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。 (3) 说明你所设计算法的时间复杂度和空间复杂度。

【习题 2-7】 (13 分) 定义距离 $d = |x - y|$, 给定 2 个长度为 n 的升序数组 a 、 b , x 是数组 a 中的某个元素, y 是数组 b 中的某个元素。请设计一个尽可能高效的算法, 计算并输出所有可能的最小距离 d 。例如 数组 $a = \{-16, -8, 5, 8, 13\}$, $b = \{-2, 0, 2, 6, 10\}$, 则最小距离为 1, 相应的 $x=5, y=6, d = |5 - 6|=1$ 。(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

【习题 2-8】 (15 分) 已知升序数组 A 含有 n 个非负整数, 没有重复元素, 设计一个时间上尽可能高效的算法求是否存在两个数和为 $k(k>0)$, 如果存在则返回 1, 不存在则返回 0。例: $A=\{1, 2, 3, 7, 8, 11, 15\}$, $k=18$, 则 $7+11=18$, 返回 1。 (1) 给出算法的基本设计思想。
(2) 根据设计思想, 采用 C、C++或 Java 语言描述算法, 关键之处给出注释。 (3) 说明你所设计算法的时间复杂度和空间复杂度。

【习题 2-9】 (13 分) 已知升序数组 (无重复元素) A、B 都含有 n 个整数, 设计一个时间上尽可能高效的算法求新的数组 C 使得 C 中只存在 A、B 中都包含的元素, 输出数组 C。

(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

【习题 2-10】 (13 分) 将一个数组最开始的若干元素搬到数组的末尾，称之为数组的旋转。已知数组 A 为某个有序数组旋转后的结果（无重复值），设计一个时间上尽可能高效的算法求该旋转数组的最小元素。例：数组 $A = \{3, 4, 5, 6, 1, 2\}$ 为有序数组 $\{1, 2, 3, 4, 5, 6\}$ 的一个旋转数组，该数组的最小值为 1。（1）给出算法的基本设计思想。（2）根据设计思想，采用 C、C++ 或 Java 语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。

【习题 2-11】 (15 分) 已知一个整数升序序列 $A = \{a_0, a_1, \dots, a_{n-1}\}$, 其中 $0 \leq a_i < n$ ($0 \leq i < n$)。连续子序列是从原序列中选取的一段连续的元素构成的序列 (至少包含一个元素), 如 $\{a_0, a_1\}$ 、 $\{a_0\}$ 都是连续子序列。设计一个在时间上尽可能高效的算法, 输出元素之和 $\leq k$ 的连续子序列个数。例: $A = \{1, 2, 3, 4, 5\}$, $n = 5$, $k = 6$, 则符合要求的连续子序列为 $\{1\}$ 、 $\{2\}$ 、 $\{3\}$ 、 $\{4\}$ 、 $\{5\}$ 、 $\{1, 2\}$ 、 $\{2, 3\}$ 、 $\{1, 2, 3\}$, 共有 8 个, 返回 8。 (1) 给出算法的基本设计思想。 (2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。 (3) 说明你所设计算法的时间复杂度和空间复杂度。

【习题 2-12】 (13 分) 已知两个非负整数序列 $A = \{a_0, a_1, \dots, a_{n-1}\}$ 、 $B = \{b_0, b_1, \dots, b_{n-1}\}$ ，求出有多少个数对 $\{a_i, b_j\}$ 使得 $a_i + b_j \leq 5n$ ，设计一个在时间上尽可能高效的算法，有多少个这样的数对。例： $A = \langle 2, 10, 8 \rangle$ ， $B = \langle 8, 12, 1 \rangle$ ， $n = 3$ ， $\langle 2, 8 \rangle$ 、 $\langle 2, 12 \rangle$ 、 $\langle 2, 1 \rangle$ 、 $\langle 10, 1 \rangle$ 、 $\langle 8, 1 \rangle$ 共满足要求的数对，返回 5。（1）给出算法的基本设计思想。（2）根据设计思想，采用 C、C++ 或 Java 语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。

【* 习题 2-13】 (13 分) 已知数组 A 中有 n 个非负元素, 每个元素 $A[i] < 2n$ 。请设计一个时间上尽可能高效的算法, 求出数组 A 中第 k 小的元素。例: $A = \{1, 2, 1, 2, 3, 3\}$, $n = 6$, $k = 3$, 按照升序得到 $\{1, 1, 2, 2, 3, 3\}$, 所以第 k 小的元素是 2 (第 3、4 大元素都是 2), 输出 2。 (1) 给出算法的基本设计思想。 (2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。 (3) 说明你所设计算法的时间复杂度和空间复杂度。

【习题 2-14】 (13 分) 已知数组 A 中有 n 个元素 (可能重复), 每个元素 $2n \leq A[i] < 3n$ 。请设计一个尽可能高效的算法, 从小到大输出 A 数组中的元素 (重复元素只输出一次)。

例: $A = \{13, 17, 16, 14, 13, 13\}$, $n = 6$, 出现的元素是 13、17、16、14, 所以从小到大依次输出: 13、14、16、17。

(1) 给出算法的基本设计思想。 (2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。 (3) 说明你所设计算法的时间复杂度和空间复杂度。

【习题 2-15】 (15 分) 已知一个整数序列 $A = (a_0, a_1, \dots, a_{n-1})$, 其中 $0 \leq a_i < n$ ($0 \leq i < n$)。从数组中选择一些数, 要求选择的数中任意两数差的绝对值 $\leq k$ 。设计一个在时间上尽可能高效的算法输出最多可能选择的元素个数。要求: (1) 给出算法的基本设计思想。
(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

【习题 2-16】 (15 分) 在数组中, 某个数字减去它右边的数字得到一个数对之差。求所有数对之差的最大值。例如, 在数组 {2, 4, 1, 16, 7, 5, 11, 9} 中, 数对之差的最大值是 11, 是 16 减去 5 的结果。设计一个在时间上尽可能高效的算法输出所有数对之差的最大值。(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间和空间复杂度。

【习题 2-17】 (15 分) 已知一个整数升序序列 $A = (a_0, a_1, \dots, a_{n-1})$ 。设计一个尽可能高效的算法返回整数 x 出现的次数。函数原型为：int countOccurrences(int A[], int n, int x)。

(1) 给出算法的基本设计思想。(2) 根据设计思想，采用 C 语言描述算法，关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

2.5.2 真题

【2010 年 42 题】 (13 分) 设将 n ($n > 1$) 个整数存放于一维数组 R 中。试设计一个在时间和空间两方面都尽可能高效的算法。将 R 中保存的序列循环左移 p ($0 < p < n$) 个位置, 即将 R 中的数据由 $(X_0, X_1, \dots, X_{n-1})$ 变换为 $(X_p, X_{p+1}, \dots, X_{n-1}, X_0, X_1, \dots, X_{p-1})$ 。要求: (1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

【2011 年 42 题】 (15 分) 一个长度为 L ($L \geq 1$) 的升序序列 S , 处在第 $\lfloor L/2 \rfloor$ 个位置的数称为 S 的中位数。例如, 若序列 $S_1 = (11, 13, 15, 17, 19)$, 则 S_1 的中位数是 15, 两个序列的中位数是含它们所有元素的升序序列的中位数。例如, 若 $S_2 = (2, 4, 6, 8, 20)$, 则 S_1 和 S_2 的中位数是 11。现在有两个等长升序序列 A 和 B , 试设计一个在时间和空间两方面都尽可能高效的算法, 找出两个序列 A 和 B 的中位数。要求: (1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

【2013 年 41 题】 (13 分) 已知一个整数序列 $A = (a_0, a_1, \dots, a_{n-1})$, 其中 $0 \leq a_i < n$ ($0 \leq i < n$)。若存在 $a_{p_1} = a_{p_2} = \dots = a_{p_m} = x$ 且 $m > n/2$ ($0 \leq p_k < n, 1 \leq k \leq m$) , 则称 x 为 A 的主元素。例如 $A = (0, 5, 5, 3, 5, 7, 5, 5)$, 则 5 为主元素; 又如 $A = (0, 5, 5, 3, 5, 1, 5, 7)$, 则 A 中没有主元素。假设 A 中的 n 个元素保存在一个一维数组中, 请设计一个尽可能高效的算法, 找出 A 的主元素。若存在主元素, 则输出该元素; 否则输出 -1。要求: (1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

【2016 年 43 题】 (13 分) 已知由 n ($n \geq 2$) 个正整数构成的一个集合 $A = \{a_k \mid 0 \leq k < n\}$, 将其划分为两个不相交的子集 A_1 和 A_2 , 元素个数分别是 n_1 和 n_2 , A_1 和 A_2 中元素之和分别为 S_1 和 S_2 。设计一个尽可能高效的划分算法, 满足 $|n_1 - n_2|$ 最小且 $|S_1 - S_2|$ 最大。要求: (1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的平均时间复杂度和空间复杂度。

【2018 年 41 题】 (13 分) 给定一个含 n ($n \geq 1$) 个整数的数组, 请设计一个在时间上尽可能高效的算法, 找出数组中未出现的最小正整数。例如, 数组 $\{-5, 3, 2, 3\}$ 中未出现的最小正整数是 1; 数组 $\{1, 2, 3\}$ 中未出现的最小正整数是 4。要求: (1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

【2020 年 41 题】 (13 分) 定义三元组 (a, b, c) (a, b, c 均为正数) 的距离 $D = |a - b| + |b - c| + |c - a|$ 。给定 3 个非空整数集合 S_1 、 S_2 和 S_3 ，按升序分别存储在 3 个数组中。请设计一个尽可能高效的算法，计算并输出所有可能的三元组 (a, b, c) ($a \in S_1, b \in S_2, c \in S_3$) 中的最小距离。例如 $S_1 = \{-1, 0, 9\}$, $S_2 = \{-25, -10, 10, 11\}$, $S_3 = \{2, 9, 17, 30, 41\}$ ，则最小距离为 2，相应的三元组为 $(9, 10, 9)$ 。要求：(1) 给出算法的基本设计思想。(2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

【2025 年 41 题】 (13 分) 有两个长度均为 n 的一维整型数组 $A[n]$ 、 $res[n]$ ，计算 $A[i]$ 与 $A[j]$ ($0 \leq i \leq j \leq n-1$) 乘积的最大值，并将其保存到 $res[i]$ 中。若 $A[] = \{1, 4, -9, 6\}$ ，则得到 $res[] = \{6, 24, 81, 36\}$ 。现给定数组 A ，请设计时间空间上尽可能高效的算法 $CalMulMax$ ，求 res 中各元素的值。函数原型为：`void CalMulMax(int A[], int res[], int n)`。(1) 给出算法的基本思想。(4 分) (2) 用 C/C++ 描述算法关键之处给出注释。(7 分) (3) 说明时间、空间复杂度。(2 分)

2.6 章节答案

2.6.1 习题

* 习题 2-1

(13 分) 已知升序数组 A 含有 n 个整数, 求是否存在值为 x 的元素, 存在返回数组下标, 不存在则返回 -1。设计一个在时间上尽可能高效的算法实现该功能。例: $A=\{1, 2, 3, 7, 8, 11, 15\}$, $n=7$, $x=8$, 则返回数组下标为 4。(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	枚举	$O(n)$	$O(1)$	11
优化解	折半查找	$O(\log n)$	$O(1)$	13

【解法1】暴力解：枚举

(1) 遍历一趟数组, 依次判断每个元素是否为 x , 是则返回数组下标, 如果遍历完还没找到则返回 -1。(2) 伪代码如下:

```
int ans(int A[], int n, int x)
{
    for (int i=0; i<n; i++)
        if (A[i]==x) return i;
    return -1;
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(1)$

【解法2】优化解：折半查找

(1) 以 0、 $n-1$ 分别作为左、右边界对升序数组 A 折半查找, 查找到 x 则返回数组下标, 查找失败则返回 -1。(2) 伪代码如下:

```
int B_Search(int A[], int L, int R, int x)
{
    int mid;
    while (L<=R)
```

```

{
    mid=(L+R)/2;           // mid 取中间数，向下取整
    if (x==A[mid]) return mid; // 命中，返回 mid
    if (x<A[mid]) R=mid-1;   // 取左一半
    else L=mid+1;           // 取右一半
}
return -1;                 // 查找失败
}

```

(3) 时间复杂度: $O(\log n)$ 空间复杂度: $O(1)$

* 习题 2-2

(13 分) 已知两个整数升序数组 A 和 B，他们的元素数分别为 n 和 m，请将他们合并成一个新的升序数组 C，并输出 C 数组。设计一个尽可能高效的算法实现该功能。例：A = {1, 2, 7}, n = 3, B = {3, 4, 5, 10}, m = 4, 则 C = {1, 2, 3, 4, 5, 7, 10}。 (1) 给出算法的基本设计思想。 (2) 根据设计思想，采用 C、C++或 Java 语言描述算法，关键之处给出注释。 (3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	快速排序	$O((m+n)\log(m+n))$	$O(m+n)$	11
优化解	多指针法	$O(m+n)$	$O(m+n)$	13

【解法1】暴力解：快速排序

(1) 先开辟一个足够大的新数组 C，将数组 A 和数组 B 的所有元素依次放入数组 C 中，然后对整个 C 数组进行快速排序，使其成为升序数组。 (2) 伪代码如下：

```

// 快速排序函数
void Qsort(int A[], int L, int R)
{
    if (L >= R) return;           // 区间≤1，返回
    int i = L, j = R;
    int pivot = A[L];             // 选首元素为基准
    // 不断交换左边比 pivot 大的数和右边比 pivot 小的数
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--; // 从右找第一个 < pivot 的数
        while (i < j && A[i] <= pivot) i++; // 从左找第一个 > pivot 的数
        if (i < j) swap(A[i], A[j]);
    }
    // 将基准值 A[L] 与 A[i] 交换
    swap(A[L], A[i]);
    // 递归处理左右两个子区间
    Qsort(A, L, i - 1);
    Qsort(A, i, R);
}

```

```

    Qsort(A, i + 1, R);
}

int ans(int A[], int n, int B[], int m)
{
    int C[m + n];
    for (int i = 0; i < n; i++)
        C[i] = A[i];
    for (int i = 0; i < m; i++)
        C[n + i] = B[i];
    Qsort(C, 0, m + n - 1);
    输出数组 C
}

```

(3) 时间复杂度: $O((m+n)\log(m+n))$ 空间复杂度: $O(\log(m+n))$

【解法2】优化解：多指针法

(1) 设置三个指针 i 、 j 、 k 分别指向数组 A 、 B 、 C 的起始位置。比较 $A[i]$ 和 $B[j]$ 的值，将较小的元素放入 $C[k]$ ，并将对应的指针 (i 或 j) 和 k 都后移一位。重复此过程，直到其中一个数组遍历完毕。最后，将另一个数组中剩余的元素直接复制到 C 数组的末尾。(2) 伪代码如下：

```

void Merge(int A[], int n, int B[], int m)
{
    int C[m + n];
    int i = 0, j = 0, k = 0;
    // 比较并合并
    while (i < n && j < m)
    {
        if (A[i] < B[j]) C[k++] = A[i++];
        else C[k++] = B[j++];
    }
    // 复制剩余元素
    while (i < n) C[k++] = A[i++];
    while (j < m) C[k++] = B[j++];
    输出数组 C
}

```

3) 时间复杂度: $O(m+n)$ 空间复杂度: $O(m+n)$

* 习题 2-3

(15 分) 已知数组 A 中有 n 个非负元素，每个元素绝对值 $< n$ ，其中只有一个数重复出现过，请输出这个数，否则输出-1。设计一个时间上尽可能高效的算法实现该功能。例： $n = 6$ ， $A = \{1, 2, 5, 2, 4, 3\}$ ，则 2 重复出现，输出 2。(1) 给出算法的基本设计思想。(2) 根据设计思想，采用 C、C++ 或 Java 语言描述算法，关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	枚举法	$O(n^2)$	$O(1)$	11
暴力解	快速排序	$O(n\log n)$	$O(\log n)$	13
优化解	散列表	$O(n)$	$O(n)$	15

【解法1】暴力解：枚举法

(1) 两层循环*i*和*j*枚举，判断*A[i]*和*A[j]*是否相等 ($i < j$)，如果相等则说明这个数重复出现过，输出*A[i]*。如果没发现相等的则输出-1。(2) 伪代码如下：

```
int ans(int A[], int n)
{
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (A[i] == A[j]) return A[i];
    return -1;
}
```

(3) 时间复杂度： $O(n^2)$ 空间复杂度： $O(1)$

【解法2】暴力解：快速排序

(1) 先将数组 *A* 快速排序后，在排序好的数组中查找是否有重复元素，然后遍历*C*数组判断如果某个元素和他前一个元素相等说明有重复元素，输出。如果最终都没有发现发现有重复元素，则输出-1。(2) 伪代码如下：

```
// 快速排序函数
void Qsort(int A[], int L, int R)
{
    if (L >= R) return; // 区间≤1, 返回
    int i = L, j = R;
    int pivot = A[L]; // 选首元素为基准
    // 不断交换左边比 pivot 大的数和右边比 pivot 小的数
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--; // 从右找第一个 < pivot 的数
        while (i < j && A[i] <= pivot) i++; // 从左找第一个 > pivot 的数
        if (i < j) swap(A[i], A[j]);
    }
    // 将基准值 A[L] 与 A[i] 交换
    swap(A[L], A[i]);
    // 递归处理左右两个子区间
    Qsort(A, L, i - 1);
    Qsort(A, i + 1, R);
}
```

```

}
int ans(int A[], int n)
{
    Qsort(A, 0, n - 1);
    for (int i = 1; i < n; i++)
        if (A[i] == A[i - 1]) return A[i];
    return -1;
}

```

(3) 时间复杂度: $O(n\log n)$ 空间复杂度: $O(\log n)$

【解法3】优化解：散列表

(1) 设置count数组表示某个数出现的次数，循环遍历数组A并更新count数组，访问A[i]时如果发现count[A[i]]已经>0说明之前A[i]已经出现过，输出A[i]。如果最终都没有发现有重复元素，则输出-1。(2) 伪代码如下：

```

int ans(int A[], int n)
{
    int count[n] = {0};
    for (int i = 0; i < n; i++)
    {
        if (count[A[i]] > 0) return A[i];
        count[A[i]]++;
    }
    return -1;
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(n)$

习题 2-4

(15 分) 已知整数数组 A 含有 n 个不相等的整数 (n 为奇数，且数组 A 中每个元素的绝对值 $\leq n$)。设计一个时间上尽可能高效的算法，求数组 A 的中位数。例：n=5, A={-3, 0, 4, -2, 3}，则中位数为 0。(1) 给出算法的基本设计思想。(2) 根据设计思想，采用 C、C++ 或 Java 语言描述算法，关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	枚举法	$O(n^2)$	$O(1)$	11
暴力解	快速排序	$O(n\log n)$	$O(\log n)$	13
优化解	散列表	$O(n)$	$O(n)$	15
优化解	多指针法	$O(n)$	$O(\log n)$	15

【解法1】暴力解：枚举法

- (1) 对每个元素 $A[i]$ ，计算比它小的元素个数 $count$ ，如果恰好为 $n/2$ ，则为中位数。
(2) 伪代码如下：

```
int ans(int A[], int n)
{
    for (int i = 0; i < n; i++)
    {
        int count = 0;
        for (int j = 0; j < n; j++)
            if (A[j] < A[i]) count++;
        if (count == n / 2) return A[i];
    }
    return -1;
}
```

- (3) 时间复杂度： $O(n^2)$ 空间复杂度： $O(1)$

【解法2】暴力解：快速排序

- (1) 将数组 A 快速排序，输出排序后数组 A 最中间的数 $A[n/2]$ 即是中位数。 (2) 伪代码如下：

```
// 快速排序函数
void Qsort(int A[], int L, int R)
{
    if (L >= R) return; // 区间≤1，返回
    int i = L, j = R;
    int pivot = A[L]; // 选首元素为基准
    // 不断交换左边比 pivot 大的数和右边比 pivot 小的数
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--; // 从右找第一个 < pivot 的数
        while (i < j && A[i] <= pivot) i++; // 从左找第一个 > pivot 的数
        if (i < j) swap(A[i], A[j]);
    }
    // 将基准值 A[L] 与 A[i] 交换
    swap(A[L], A[i]);
    // 递归处理左右两个子区间
    Qsort(A, L, i - 1);
    Qsort(A, i + 1, R);
}

int ans(int A[], int n)
{
    Qsort(A, 0, n - 1);
    return A[n/2];
}
```

(3) 时间复杂度: $O(n\log n)$ 空间复杂度: $O(\log n)$

【解法3】优化解：散列表

(1) 设置散列表 $\text{count}[2n+1]$, 散列函数 $f(x) = x - n$, 其中 $\text{count}[i]$ 表示 $i-n$ 这个数出现的次数, 循环遍历数组 A 并更新 count 数组。之后 i 从 0 遍历到 $2n$, 每次将 $\text{count}[i]$ 累加, 当累加变量 $\geq n/2$ 时输出 $i-n$ 并退出程序。(2) 伪代码如下:

```
int ans(int A[], int n)
{
    int count[2 * n + 1] = {0}; // 散列表记录x-n出现的次数
    for (int i = 0; i < n; i++)
        count[A[i] + n]++;
    int sum = 0; // 累计到目前为止已经是第几小的数了
    for (int i = 0; i < 2*n+1; i++)
    {
        sum = sum + count[i];
        if (sum > n/2)
            return i-n;
    }
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(n)$

【解法4】优化解：多指针法

(1) 对无序数组使用多指针法, 类似快速排序, 但是在递归调用区间 $[L, i-1]$ 和 $[i+1, R]$ 时, 因为中位数只可能出现在下标包含 $n/2$ 的区间中, 所以增加一个判断, 只调用其中一个区间。

(2) 伪代码如下:

```
void Qsort_Select(int A[], int L, int R)
{
    if (L >= R) return; // 区间≤1, 返回
    int i = L, j = R;
    int pivot = A[L]; // 选首元素为基准
    // 不断交换左边比 pivot 大的数和右边比 pivot 小的数
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--; // 从右找第一个 < pivot 的数
        while (i < j && A[i] <= pivot) i++; // 从左找第一个 > pivot 的数
        if (i < j) swap(A[i], A[j]);
    }
    // 将基准值 A[L] 与 A[i] 交换
    swap(A[L], A[i]);
    if (L <= n/2 && i-1 >= n/2)
        Quick_Select(A, L, i-1, n); // 左区间包括中位数
    else
        Quick_Select(A, i+1, R, n); // 右区间包括中位数
}
```

```

}
int ans(int A[], int n)
{
    Quick_Select(A, 0, n-1, n);
    return A[n/2];
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(\log n)$

习题 2-5

(15 分) 已知数组 A 含有 n 个整数, 现在要将小于 k 的元素放在数组右侧, 大于等于 k 的元素放在左侧, 设计一个时间上尽可能高效的算法实现这个功能。例: $n = 7$, $A = \{4, 5, 9, 3, 7, 10, 1\}$, $k = 6$, 则处理之后的数组可以为 $\{10, 7, 9, 3, 5, 4, 1\}$, 其中 10、7、9 比 6 大, 在数组左侧; 而 3、5、4、1 小于等于 6, 在数组右侧。(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	快速排序	$O(n \log n)$	$O(\log n)$	13
优化解	多指针法	$O(n)$	$O(1)$	15

【解法1】暴力解: 快速排序

(1) 将数组 A 快速排序得到升序数组, 然后将数组 A 翻转并输出即可得到降序数组, 满足小于 k 的元素在右侧, 大于 k 的元素在左侧。(2) 伪代码如下:

```

// 快速排序函数
void Qsort(int A[], int L, int R)
{
    if (L >= R) return; // 区间≤1, 返回
    int i = L, j = R;
    int pivot = A[L]; // 选首元素为基准
    // 不断交换左边比 pivot 大的数和右边比 pivot 小的数
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--; // 从右找第一个 < pivot 的数
        while (i < j && A[i] <= pivot) i++; // 从左找第一个 > pivot 的数
        if (i < j) swap(A[i], A[j]);
    }
    // 将基准值 A[L] 与 A[i] 交换
    swap(A[L], A[i]);
    // 递归处理左右两个子区间
    Qsort(A, L, i - 1);
}

```

```

    Qsort(A, i + 1, R);
}
int ans(int A[], int n, int k)
{
    Qsort(A, 0, n-1);
    for (int i=0; i<n/2; i++)
        swap(A[i], A[n-i-1]);
    输出数组 A
}

```

(3) 时间复杂度: $O(n\log n)$ 空间复杂度: $O(\log n)$

【解法2】优化解：多指针法

(1) 该方法类似快速排序，只需要把左侧小于 k 的元素和右侧大于 k 的元素不断交换即可。左右分别设置指针 i 、 j ， j 从右往左移动直到 $A[j]>k$ ， i 从左往右移动直到 $A[i]<k$ ，交换 $A[i]$ 和 $A[j]$ 。不断重复该过程直到 i 和 j 相遇。(2) 伪代码如下：

```

void Multiointers(int A[], int L, int R, int k)
{
    if (L>=R) return;          // 区间≤1, 退出
    int pivot, i=L, j=R;       // i,j 为左右指针
    pivot=k;
    while (i<j)
    {
        while (i<j && A[j]<=pivot) j--; // 右边≤则不处理
        while (i<j && A[i]>=pivot) i++; // 左边≥则不处理
        if (i<j) swap(A[i], A[j]);    // A[j]>k, A[i]<k
    }
}

int ans(int A[], int n, int k)
{
    Multiointers(A, 0, n-1, k);
    按顺序输出A数组
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(1)$

习题 2-6

(13 分) 有一个数组 A 由 n 个互不相同的正整数组成 (n 为奇数)，该序列中的数先减后增，中位数是最小值，输入一个整数 key 。请设计一个时间和空间上尽可能高效的算法，判断 key 是否在该数组中，如果是则返回数组下标（下标从 0 开始），否则返回 -1。例如对应 $A = \{10, 7, 5, 2, 6, 13, 15\}$ ， $key = 5$ 时返回下标 2，若 $key = 8$ 则返回 -1。(1) 给出算法的基本设计思想。(2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	枚举	$O(n)$	$O(1)$	10
优化解	折半查找	$O(\log n)$	$O(1)$	13

【解法1】暴力解：枚举，扫描一遍数组

(1) 循环 i 扫描一次数组，如果 $A[i] == \text{key}$ 则返回 i ，如果没找到则返回-1。 (2) 伪代码如下：

```
int ans(int A[], int n, int key)
{
    for (int i = 0; i < n; i++)
        if (A[i] == key) return i;
    return -1;
}
```

(3) 时间复杂度： $O(n)$ 空间复杂度： $O(1)$

【解法2】优化解：折半查找两次

(1) $A[0 \dots n/2]$ 递减，前半段折半查找一次； $A[n/2 \dots n-1]$ 递增，后半段再折半查找一次。（注意：因为一个是降序一个是升序，这两次写法会有不同，也可以合成一种写法） (2) 伪代码如下：

```
// 在升序数组中查找
int B_Search_up(int A[], int L, int R, int x)
{
    int mid;
    while (L <= R) // 区间非空则继续
    {
        mid = (R + L) / 2; // 防溢出
        if (A[mid] == x)
            return mid; // 找到了，立即返回
        else
            if (x < A[mid]) R = mid - 1;
            else L = mid + 1; // x 在右边，mid 被排除
    }
    return -1; // 循环结束，说明没找到
}

// 在降序数组中查找
int B_Search_down(int A[], int L, int R, int x)
{
    int mid;
    while (L <= R) // 区间非空则继续
    {
```

```

mid = (R + L) / 2;           // 防溢出
if (A[mid] == x)
    return mid;             // 找到了，立即返回
else
    if (x > A[mid]) R = mid - 1;
    else L = mid + 1;       // x 在右边，mid 被排除
}
return -1;                  // 循环结束，说明没找到
}

int ans(int A[], int n, int key)
{
    int i;
    i=B_Search_down(A, 0, n/2-1, key); // 折半查找降序序列
    if (i!=-1)
        return i;             // 折半查找降序成功
    i=B_Search_up(A, n/2, n-1, key);   // 折半查找升序序列
    return i;
}

```

(3) 时间复杂度: $O(\log n)$ 空间复杂度: $O(1)$

习题 2-7

(13 分) 定义距离 $d = |x - y|$, 给定 2 个长度为 n 的升序数组 a 、 b , x 是数组 a 中的某个元素, y 是数组 b 中的某个元素。请设计一个尽可能高效的算法, 计算并输出所有可能的最小距离 d 。例如 数组 $a = \{-16, -8, 5, 8, 13\}$, $b = \{-2, 0, 2, 6, 10\}$, 则最小距离为 1, 相应的 $x=5$, $y=6$, $d = |5 - 6| = 1$ 。(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举	$O(n^2)$	$O(1)$	10
优化解	多指针法	$O(n)$	$O(1)$	13

【解法1】暴力解：枚举

(1) 使用双指针 i, j 分别指向 a, b , 同步遍历。每次移动指向较小元素的指针, 才有可能获得更小的距离。(2) 伪代码如下:

```

int ans(int a[], int b[], int n)
{
    int D_min = INT_MAX; // D_min初值为最大
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            D_min = min(D_min, abs(a[i] - b[j]));
}

```

```
return D_min;
}
```

(3) 时间复杂度: $O(n^2)$ 空间复杂度: $O(1)$

【解法2】优化解：多指针法

(1) 使用双指针 i, j 分别指向 a, b ，同步遍历。每次移动指向较小元素的指针，才有可能获得更小的距离。(2) 伪代码如下：

```
int ans(int a[], int b[], int n)
{
    int D_min = INT_MAX; // D_min初值为最大
    int i = 0, j = 0;
    while (i < n && j < n)
    {
        D_min = min(D_min, abs(a[i] - b[j]));
        if (a[i] < b[j]) i++;
        else j++;
    }
    return D_min;
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(1)$

习题 2-8

(15 分) 已知升序数组 A 含有 n 个非负整数，没有重复元素，设计一个时间上尽可能高效的算法求是否存在两个数和为 $k(k>0)$ ，如果存在则返回 1，不存在则返回 0。例： $A=\{1, 2, 3, 7, 8, 11, 15\}$ ， $k=18$ ，则 $7+11=18$ ，返回 1。(1) 给出算法的基本设计思想。(2) 根据设计思想，采用 C、C++或 Java 语言描述算法，关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举	$O(n^2)$	$O(1)$	11
优化解	折半查找	$O(n\log n)$	$O(1)$	13
优化解	散列表	$O(n)$	$O(k)$	15
优化解	多指针法	$O(n)$	$O(1)$	15

注意 $A[i] + A[j] == k$ 时还需要保证 i 和 j 不相等，这个条件一定要注意。

【解法1】暴力解：枚举

(1) 二重循环遍历数组，当*i*和*j*不相等且 $A[i]+A[j] == k$ 时返回1，如果扫描完都没找到则返回0。(2) 伪代码如下：

```
int ans(int A[], int n, int k)
{
    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            if (i!=j && A[i]+A[j]==k) return 1;
    return 0;
}
```

(3) 时间复杂度： $O(n^2)$ 空间复杂度： $O(1)$

【解法2】优化解：折半查找

(1) 循环遍历数组，如果 $A[i] \leq k$ 则在整个数组中折半查找 $k-A[i]$ ，若折半查找能够找到 $k-A[i]$ 则返回1。如果遍历完整个链表都无法找到则返回0。(2) 伪代码如下：

```
// 折半查找
int B_Search(int A[], int L, int R, int x)
{
    int mid;
    while (L <= R)
    {
        mid = (R + L) / 2;
        if (A[mid] == x)
            return mid;
        else
            if (x < A[mid]) R = mid - 1;
            else L = mid + 1;
    }
    return -1;
}

int ans(int A[], int n, int k)
{
    for (int i = 0; i < n; i++)
    {
        int j = B_Search(A, 0, n-1, k-A[i]);
        if (i != j) return 1;    // 需要保证 i 和 j 不相等
    }
    return 0;
}
```

(3) 时间复杂度： $O(n \log n)$ 空间复杂度： $O(1)$

【解法3】优化解：散列表

(1) 设置 count 数组表示某个数出现的次数 (数组下标 0~k), 循环遍历数组 A 并更新 count 数组, 遍历 i 访问 A[i], 如果 A[i] ≤ k 则判断 count[k-A[i]] 已经 > 0 说明之前 k-A[i] 已经出现过, 说明 A[i] 和 k-A[i] 都已经找到, 输出 1。如果最终都没有找到, 则输出 0。(2) 伪代码如下:

```
int ans(int A[], int n, int k)
{
    int count[k+1] = {0};           // 散列表记录某个数出现的次数
    for (int i = 0; i < n; i++)
    {
        if (A[i] > k) break;
        if (count[k-A[i]] > 0) return 1;    // 找到A[i]和k-A[i]
        count[A[i]]++;
    }
    return 0;
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(k)$

【解法4】优化解: 多指针法

(1) 设置变量 i、j 存储数组下标, 初始 i = 0, j = n-1, 此前往后循环遍历 i, 对于每个 i 从上次的 j 出发往左找直到 A[i] + A[j] ≤ k, 对于这次 i 的取值只可能是当前的 j 能满足 A[i] + A[j] 等于 k, 若能够找到 A[i] + A[j] 等于 k 则返回 1, 不断重复这个过程直到 i 和 j 相遇, 如果一直找不到就返回 0。(2) 伪代码如下:

```
int ans(int A[], int n, int k)
{
    int j = n-1;
    for (int i = 0; i < n; i++)
    {
        while (i < j && A[i]+A[j]>k) j--;
        if (i >= j) return 0;
        if (A[i] + A[j] == k) return 1;
    }
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(1)$

习题 2-9

(13 分) 已知升序数组 (无重复元素) A、B 都含有 n 个整数, 设计一个时间上尽可能高效的算法求新的数组 C 使得 C 中只存在 A、B 中都包含的元素, 输出数组 C。(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	枚举法	$O(n^2)$	$O(n)$	11
优化解	多指针法	$O(n)$	$O(n)$	13

【解法1】暴力解：枚举法 1) 将数组A中的每个数和数组B中的每个数比较，如果相同就存入数组C，最后输出数组C。2) 伪代码如下：

```
int ans(int A[], int B[], int n)
{
    int C[n];
    int k = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (A[i] == B[j]) C[k++] = A[i];
    输出数组 C
}
```

(3) 时间复杂度： $O(n^2)$ 空间复杂度： $O(n)$

【解法2】优化解：多指针法

(1) 使用双指针 i, j 同步遍历 A, B。若 $A[i] == B[j]$ 则为交集元素；否则移动指向较小元素的指针。(2) 伪代码如下：

```
int ans(int A[], int B[], int n)
{
    int C[n];
    int i = 0, j = 0, k = 0; // i、j作为遍历A、B的下标
    while (i < n && j < n) // 直到有一个数组遍历完
    {
        if (A[i] == B[j]) // 数组A和B中都存在的元素存入C中
        {
            C[k++] = A[i++];
            j++;
        }
        if (A[i] < B[j]) i++; // A[i]更小，i指针后移
        if (A[i] > B[j]) j++; // B[j]更小，j指针后移
    }
    输出数组C
}
```

(3) 时间复杂度： $O(n)$ 空间复杂度： $O(n)$

习题 2-10

(13 分) 将一个数组最开始的若干元素搬到数组的末尾，称之为数组的旋转。已知数组 A 为某个有序数组旋转后的结果（无重复值），设计一个时间上尽可能高效的算法求该旋转数组的最小元素。例：数组 $A = \{3, 4, 5, 6, 1, 2\}$ 为有序数组 $\{1, 2, 3, 4, 5, 6\}$ 的一个旋转数组，该数组的最小值为 1。（1）给出算法的基本设计思想。（2）根据设计思想，采用 C、C++ 或 Java 语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	快速排序	$O(n\log n)$	$O(\log n)$	9
暴力解	枚举	$O(n)$	$O(1)$	11
优化解	折半查找	$O(\log n)$	$O(1)$	13

【解法1】暴力解：快速排序 1) 对数组A快速排序然后输出最小值A[0]。2) 伪代码如下：

```
// 快速排序
void Qsort(int A[], int L, int R)
{
    if (L >= R) return; // 区间≤1, 返回
    int i = L, j = R;
    int pivot = A[L]; // 选首元素为基准
    // 不断交换左边比 pivot 大的数和右边比 pivot 小的数
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--; // 从右找第一个 < pivot 的数
        while (i < j && A[i] <= pivot) i++; // 从左找第一个 > pivot 的数
        if (i < j) swap(A[i], A[j]);
    }
    // 将基准值 A[L] 与 A[i] 交换
    swap(A[L], A[i]);
    // 递归处理左右两个子区间
    Qsort(A, L, i - 1);
    Qsort(A, i + 1, R);
}

int ans(int A[], int n)
{
    Qsort(A, 0, n-1);
    return A[0];
}
```

(3) 时间复杂度： $O(n\log n)$ 空间复杂度： $O(\log n)$

【解法2】暴力解：枚举

(1) 遍历一趟数组，依次判断每个元素是否比当前已存在的最小值A_min更小，如果是则更新当前最小值。遍历完数组之后输出A_min。 (2) 伪代码如下：

```
int ans(int A[], int n)
{
    int A_min = A[0];
    for (int i = 1; i < n; i++)
        A_min = min(A[i], A_min);
    return A_min;
}
```

(3) 时间复杂度： $O(n)$ 空间复杂度： $O(1)$

【解法3】优化解：折半查找

(1) 对数组进行折半查找，如果 $L == R$ 或者 $A[mid]$ 比 $A[mid-1]$ 小那就说明已经找到最小值所在，返回 $A[mid]$ 。每次二分，判断 mid 所在区间是否包含最小值。由于数组整体是旋转有序数组：

- 若 $A[mid] \leq A[R]$ ，说明右半部分有序，最小值一定在左半部分（含 mid）。
 - 若 $A[mid] > A[R]$ ，说明最小值一定在右半部分（不含 mid）。
- (2) 伪代码如下：

```
int B_Search(int A[], int L, int R)
{
    int mid;
    while (L < R)
    {
        // 如果L>R则退出循环
        mid = (L+R)/2; // mid取中间数，向下取整
        if (A[mid] <= A[R]) R = mid; // 取左一半
        else L = mid + 1; // 取右一半
    }
    return L;
}

int ans(int A[], int n)
{
    int i = B_Search(A, 0, n-1);
    return A[i];
}
```

(3) 时间复杂度： $O(\log n)$ 空间复杂度： $O(1)$

因为本题比较元素不是要查找到 x 的位置，而是根据某一种条件来缩小区间，所以用到的是折半查找的写法 2。

习题 2-11

(15 分) 已知一个整数升序序列 $A = \{a_0, a_1, \dots, a_{n-1}\}$, 其中 $0 \leq a_i < n$ ($0 \leq i < n$)。连续子序列是从原序列中选取的一段连续的元素构成的序列 (至少包含一个元素), 如 $\{a_0, a_1\}$ 、 $\{a_0\}$ 都是连续子序列。设计一个在时间上尽可能高效的算法, 输出元素之和 $\leq k$ 的连续子序列个数。例: $A = \{1, 2, 3, 4, 5\}$, $n = 5$, $k = 6$, 则符合要求的连续子序列为 $\{1\}$ 、 $\{2\}$ 、 $\{3\}$ 、 $\{4\}$ 、 $\{5\}$ 、 $\{1, 2\}$ 、 $\{2, 3\}$ 、 $\{1, 2, 3\}$, 共有 8 个, 返回 8。 (1) 给出算法的基本设计思想。 (2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。 (3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	枚举	$O(n^2)$	$O(n)$	12
优化解	多指针法	$O(n)$	$O(1)$	15

【解法1】暴力解：枚举

(1) 两层循环枚举, 外层 i 枚举连续子序列的左边界, 内层 j 枚举右边界, 判断连续子序列 $A[i \sim j]$ 是否满足元素之和 $\leq k$ (我们可以预先设置一个数组 S , $S[i+1] = A[0] + A[1] + \dots + A[i]$, 只需要判断 $S[j+1] - S[i]$ 是否 $\leq k$ 即可), 如果是则计数器 $count$ 加 1, 最后输出 $count$ 。 (2) 伪代码如下:

```
int ans(int A[], int n, int k)
{
    int count = 0;
    int S[n + 1] = {0};
    for (int i = 0; i < n; i++) S[i + 1] = S[i] + A[i];
    for (int i = 0; i < n; i++)
        for (int j = i; j < n; j++)
            if (S[j + 1] - S[i] <= k) count++;
    return count;
}
```

(3) 时间复杂度: $O(n^2)$ 空间复杂度: $O(n)$

【解法2】优化解：多指针法

(1) 用指针 i, j 维护一个窗口, 保证其和 $current_sum \leq k$ 。 j 作为右边界向右移动, 同时累加窗口内的元素和。当窗口和超过 k 时, 移动左边界 i 来缩小窗口, 直到和再次满足条件。每当 j 移动到新位置, 都将 j 为右边界、且和小于等于 k 的子序列个数加到累加和 $count$ 里, 这些子序列的起点就是从 i 到 j , 个数为 $j-i+1$ 。最终得到的累加和 $count$ 就是所有符合条件的连续子序列总数。 (2) 伪代码如下:

```

int ans(int A[], int n, int k)
{
    int count = 0;
    int sum = 0;
    int i = 0;           // 窗口左边界
    for (int j = 0; j < n; j++) // j 是窗口右边界
    {
        sum += A[j];
        while (sum > k)
        {
            sum -= A[i];
            i++;
        }
        count += (j - i + 1);
    }
    return count;
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(1)$

习题 2-12

(13 分) 已知两个非负整数序列 $A = \{a_0, a_1, \dots, a_{n-1}\}$ 、 $B = \{b_0, b_1, \dots, b_{n-1}\}$, 求出有多少个数对 $\{a_i, b_j\}$ 使得 $a_i + b_j \leq 5n$, 设计一个在时间上尽可能高效的算法, 有多少个这样的数对。例: $A = \langle 2, 10, 8 \rangle$, $B = \langle 8, 12, 1 \rangle$, $n = 3$, $\langle 2, 8 \rangle$ 、 $\langle 2, 12 \rangle$ 、 $\langle 2, 1 \rangle$ 、 $\langle 10, 1 \rangle$ 、 $\langle 8, 1 \rangle$ 共满足要求的数对, 返回 5。 (1) 给出算法的基本设计思想。 (2) 根据设计思想, 采用 C、C++或 Java 语言描述算法, 关键之处给出注释。 (3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	枚举	$O(n^2)$	$O(1)$	11
暴力解	散列表	$O(n)$	$O(n)$	13

【解法1】暴力解: 枚举

(1) 两层循环 i 和 j 遍历数组 (保证 $i < j$) , 如果 $A[i] + B[j] \leq 5n$ 则计数器 $t+1$, 遍历完后返回 t 。 (2) 伪代码如下:

```

int ans(int A[], int B[], int n)
{
    int t = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)

```

```

        if (A[i] + B[j] <= 5 * n) t++;
    return t;
}

```

(3) 时间复杂度: $O(n^2)$ 空间复杂度: $O(1)$

【解法2】优化解：散列表

(1) 使用 count 数组保存A数组中每个数出现的次数，count[i] 表示i出现的次数，循环统计完 count 后计算出数组 S，S[i] 表示小于等于 i 的元素有多少个。然后循环扫描 B 数组看 $\leq 5n - B[j]$ 的数有多少个，加到 sum 中，循环完后返回 sum。 (2) 伪代码如下：

```

int ans(int A[], int B[], int n)
{
    int pair_count = 0;
    int count[5*n+1], S[5*n+1];    // S[i]表示≤i的数有多少个
    for (int i = 0; i <= 5*n; i++)
        count[i] = 0;              // 初始化count数组
    for (int i = 0; i < n; i++)
        if (A[i] <= 5*n)
            count[A[i]]++;
    S[0] = count[0];
    for (int i = 1; i <= 5*n; i++)
        S[i] = S[i-1] + count[i];  // 统计S数组
    for (int j = 0; j < n; j++)
        if (B[j] <= 5*n)
            pair_count = pair_count + S[5*n-B[j]];
    return pair_count;
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(n)$

* 习题 2-13

(13 分) 已知数组 A 中有 n 个非负元素，每个元素 $A[i] < 2n$ 。请设计一个时间上尽可能高效的算法，求出数组 A 中第 k 小的元素。例：A = {1, 2, 1, 2, 3, 3}，n = 6，k = 3，按照升序得到 {1, 1, 2, 2, 3, 3}，所以第 k 小的元素是 2（第 3、4 大元素都是 2），输出 2。（1）给出算法的基本设计思想。（2）根据设计思想，采用 C、C++ 或 Java 语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	快速排序	$O(n \log n)$	$O(\log n)$	11
优化解	散列表	$O(n)$	$O(n)$	13

【解法1】暴力解：快速排序

(1) 对数组快速排序，然后输出 $A[k-1]$ 。(2) 伪代码如下：

```
// 快速排序
void Qsort(int A[], int L, int R)
{
    if (L >= R) return; // 区间≤1, 返回
    int i = L, j = R;
    int pivot = A[L]; // 选首元素为基准
    // 不断交换左边比 pivot 大的数和右边比 pivot 小的数
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--; // 从右找第一个 < pivot 的数
        while (i < j && A[i] <= pivot) i++; // 从左找第一个 > pivot 的数
        if (i < j) swap(A[i], A[j]);
    }
    // 将基准值 A[L] 与 A[i] 交换
    swap(A[L], A[i]);
    // 递归处理左右两个子区间
    Qsort(A, L, i - 1);
    Qsort(A, i + 1, R);
}

int ans(int A[], int n, int k)
{
    Qsort(A, 0, n-1);
    return A[k-1];
}
```

(3) 快排选择枢值时会加上优化不会退化到最坏情况，代码中省略优化部分。时间复杂度： $O(n \log n)$ 空间复杂度： $O(\log n)$

【解法2】暴力解：散列表

(1) 用 count 数组保存每个元素出现的次数，count[i] 表示 i 出现的次数。统计完后再从 0 开始扫描一遍 count 数组，k 减去 count[i]，如果 $k \leq 0$ 了则输出 i。(2) 伪代码如下：

```
int ans(int A[], int n, int k)
{
    int count[2*n];
    for (int i = 0; i < 2*n; i++) count[i] = 0; // 初始化count数组
    for (int i = 0; i < n; i++) count[A[i]]++;
    for (int i = 0; i < 2*n; i++)
    {
        k = k - count[i];
        if (k <= 0) return i;
    }
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(n)$

习题 2-14

(13 分) 已知数组 A 中有 n 个元素 (可能重复), 每个元素 $2n \leq A[i] < 3n$ 。请设计一个尽可能高效的算法, 从小到大输出 A 数组中的元素 (重复元素只输出一次)。例: $A = \{13, 17, 16, 14, 13, 13\}$, $n = 6$, 出现的元素是 13、17、16、14, 所以从小到大依次输出: 13、14、16、17。(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	快速排序	$O(n \log n)$	$O(\log n)$	11
优化解	散列表	$O(n)$	$O(n)$	13

【解法1】暴力解: 快速排序

(1) 对数组快速排序, 然后从前往后遍历, 只输出和前一个元素不一样的元素, 这样就能够保证每个元素只输出一次。(2) 伪代码如下:

```
// 快速排序
void Qsort(int A[], int L, int R)
{
    if (L >= R) return; // 区间≤1, 返回
    int i = L, j = R;
    int pivot = A[L]; // 选首元素为基准
    // 不断交换左边比 pivot 大的数和右边比 pivot 小的数
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--; // 从右找第一个 < pivot 的数
        while (i < j && A[i] <= pivot) i++; // 从左找第一个 > pivot 的数
        if (i < j) swap(A[i], A[j]);
    }
    // 将基准值 A[L] 与 A[i] 交换
    swap(A[L], A[i]);
    // 递归处理左右两个子区间
    Qsort(A, L, i - 1);
    Qsort(A, i + 1, R);
}

void ans(int A[], int n)
{
    Qsort(A, 0, n - 1);
    for (int i = 0; i < n; i++)
        if (i == 0 || A[i] != A[i - 1]) // A[i] 这个元素第一次出现则输出
```

```
printf("%d ", A[i]);
}
```

(3) 时间复杂度: $O(n \log n)$ 空间复杂度: $O(\log n)$

【解法2】优化解：散列表

(1) count数组表示某个元素是否出现, count[x]表示x+2n出现的次数 ($0 \leq x < 2n$), 遍历A数组统计每个元素出现的次数, 即count[A[i]+2n]++。遍历完后依次扫描count数组判断有哪些元素出现过, 从小到大输出这些元素。(2) 伪代码如下:

```
void ans(int A[], int n)
{
    int count[n] = {0};
    for (int i = 0; i < n; i++)
        count[A[i]-2*n]++; // 统计 A 中每个元素出现次数
    for (int i = 0; i < n; i++)
        if (count[i] > 0)
            printf("%d", i+2*n); // 若 count[i]>0 说明 i+2n 出现过
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(n)$

习题 2-15

(15 分) 已知一个整数序列 $A = (a_0, a_1, \dots, a_{n-1})$, 其中 $0 \leq a_i < n$ ($0 \leq i < n$)。从数组中选择一些数, 要求选择的数中任意两数差的绝对值 $\leq k$ 。设计一个在时间上尽可能高效的算法输出最多可能选择的元素个数。要求: (1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	枚举	$O(n^2)$	$O(1)$	11
优化解	快速排序	$O(n \log n)$	$O(\log n)$	13
优化解	散列表	$O(n)$	$O(n)$	15

【解法1】暴力解：枚举，枚举最小元素

(1) 假设我们选择的数中最小的那个为 $A[i]$ ，那么我们可以选择的数范围就是 $A[i] \sim A[i] + k$ ，只要确定了 $A[i]$ 再访问一遍数组就能够算出有多少个可选元素 m ，所以使用两层循环，外层遍历数组，内层再遍历一次整个数组判断可选元素个数，选择最多可能的可选元素，输出 m 。

(2) 伪代码如下：

```
int ans(int A[], int n, int k)
{
    int m; // m是本次遍历选择元素数
    int m_max = 0; // m_max初值为0
    for (int i = 0; i < n; i++){ // A[i]为本次枚举最小元素
        m = 0; // 每次枚举时都需要清空m
        for (int j = 0; j < n; j++){
            if (A[j] >= A[i] && A[j] <= A[i] + k)
                m++; // A[j]处于选择范围则m++
            m_max = max(m_max, m); // 如果m大则更新m_max
        }
    }
    return m_max; // 输出最大m值
}
```

(3) 时间复杂度： $O(n^2)$ 空间复杂度： $O(1)$

【解法2】优化解：快速排序，然后数组指针后移

(1) 先快速排序将A数组变成升序，设置数组指针 i 和 j （初始为0），枚举 i 从 $0 \dots n-1$ ， j 从上一轮的 j 开始往后移动，如果 $A[j+1] \leq A[i] + k$ 说明 $A[j+1]$ 仍然处于 $A[i] \dots A[i] + k$ 这个范围，此时后移，直到 j 移动到最后一个元素说明已经不会有更大的 $m = j - i + 1$ 。（2）伪代码如下：

```
// 快速排序
void Qsort(int A[], int L, int R)
{
    if (L >= R) return; // 区间≤1，返回
    int i = L, j = R;
    int pivot = A[L]; // 选首元素为基准
    // 不断交换左边比 pivot 大的数和右边比 pivot 小的数
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--; // 从右找第一个 < pivot 的数
        while (i < j && A[i] <= pivot) i++; // 从左找第一个 > pivot 的数
        if (i < j) swap(A[i], A[j]);
    }
    // 将基准值 A[L] 与 A[i] 交换
    swap(A[L], A[i]);
    // 递归处理左右两个子区间
    Qsort(A, L, i - 1);
    Qsort(A, i + 1, R);
}

int ans(int A[], int n, int k)
{
    int m; // m是本次遍历选择元素数
    int m_max = 0; // m_max初值为0
    int i = 0, j = 0; // A[i~j]是选择的元素
```

```

Qsort(A, 0, n-1); // 快速排序
for (i = 0; i < n; i++){
    while (j+1<n && A[j+1]<=A[i]+k)
        j++; // j+1也在范围内, j后移
    m = j-i+1; // 求出m
    m_max = max(m_max, m); // 如果m大则更新m_max
    if (j == n-1) break; // j 已到末尾
}
return m_max; // 输出最大m值
}

```

(3) 时间复杂度: $O(n\log n)$ 空间复杂度: $O(\log n)$

快排的时间复杂度为 $O(n\log n)$, 指针后移的时间复杂度为 $O(n)$, 所以总时间复杂度是 $O(n\log n)$ 。

【解法3】优化解：散列表

(1) 数组中每个元素大小都在 $0 \dots n-1$, 可以使用数组 $B[n]$ 表示每个值出现的次数, 扫描一次整个数组统计每个数出现的次数, 然后 R 从 0 扫描到 $n-1$, 维持一个队列, 队头 L , 队尾 R , 保证这个队列长度不超过 k , m 表示队列中元素出现次数的总和, 此时可取数的范围就是 $[L \dots R]$, 一个新元素 R 入队后 ($m += B[R]$), 如果队列长度超过 k 了则将 L 出队 ($m -= B[L]$), 统计最大的 m 即最大队列中元素出现的次数和。 (2) 伪代码如下:

```

int ans(int A[], int n, int k)
{
    int m = 0; // 本次选择的元素数
    int m_max = 0; // m_max 初值为 0
    int B[maxn];
    memset(B, 0, sizeof(B));
    for (int i = 0; i < n; i++) B[A[i]]++; // A[i]出现次数+1
    int L = 0, R = 0;
    for (R = 0; R < n; R++)
    {
        m += B[R]; // m 加上 B[R]
        if (R-L > k)
        {
            m -= B[L]; // 如果范围差大于 k 则 L 右移
            L++; // [L+1,R], 减 B[L]
        }
        m_max = max(m_max, m); // 如果 m 大则更新 m_max
    }
    return m_max; // 输出最大m值
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(n)$

习题 2-16

(15 分) 在数组中, 某个数字减去它右边的数字得到一个数对之差。求所有数对之差的最大值。例如, 在数组 {2, 4, 1, 16, 7, 5, 11, 9} 中, 数对之差的最大值是 11, 是 16 减去 5 的结果。设计一个在时间上尽可能高效的算法输出所有数对之差的最大值。(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	枚举	$O(n^2)$	$O(1)$	13
优化解	预处理	$O(n)$	$O(1)$	15

【解法1】暴力解：枚举，枚举数对中的两个元素

(1) 使用两层循环, 外层遍历数组, 内层j再从i+1开始遍历一次整个数组(默认 $i < j$), $A[i]$ 是数对中的前一个元素, $A[j]$ 即是数对中的后一个元素, 求出 $m=A[i]$ 和 $A[j]$ 之差的绝对值, 输出最大的m。(2) 伪代码如下:

```
int ans(int A[], int n)
{
    int m;                // m 是数对之差的绝对值
    int m_max = INT_MIN;   // m_max 初值为最小 int
    for (int i = 0; i < n-1; i++) // 枚举前一个元素
        for (int j = i+1; j < n; j++) // 枚举后一个元素
        {
            m = A[i] - A[j]; // A[i] 和 A[j] 之差
            m_max = max(m_max, m); // 如果 m 大则更新 m_max
        }
    return m_max; // 输出最大 m 值
}
```

(3) 时间复杂度: $O(n^2)$ 空间复杂度: $O(1)$

【解法2】优化解：预处理，保存 i 之前的最大元素

(1) j 遍历一遍数组, t 保存 $A[0 \sim j-1]$ 的最大元素, 相当于枚举 j 作为数对后一个元素, 而 t 作为数对第一个元素, 数对差为 $m = t - A[j]$, 保存最大数对差 m_max。(2) 伪代码如下:

```
int ans(int A[], int n)
{
    int m_max = INT_MIN; // m_max初值为最小int
    int m;
    // 预处理, 数组B保存最大值
```

```

int B[n];
B[0] = A[0];
for (int i = 1; i < n; i++)           // 预处理
    B[i] = max(A[i], B[i-1]);
for (int i = 1; i < n; i++)
    m_max = max(m_max, B[i-1]-A[i]); // 更新 m_max
return m_max;                        // 输出最大m值
}

```

进一步优化可以发现只需要维护一个最大值变量，无需额外数组，空间复杂度可以降到 $O(1)$ ：

```

int ans(int A[], int n)
{
    int m_max = INT_MIN; // m_max 初值为最小 int
    int m;
    int t = A[0];        // t 来保存 A[i] 之前的最大值，初值为 A[0]
    for (int j = 1; j < n; j++) // 后一个元素下标从 1 开始
    {
        m = t - A[j];
        m_max = max(m_max, m); // 如果 m 大则更新 m_max
        t = max(t, A[j]);      // 如果 A[j] 大则更新 t
    }
    return m_max;            // 输出最大 m 值
}

```

(3) 时间复杂度： $O(n)$ 空间复杂度： $O(1)$ (这里保存状态只用了一个变量，所以空间复杂度仍然是 $O(1)$ 。)

习题 2-17

(15 分) 已知一个整数升序序列 $A = (a_0, a_1, \dots, a_{n-1})$ 。设计一个尽可能高效的算法返回整数 x 出现的次数。函数原型为：int countOccurrences(int A[], int n, int x)。(1) 给出算法的基本设计思想。(2) 根据设计思想，采用 C 语言描述算法，关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	得分
暴力解	枚举	$O(n)$	$O(1)$	13
优化解	折半查找	$O(\log n)$	$O(1)$	15

【解法1】暴力解：枚举，枚举所有元素，判断是否等于 x

(1) 从头到尾完整地遍历一次数组 A。在遍历的过程中，设置一个计数器，每当遇到一个值等于目标整数 x 的元素时，就将计数器加一。遍历结束后，计数器的值就是 x 在数组中出现的总次数。(2) **C语言算法描述**

```
int countOccurrences(int A[], int n, int x)
{
    int count = 0; // 初始化计数器
    // 枚举，遍历整个数组 A
    for (int i = 0; i < n; i++)
    {
        // 如果当前元素等于 x，计数器加一
        if (A[i] == x) count++;
    }
    return count; // 返回最终计数值
}
```

(3) 时间复杂度：O(n) 空间复杂度：O(1)

【解法2】优化解：折半查找，

(1) 如果数组 A 是有序的，我们可以利用二分查找来快速定位元素。这个方法的思想是：

1. **定位左边界**：首先，折半查找**第一个大于或等于 x** 的元素下标，记为 left。这确定了 x 首次出现的位置。
2. **有效性判断**：检查 A[left] 是否等于 x。如果不等，说明数组中根本不存在 x，直接返回 0。
3. **定位右边界**：接着，再次折半查找**第一个大于或等于 x+1** 的元素下标，记为 right。这个下标需要进行一次调整来精确定位最后一个 x 的位置：
 - 如果 A[right] 的值大于 x（即查找成功），那么最后一个 x 实际在 right 的前一个位置，因此我们将 right 减一。
 - 如果 A[right] 的值等于 x（即你查找失败，这只在 x 是数组末尾元素时发生），那么 right 本身就已经是最后一个 x 的位置，无需变动。
4. **计算次数**：此时，left 和 right 已分别指向第一个和最后一个 x 的下标。因此，x 出现的总次数就是 right - left + 1。(2)

```
// 折半查找模板二：查找第一个大于或等于 x 的元素的下标
int B_Search(int A[], int L, int R, int x)
{
    int mid;
    while (L < R)
    {
        mid = (L + R) / 2;
        if (A[mid] >= x) R = mid;
        else L = mid + 1;
    }
    return L;
}
```

```
int countOccurrences(int A[], int n, int x) {  
    // 1. 查找最左侧的 x 下标  
    int left = B_Search(A, 0, n-1, x);  
    // 2. 如果它不等于 x 则说明数组中没有 x  
    if (A[left] != x) return 0;  
    // 3. 查找最右侧的 x 下标  
    int right = B_Search(A, 0, n-1, x+1);  
    if (A[right] >= x+1)  
        right--;  
    // 4. 次数 = right - left + 1  
    return right - left + 1;  
}
```

(3) 时间复杂度: $O(\log n)$ 空间复杂度: $O(1)$

2.6.2 真题

2010 年 42 题

(13 分) 设将 n ($n > 1$) 个整数存放于一维数组 R 中。试设计一个在时间和空间两方面都尽可能高效的算法。将 R 中保存的序列循环左移 p ($0 < p < n$) 个位置, 即将 R 中的数据由 $(X_0, X_1, \dots, X_{n-1})$ 变换为 $(X_p, X_{p+1}, \dots, X_{n-1}, X_0, X_1, \dots, X_{p-1})$ 。要求: (1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举	$O(n)$	$O(n)$	10
特殊解	技巧	$O(n)$	$O(1)$	13

【解法1】暴力解: 枚举, 另设一个数组, 枚举每个元素再移动

(1) 原序列保存在数组 A 中, 设另一个数组 B , 将 $A[0], A[1], \dots, A[p-1]$ 放到 $B[n-p], B[n-p+1], \dots, B[n-1]$ 中, 将 $A[p], A[p+1], \dots, A[n-1]$ 放到 $B[0], B[1], \dots, B[n-p-1]$ 中, 数组 B 即是所求。(2) 伪代码如下:

```
void ans(int A[], int n, int p)
{
    int B[n];
    for (int i=0; i<p; i++)
        B[i+n-p]=A[i];
    for (int i=p; i<n; i++)
        B[i-p]=A[i];
    for (int i=0; i<n; i++)
        printf("%d ", B[i]);    // 输出元素重新放置后的数组
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(n)$

【解法2】特殊解: 技巧, 数组翻转

(1) 原序列保存在数组 A 中, 将数组 A 的前 p 个元素 $A[0] \sim A[p-1]$ 翻转, 再将后面的 $A[p] \sim A[n-1]$ 翻转, 最后整体翻转 $A[0] \sim A[n-1]$ 。(2) 伪代码如下:

```
void Reverse(int A[], int L, int R)
{
    // 翻转 A[L] ~ A[R]
```

```

int num=(R-L+1)/2;
for (int i=0; i<num; i++)
    swap(A[L+i], A[R-i]);
}
void ans(int A[], int n, int p)
{
    Reverse(A, 0, p-1);        // 翻转A[0~p-1]
    Reverse(A, p, n-1);        // 翻转A[p~n-1]
    Reverse(A, 0, n-1);        // 翻转A[0~n-1]
    for (int i=0; i<n; i++)
        printf("%d ", A[i]);    // 输出翻转后的数组
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(1)$

三次翻转顺序很关键: 先左段、再右段、最后整体。

2011 年 42 题

(15 分) 一个长度为 L ($L \geq 1$) 的升序序列 S , 处在第 $\lfloor L/2 \rfloor$ 个位置的数称为 S 的中位数。例如, 若序列 $S_1 = (11, 13, 15, 17, 19)$, 则 S_1 的中位数是 15, 两个序列的中位数是含它们所有元素的升序序列的中位数。例如, 若 $S_2 = (2, 4, 6, 8, 20)$, 则 S_1 和 S_2 的中位数是 11。现在有两个等长升序序列 A 和 B , 试设计一个在时间和空间两方面都尽可能高效的算法, 找出两个序列 A 和 B 的中位数。要求: (1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	快速排序	$O(n \log n)$	$O(n)$	11
优化解	多指针法+新数组	$O(n)$	$O(n)$	13
优化解	多指针法	$O(n)$	$O(1)$	14
优化解	折半查找	$O(\log n)$	$O(1)$	15

【解法1】暴力解: 都放入新数组中快速排序

(1) 将数组 A 和数组 B 中的数据都存入数组 C 中, C 数组长度为 $2n$, 对数组 C 进行快速排序, 输出排序后数组中的 $C[n-1]$ 。(2) 伪代码如下:

```

void Qsort(int A[], int L, int R)
{
    if (L >= R)

```

```

    return; // 区间≤1, 退出
int pivot, i=L, j=R; // i,j 为左右指针
pivot=A[L]; // pivot 为基准
while (i<j){
    while (i<j && A[j]>=pivot)
        j--;
    while (i<j && A[i]<=pivot)
        i++;
    if (i<j)
        swap(A[i], A[j]); // A[j]<pivot, A[i]>pivot
}
swap(A[L], A[i]); // 基准归位
Qsort(A, L, i-1); // 递归处理左区间
Qsort(A, i+1, R); // 递归处理右区间
}
int ans(int A[], int B[], int n)
{
    int C[2*n];
    for (int i=0; i<n; i++)
    {
        C[i]=A[i];
        C[n+i]=B[i];
    }
    Qsort(C, 0, 2*n-1); // 快速排序处理C[0]~C[2n-1]
    return C[n-1]; // 中位数输出C[n-1]
}

```

(3) 时间复杂度: $O(n\log n)$ 空间复杂度: $O(n)$

这种解法完全不利用 A、B 原本已升序的特性, 因此效率较低。

【解法2】优化解: 多指针法, 归并两个数组

(1) 采用归并的方式合并数组A和B到新的数组C中, 给A、B数组设置变量i和j保存数组下标, 初始时都为0, 每次比较A[i]和B[j], 数组C的下一个空位保存小的那个数, 并且对应指针后移, 直到一个数组遍历完, 将另一个数组中剩下的元素依次保存到C空位中。(2) 伪代码如下:

```

int ans(int A[], int B[], int n)
{
    int C[2*n];
    int i=0, j=0, k=0;
    while (i<n && j<n)
        if (A[i]<B[j])
            C[k++]=A[i++];
        else
            C[k++]=B[j++];
    while (i<n) C[k++]=A[i++];
    while (j<n) C[k++]=B[j++];
    return C[n-1]; // 输出C[n-1]
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(1)$

【解法3】优化解：数组指针后移 1) 给 A、B 数组设置变量 i 和 j 保存数组下标，计数 k 表示此时是第几小的元素，初始时都为 0，每次比较 $A[i]$ 和 $B[j]$ ，取较小的那个数对应指针后移且 $k++$ ，直到 k 等于 n，输出这个数。2) 伪代码如下：

```
int ans(int A[], int B[], int n)
{
    int i=0, j=0, k;
    for (k=1; k<n; k++) // 只需要移动 n-1 次，最后一次取当前较小值即为中位数。
        if (A[i]<B[j]) i++;
        else j++;
    return min(A[i], B[j]); // 输出A[i]和B[j]中的小值
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(1)$

归并两个有序数组时，中位数一定是整个合并序列第 n 小的元素，因此只需要知道该位置的元素，无需完整合并后的数组。

【解法4】优化解：多有序数组折半查找

(1) 对 A、B 数组进行折半查找，设 L_1 、 R_1 为数组 A 的左右查找边界， L_2 、 R_2 为 B 的左右查找边界。数组 $A[L_1 \text{ 至 } R_1]$ 和 $B[L_2 \text{ 至 } R_2]$ 的中位数，设为 a 和 b，求序列 A、B 的中位数过程如下：① 若 $a < b$ 则舍弃序列 A 中较小的一半 $A[L_1 \dots \text{mid}-1]$ ，同时舍弃序列 B 中较大的一半 $B[\text{mid}+1 \dots R_2]$ 。② 若 $a \geq b$ 则舍弃序列 A 中较大的一半 $A[\text{mid}+1 \dots R_1]$ ，同时舍弃序列 B 中较小的一半 $B[L_1 \dots \text{mid}-1]$ 。重复过程①、②，且需要保证数组 A、B 中的元素个数相同，如果不同则从长的数组中舍弃一个（因为我们取 mid 时都是向下取整，所以这里舍弃最小的那个值），直到两个序列中均只含一个元素时为止，较小者即为所求的中位数。本质上不断缩小搜索区间，每次抛弃较小半区，直至剩下的最后一个候选区间。（2）伪代码如下：

```
void ans(int A[], int B[], int n)
{
    int mid1, mid2, L1=0, L2=0, R1=n-1, R2=n-1;
    while (L1<R1) // 如果L1>=R1则退出循环
    {
        mid1=(L1+R1)/2;
        mid2=(L2+R2)/2; // 取中间数，向下取整
        if (A[mid1]<B[mid2]){
            L1=mid1;
            R2=mid2; // 更新查找范围
            if (R1-L1!=R2-L2)
                L1++; // 保证R1-L1=R2-L2
        }
        else{
            R1=mid1;
        }
    }
}
```

```

        L2=mid2;                // 更新查找范围
        if (R1-L1!=R2-L2)
            L2++;                // 保证R1-L1=R2-L2
    }
}
printf("%d", min(A[L1], B[L2])); // 输出A[i]和B[j]中的小值
}

```

(3) 时间复杂度: $O(\log n)$ 空间复杂度: $O(1)$

注意: 本题的 $O(\log n)$ 解法思想深刻, 但代码实现细节非常多, 在考场上一次写对的难度很高。对于大部分同学, 熟练掌握 $O(n)$ 的解法 (如解法3: 多指针法) 是更稳妥的策略, 可以确保拿到大部分分数。此 $O(\log n)$ 解法建议学有余力的同学深入理解。

2013 年 41 题

(13 分) 已知一个整数序列 $A = (a_0, a_1, \dots, a_{n-1})$, 其中 $0 \leq a_i < n$ ($0 \leq i < n$)。若存在 $a_{p_1} = a_{p_2} = \dots = a_{p_m} = x$ 且 $m > n/2$ ($0 \leq p_k < n, 1 \leq k \leq m$), 则称 x 为 A 的主元素。例如 $A = (0, 5, 5, 3, 5, 7, 5, 5)$, 则 5 为主元素; 又如 $A = (0, 5, 5, 3, 5, 1, 5, 7)$, 则 A 中没有主元素。假设 A 中的 n 个元素保存在一个一维数组中, 请设计一个尽可能高效的算法, 找出 A 的主元素。若存在主元素, 则输出该元素; 否则输出 -1。要求: (1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举	$O(n^2)$	$O(1)$	11
暴力解	快速排序	$O(n \log n)$	$O(\log n)$	13
优化解	散列表	$O(n)$	$O(n)$	14
特殊解	技巧	$O(n)$	$O(1)$	15

【解法1】暴力解: 枚举, 分别判断每个数是不是主元素

(1) 两层循环: 外层循环枚举数组中每个候选元素, 内层循环统计该元素在数组中出现的次数, 若次数大于 $n/2$ 则为主元素。(2) 伪代码如下:

```

int ans(int A[], int n)
{
    int m;                // m统计当前枚举元素出现次数
    for (int i=0; i<n; i++)
    {
        m=0;              // 每次换元素的时候m要清零
        for (int j=0; j<n; j++)

```

```

        if (A[i]==A[j])
            m++;
        if (m>n/2)
            return A[i];    // 找到了主元素
    }
    return -1;              // 未找到主元素
}

```

(3) 时间复杂度: $O(n^2)$ 空间复杂度: $O(1)$

【解法2】暴力解：快速排序，然后查找主元素

(1) 先快速排序得到升序序列，权值相同的元素都会相邻，扫描一趟数组，L是之前相邻的相同元素最左边的元素下标，i是此时正在扫描的元素下标，如果A[i]不等于A[L]则说明碰到了新的元素。(2) 伪代码如下：

```

void Qsort(int A[], int L, int R)
{
    if (L>=R)
        return;          // 区间≤1, 退出
    int pivot, i=L, j=R;  // i, j 为左右指针
    pivot=A[L];           // pivot 为基准
    while (i<j){
        while (i<j && A[j]>=pivot)
            j--;
        while (i<j && A[i]<=pivot)
            i++;
        if (i<j)
            swap(A[i], A[j]); // A[j]<pivot, A[i]>pivot
    }
    swap(A[L], A[i]);      // 基准归位
    Qsort(A, L, i-1);     // 递归处理左区间
    Qsort(A, i+1, R);     // 递归处理右区间
}

int ans(int A[], int n)
{
    Qsort(A, 0, n-1);
    int t=A[(n-1)/2];     // 主元素只可能是A[(n-1)/2]
    int m=0;              // m统计当前枚举元素出现次数
    for (int i=0; i<n; i++)
        if (A[i]==t)
            m++;
    if (m>n/2)
        return t;         // 找到了主元素
    return -1;            // 未找到主元素
}

```

(3) 时间复杂度: $O(n\log n)$ 空间复杂度: $O(\log n)$

【解法3】优化解：散列表

(1) 设置一个数组count[0...n-1]统计每个数字出现的次数，比如count[i]=1表示数字i出现了1次，遍历一次A数组统计所有数字出现的次数，处理完后扫描一次count数组找到是否有主元素，如果有则输出主元素，否则输出-1。(2) 伪代码如下：

```
int ans(int A[], int n)
{
    int count[n]={0}; // count统计数字出现频率
    for (int i=0; i<n; i++)
        count[A[i]]++; // 0<=A[i]<=n, 不会越界
    for (int j=0; j<n; j++)
        if (count[j]>n/2)
            return j; // 找到了主元素，输出主元素
    return -1; // 未找到主元素，输出-1
}
```

(3) 时间复杂度： $O(n)$ 空间复杂度： $O(n)$

【解法4】特殊解：技巧

(1) 利用抵消法（摩尔投票法）：扫描数组，用变量j记录当前候选主元素，t记录其票数。遇到与j相同的元素时票数加1，不同则减1，票数为0时更新候选元素。第一轮结束后，j为可能主元素，再次遍历确认其出现次数是否超过n/2，若超过则为主元素，否则不存在主元素。(2) 伪代码如下：

```
int ans(int A[], int n)
{
    int j=A[0], t=1; // 取A[0]作为可能主元素
    for (int i=1; i<n; i++)
        if (A[i]==j) t++;
        else
        {
            t--;
            if (t<0)
            {
                j=A[i];
                t=1;
            }
        }
    int m=0; // m统计j出现的次数
    for (int i=0; i<n; i++)
        if (A[i]==j) m++;
    if (m>n/2) return j; // j是主元素，输出j
    else return -1; // j不是主元素，输出-1
}
```

(3) 时间复杂度： $O(n)$ 空间复杂度： $O(1)$

摩尔投票法

摩尔投票法最早由 **Boyer 和 Moore** 在 1981 年提出，原本用于处理在线序列中寻找主元素问题。主元素定义为：在长度为 n 的序列中，出现次数严格超过 $n/2$ 的元素。

其核心思想是“抵消法”——把不同的元素互相抵消掉，主元素由于数量超过一半，最终必然“存活”下来。

摩尔投票法正确性证明：

- 主元素出现次数 $> n/2$ ，非主元素出现次数 $< n/2$ 。
- 每次遇到不同元素，抵消一对“主元素-非主元素”。
- 最多能抵消掉 $n-m$ 次（ m 是主元素出现次数）。
- 剩余主元素个数为 $m - (n-m) = 2m - n > 0$ ，说明主元素一定存活到最后。
- 最后留下的候选人若存在主元素，必然是主元素。
- 再次扫描确认其出现次数是否 $> n/2$ 即可。

2016 年 43 题

(13分) 已知由 n ($n \geq 2$) 个正整数构成的一个集合 $A = \{a_k | 0 \leq k < n\}$ ，将其划分为两个不相交的子集 A_1 和 A_2 ，元素个数分别是 n_1 和 n_2 ， A_1 和 A_2 中元素之和分别为 S_1 和 S_2 。设计一个尽可能高效的划分算法，满足 $|n_1 - n_2|$ 最小且 $|S_1 - S_2|$ 最大。要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。（3）说明你所设计算法的平均时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	快速排序	$O(n \log n)$	$O(\log n)$	9
优化解	多指针法	$O(n)$	$O(\log n)$	12
优化解	多指针法	$O(n)$	$O(1)$	13

【解法1】暴力解：快速排序 1) 对数组 $A[0 \dots n-1]$ 进行快速排序得到升序，排序后集合 S_1 是 $A[0 \dots n/2-1]$ （向下取整），集合 S_2 是 $A[n/2 \dots n-1]$ （向下取整）。2) 伪代码如下：

```
void Qsort(int A[], int L, int R, int )
{
    if (L >= R)
        return; // 区间≤1，退出
    int pivot, i=L, j=R; // i, j 为左右指针
    pivot=A[L]; // pivot 为基准
```

```

while (i<j){
    while (i<j && A[j]>=pivot)
        j--;
    while (i<j && A[i]<=pivot)
        i++;
    if (i<j)
        swap(A[i], A[j]); // A[j]<pivot, A[i]>pivot
}
swap(A[L], A[i]); // 基准归位
Qsort(A, L, i-1); // 递归处理左区间
Qsort(A, i+1, R); // 递归处理右区间
}

void ans(int A[], int n)
{
    Qsort(A, 0, n-1);
    输出A1: A[0~n/2-1]
    输出A2: A[n/2~n-1] // 奇数时中间数分给A2
}

```

(3) 时间复杂度: $O(n\log n)$ 空间复杂度: $O(\log n)$

【解法2】特殊解：多指针法（递归）

(1) 对数组 $A[0\sim n-1]$ 进行类似快速排序的做法，在处理左右区间时只处理可能包含中位数的区间，即如果区间的范围是 $[L, R]$ ，则只有 $L \leq n/2-1 \leq R$ 才会处理该区间。对快排的递归调用稍作修改即可。(2) 伪代码如下：

```

void Quick_Select(int A[], int L, int R, int n)
{
    if (L>=R)
        return; // 区间≤1，退出
    int pivot, i=L, j=R; // i, j 为左右指针
    pivot=A[L]; // pivot 为基准
    while (i<j)
    {
        while (i<j && A[j]>=pivot)
            j--;
        while (i<j && A[i]<=pivot)
            i++;
        if (i<j)
            swap(A[i], A[j]); // A[j]<pivot, A[i]>pivot
    }
    swap(A[L], A[i]); // 基准归位
    if (n/2-1==i) break;
    if (n/2-1>=L && n/2-1<=i-1)
        Quick_Select(A, L, i-1, n); // n/2-1在左区间范围中，选择左区间递归处理
    else
        Quick_Select(A, i+1, R, n); // n/2-1在右区间范围中，选择右区间递归处理
}

void ans(int A[], int n)
{
    Quick_Select(A, 0, n-1, n);
    输出A1: A[0~n/2-1]
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(\log n)$ (注: 关于此方法的时间复杂度记住即可, 不需要掌握, 《算法导论》上有完整证明)

✎ 时间复杂度证明

- 这是一个「快速选择」(Quick Select) 的递归算法, 类似快排。
- 每次划分时, 仍然执行一次 partition (类似快排的分区), 其时间是 $O(\text{区间长度})$, 第一次是 $O(n)$, 下一次是 $O(n/2)$, 再下一次是 $O(n/4)$, 以此类推。
- 但与快排不同的是:
 - 快排每次递归处理**两个子区间**;
 - 而这里的 Quick Select 只递归处理包含第 $n/2$ 个元素的那一侧子区间 —— **只递归一个子区间**。
- 因此, 总时间复杂度满足如下递归关系:

$$T(n) = T(n/2) + O(n)$$

- 这是典型的「二分递归 + 线性工作量」, 根据递推公式展开:

$$T(n) = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \cdots \approx 2n = O(n)$$

注意这样求的其实是平均时间复杂度是 $O(n)$, 但对于和快速排序有关的方法, 我们都可以写平均时间复杂度来。

【解法3】特殊解: 多指针法 (非递归)

(1) 对数组 $A[0 \sim n-1]$ 进行类似快速排序的做法, 在处理左右区间时只处理可能包含中位数的区间, 即如果区间的范围是 $[l, r]$, 则只有 $l \leq n/2 - 1 \leq r$ 才会处理该区间, 可以不用递归来写, 每次划分后修改 l 和 r 的范围即可。 (2) 伪代码如下:

```
void ans(int A[], int n)
{
    int L=0, R=n-1;
    while (L<R){
        int pivot, i=L, j=R;
        pivot=A[L];
        while (i<j){
            while (i<j && A[j]>=pivot)
                j--;
            while (i<j && A[i]<=pivot)
                i++;
            if (i<j)
                swap(A[i], A[j]);
        }
        // 类快排划分过程
        // i, j 为左右指针
        // pivot 为基准
        // A[j]<pivot, A[i]>pivot
    }
}
```

```

    }
    swap(A[L], A[i]);                // 基准归位
    if (n/2-1==i) break;            // 分成所需要的两个区间
    if (n/2-1>=L && n/2-1<=i-1) R=i-1; // n/2-1在左，改右边界
    else L=i+1;                     // n/2-1在右，改左边界
}
输出A1: A[0~n/2-1]
输出A2: A[n/2~n-1]                // 奇数时中间数分给A2
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(1)$

2018 年 41 题

(13分) 给定一个含 n ($n \geq 1$) 个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如，数组{-5, 3, 2, 3}中未出现的最小正整数是1；数组{1, 2, 3}中未出现的最小正整数是4。要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用C或C++语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举	$O(n^2)$	$O(1)$	10
暴力解	快速排序	$O(n \log n)$	$O(\log n)$	12
优化解	散列表	$O(n)$	$O(n)$	13

【解法1】暴力解：枚举最小正整数

(1) 可以证明最小未出现的正整数 t 一定 $\leq n+1$ （因为如果 $t > n+1$ ，则 $1 \sim t-1$ 都会出现一次，但只有 n 个整数，所以 $t-1 \leq n$ ，即 $t \leq n+1$ ），枚举所有可能的未出现最小正整数，扫描一次数组判断是否符合。（2）伪代码如下：

```

int ans(int A[], int n)
{
    for (int i=1; i<=n+1; i++)
    {
        bool flag=false;    // 先假设i未出现
        for (int j=0; j<n; j++)
            if (i==A[j])
                flag=true;    // 此时i出现了，flag变成true
        if (flag==false)    // 若i未出现过则输出i
            return i;
    }
}

```

(3) 时间复杂度: $O(n^2)$ 空间复杂度: $O(1)$

【解法2】暴力解：快速排序，然后扫描一遍数组

(1) 先对数组A快速排序得到升序序列，然后遍历数组找到第一个未出现的最小正整数。

(2) 伪代码如下：

```
void Qsort(int A[], int L, int R)
{
    if (L >= R)
        return; // 区间≤1, 退出
    int pivot, i=L, j=R; // i, j 为左右指针
    pivot=A[L]; // pivot 为基准
    while (i < j) {
        while (i < j && A[j] >= pivot)
            j--;
        while (i < j && A[i] <= pivot)
            i++;
        if (i < j)
            swap(A[i], A[j]); // A[j] < pivot, A[i] > pivot
    }
    swap(A[L], A[i]); // 基准归位
    Qsort(A, L, i-1); // 递归处理左区间
    Qsort(A, i+1, R); // 递归处理右区间
}

int ans(int A[], int n)
{
    Qsort(A, 0, n-1);
    int t=1; // 假设初始最小未出现正整数t=1
    for (int i=0; i<n; i++)
        if (A[i] == t)
            t++; // t已出现，此时最小未出现正整数是t+1
    return t; // 输出最小未出现正整数
}
```

(3) 时间复杂度: $O(n \log n)$ 空间复杂度: $O(\log n)$

【解法3】优化解：散列表

(1) 可以证明最小未出现的正整数t一定 $\leq n+1$ （因为如果 $t > n+1$ ，则1~t-1都会出现一次，但只有n个整数，所以 $t-1 \leq n$ ，即 $t \leq n+1$ ），设置一个数组count，count[i]表示i是否在原数组A中出现过，扫描一遍数组A，更新count数组，之后再扫描一遍count数组输出未出现的最小正整数。

(2) 伪代码如下：

```
int ans(int A[], int n)
{
    int count[n+2] = {0}; // 0~n+1
    for (int i=0; i<n; i++) // 扫描数组A
```

```

    if (A[i]>0 && A[i]<=n+1) // 当0<A[i]<=n+1时才修改
        count[A[i]]++;
    for (int j=1; j<=n+1; j++) // 扫描count数组
        if (count[j]==0) // j未出现过，是最小未出现正整数
            return j; // 输出j
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(n)$

2020 年 41 题

(13 分) 定义三元组 (a, b, c) (a, b, c 均为正数) 的距离 $D = |a - b| + |b - c| + |c - a|$ 。给定 3 个非空整数集合 S_1 、 S_2 和 S_3 ，按升序分别存储在 3 个数组中。请设计一个尽可能高效的算法，计算并输出所有可能的三元组 (a, b, c) ($a \in S_1, b \in S_2, c \in S_3$) 中的最小距离。例如 $S_1 = \{-1, 0, 9\}$, $S_2 = \{-25, -10, 10, 11\}$, $S_3 = \{2, 9, 17, 30, 41\}$ ，则最小距离为 2，相应的三元组为 $(9, 10, 9)$ 。要求：(1) 给出算法的基本设计思想。(2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举	$O(nml)$	$O(1)$	10
优化解	枚举+多指针法	$O(mn)$	$O(1)$	11
优化解	枚举+折半查找	$O(n * \log(m * l))$	$O(1)$	12
优化解	多指针法	$O(m + n + l)$	$O(1)$	13

【解法1】暴力解：枚举，三重循环将abc都扫描一次 1) 数组A、B、C的长度分别为n、m、l，三重循环枚举a、b、c，对于每一组a、b、c求出对应的距离D，并从中选出最小的距离。2) 伪代码如下：

```

int ans(int A[],int n,int B[],int m,int C[],int l)
{
    int D_min=INT_MAX, D; // D_min初值为最大整数
    for (int i=0; i<n; i++)
        for (int j=0; j<m; j++)
            for (int k=0; k<l; k++){ // 三重循环枚举abc
                D=abs(A[i]-B[j])+abs(B[j]-C[k])+abs(C[k]-A[i]);
                D_min=min(D_min, D); // 更新最小D值
            }
    return D_min; // 输出最小D值
}

```

(3) 时间复杂度: $O(nml)$ 或 $O(n^3)$ 空间复杂度: $O(1)$

【解法2】优化解：枚举+多指针法。两重循环扫描ab，而c数组指针后移 1) 数组A、B、C的长度分别为n、m、l，两重循环枚举a、b，对数组C设置变量k存储数组下标不断后移，初始k=0，只要移动后的D更小，就选择往后移动，然后对于每一组a、b、c求出对应的距离D，并从中选出最小的距离。 2) 伪代码如下：

```
int DD(int a,int b,int c)
{
    // DD(a,b,c)=|a-b|+|b-c|+|b-a|
    return abs(a-b)+abs(b-c)+abs(c-a);
}

int ans(int A[],int n,int B[],int m,int C[],int l)
{
    int D, D_min=INT_MAX; // D_min初值为最大整数
    int k=0;
    for (int i=0; i<n; i++)
        for (int j=0; j<m; j++)
        {
            // 两重循环枚举a、b
            D = DD(A[i], B[j], C[k]); // 枚举a、b得到的初始D
            while (k<l-1 && DD(A[i], B[j], C[k+1])<D)
                D = DD(A[i], B[j], C[++k]); // 更新下标k和距离D
            D_min = min(D_min, D); // 更新最小D值
        }
    return D_min; // 输出最小D值
}
```

(3) 时间复杂度： $O(mn)$ 或 $O(n^2)$ 空间复杂度： $O(1)$

【解法3】优化解：枚举+折半查找。一重循环扫描a，两轮折半查找b、c

(1) 本解法可以证明当a确定时，b只能是数组B中 $< a$ 且最接近a的那个数或 $\geq a$ 且最接近a的那个数，c也只能是数组C中 $< a$ 且最接近a的那个数或 $\geq a$ 且最接近a的那个数，所以只需要枚举a并折半查找确定b和c可能的值然后选择最小的D。数组A、B、C的长度分别为n、m、l，一重循环枚举 $a=A[i]$ ，通过两次折半查找可以查找到插入位点j和k保证 $B[j] < A[i] < B[j+1]$ ， $C[k] < A[i] < C[k+1]$ （如果插入位点是最左端和最右端另外判断），然后判断 $\langle A[i], B[j], C[k] \rangle$ 、 $\langle A[i], B[j+1], C[k] \rangle$ 、 $\langle A[i], B[j], C[k+1] \rangle$ 、 $\langle A[i], B[j+1], C[k+1] \rangle$ 哪种情况D最小即可，就能得到a确定的情况下最小可能的D，枚举a更新D_min即可。最后输出最小值D_min。 (2) 伪代码如下：

```
int DD(int a,int b,int c)
{
    // DD(a,b,c)=|a-b|+|b-c|+|b-a|
    return abs(a-b)+abs(b-c)+abs(c-a);
}

int ans(int A[],int n,int B[],int m,int C[],int l)
{
    int D, D_min=INT_MAX; // D_min初值为最大整数
    for (int i=0; i<n; i++)
    {
        int j=B_Search(B, 0, m-1, A[i]); // 在B中根据A[i]折半查找
```

```

int k=B_Search(C, 0, l-1, A[i]);    // 在C中根据A[i]折半查找
if (j>=0 && k>=0)
    D_min=min(D_min, DD(A[i], B[j], C[k]));
if (j+1<m && k>=0)
    D_min=min(D_min, DD(A[i], B[j+1], C[k]));
if (j>=0 && k+1<l)
    D_min=min(D_min, DD(A[i], B[j], C[k+1]));
if (j+1<m && k+1<l)
    D_min=min(D_min, DD(A[i], B[j+1], C[k+1]));
}
return D_min;    // 输出最小D值
}

```

(3) 时间复杂度: $O(n * (\log m + \log l))$ 或 $O(n \log n)$ 空间复杂度: $O(1)$

【解法4】优化解：多指针法

(1) 数组A、B、C的长度分别为n、m、l，对数组A、B、C分别设置变量i、j、k存储数组下标不断后移，初始i=j=k=0，可以证明如果下标是往后移动，只有当移动A[i]、B[j]、C[k]最小的那个时，D=2L3才有可能减小，所以每次都只后移所指元素值最小的对应数组下标变量，一直重复进行，并更新D的最小值D_min，直到所指元素值最小的对应数组无法在后移，最后输出最小值D_min。(2) 伪代码如下：

```

int DD(int a,int b,int c)    // DD(a,b,c)=|a-b|+|b-c|+|b-a|
{
    return abs(a-b)+abs(b-c)+abs(c-a);
}
int ans(int A[],int n,int B[],int m,int C[],int l)
{
    int D_min=INT_MAX;    // D_min初值为最大整数
    int i=0, j=0, k=0;    // 数组A、B、C的下标i、j、k
    while(i<n && j<m && k<l)
    {
        D_min=min(D_min, DD(A[i], B[j], C[k])); // 更新D_min
        if (A[i]<B[j] && A[i]<C[k])
            i++;
        else if (B[j]<C[k])
            j++;
        else
            k++;
    }
    return D_min;    // 输出最小D值
}

```

(3) 时间复杂度: $O(m + n + l)$ 或 $O(n)$ 空间复杂度: $O(1)$

(13 分) 有两个长度均为 n 的一维整型数组 $A[n]$ 、 $res[n]$ ，计算 $A[i]$ 与 $A[j]$ ($0 \leq i \leq j \leq n-1$) 乘积的最大值，并将其保存到 $res[i]$ 中。若 $A[] = \{1, 4, -9, 6\}$ ，则得到 $res[] = \{6, 24, 81, 36\}$ 。现给定数组 A ，请设计时间空间上尽可能高效的算法 $CalMulMax$ ，求 res 中各元素的值。函数原型为：void $CalMulMax(int A[], int res[], int n)$ 。(1) 给出算法的基本思想。(4 分) (2) 用 C/C++ 描述算法关键之处给出注释。(7 分) (3) 说明时间、空间复杂度。(2 分)

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举	$O(n^2)$	$O(1)$	10
优化解	预处理	$O(n)$	$O(n)$	12
优化解	预处理	$O(n)$	$O(1)$	13

【解法1】暴力解（双重循环枚举，10分左右） 1) 算法的基本思想 使用两个嵌套的for循环遍历数组A的所有可能的(i, j)组合，其中 $0 \leq i \leq j \leq n-1$ 。对于每个i，遍历所有 $j \geq i$ ，计算 $A[i] * A[j]$ ，并记录其中的最大值到 $res[i]$ 中。 2) C代码实现

```
// 暴力破解法实现
void CalMulMax(int A[], int res[], int n)
{
    for(int i = 0; i < n; i++)
    {
        // 遍历每个起始位置i
        int maxMul = A[i] * A[i]; // 初始化
        for(int j = i; j < n; j++)
        {
            // 遍历所有 j >= i
            int currentMul = A[i] * A[j];
            if (currentMul > maxMul) // 当前乘积更大，更新
                maxMul = currentMul;
        }
        res[i] = maxMul; // 将最大乘积存入res[i]
    }
    // 输出数组res
}
```

3) 时间复杂度： $O(n^2)$ ，因为使用了两个嵌套的for循环。空间复杂度： $O(1)$ ，使用了常数级别的额外空间。

【解法2】预处理，使用辅助数组B和C（12分左右） 1) 算法的基本思想观察可知：

- 如果 $A[i]$ 是正数，想要乘积最大，自然希望乘以区间内最大的数；
- 如果 $A[i]$ 是负数，想要乘积最大，需要乘以区间内最小的数（即最小的负数，两个负数相乘为正，值越大）。因此，对于每个位置 i ，提前预处理出：
- 从位置 i 到末尾的**最大值** $B[i]$ ；

- 从位置 i 到末尾的**最小值** $C[i]$ 。最终对于每个 i ，只需要在 $A[i] * B[i]$ 和 $A[i] * C[i]$ 之间取较大值作为 $res[i]$ ，即： $res[i]=\max(A[i]*B[i], A[i]*C[i])$ 2) C代码实现

```
// 使用辅助数组B和C的实现
void CalMulMax(int A[], int res[], int n)
{
    int B[n];    // B[i]表示A[i..n-1]中的最大值
    int C[n];    // C[i]表示A[i..n-1]中的最小值
    // 从后向前遍历，填充B和C数组
    B[n-1] = A[n-1];
    C[n-1] = A[n-1];
    for(int i = n-2; i >= 0; i--)
    {
        B[i] = max(B[i+1], A[i]);
        C[i] = min(C[i+1], A[i]);
    }
    // 计算res数组，最大值只可能是A[i]*B[i]或A[i]*C[i]
    for(int i = 0; i < n; i++)
        res[i] = max(A[i]*B[i], A[i]*C[i]);
    输出数组res
}
```

3) 时间复杂度： $O(n)$ ，因为预处理B和C数组各需一次线性遍历，计算res数组也只需一次线性遍历。空间复杂度： $O(n)$ ，需要额外的两个辅助数组B和C，每个大小为n。

【解法3】预处理，使用辅助变量B和C (13分左右) 1) 算法的基本思想 只需要在解法二的基础上，用变量存最小值和最大值即可。 2) C代码实现

```
// 使用辅助变量B和C的实现
void CalMulMax(int A[], int res[], int n)
{
    int B;    // B表示A[i..n-1]中的最大值
    int C;    // C表示A[i..n-1]中的最小值
    B = A[n-1];
    C = A[n-1];
    res[n-1] = A[n-1]*A[n-1];
    // 计算res数组，最大值只可能是A[i]*B[i]或A[i]*C[i]
    for(int i = n-2; i >= 0; i--)
    {
        B = max(B, A[i]);
        C = min(C, A[i]);
        res[i] = max(A[i]*B, A[i]*C);
    }
    输出数组res
}
```

3) 时间复杂度： $O(n)$ 。空间复杂度： $O(1)$ 。

3 链表

由 2009 年真题引入

【2009 年真题，15 分】已知一个带有表头结点的单链表，结点结构为

data	link
------	------

假设该链表只给出了头指针 list。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第 k 个位置上的结点（k 为正整数）。若查找成功，算法输出该结点的 data 域的值，并返回 1；否则，只返回 0。要求：1) 描述算法的基本设计思想。2) 描述算法的详细实现步骤。3) 根据设计思想和实现步骤，采用程序设计语言描述算法（使用 C、C++ 或 Java 语言实现），关键之处请给出简要注释。

3.1 链表概述

3.1.1 特点

我们要在学习了顺序表的基础上再来学习链表，链表和顺序表比较：

- 共同点：都是线性表。这意味着顺序表的很多方法在链表中依然可以使用，如：散列表、多指针法、预处理，主要考虑前两者。
- 不同点：
 - 链表不可以随机访问。即很多需要用到随机访问的方法都不能使用，如快速排序（双链表可以使用，但不推荐）、折半查找。
 - 单链表只能单向从头到尾依次访问。即访问顺序只能是从头到尾，那我们需要掌握很多对应的技巧，如：快慢指针、头插法等。

做题推荐流程：暴力解法->判断能否优化->优化解。对链表有一个技巧：如果算法的**时间复杂度是 $O(n\log n)$ 或以上**，则可以优化，否则就不需要。

关于这种技巧，请大家学习完本章再试着回答为什么可以这样判断。

3.1.2 题目的要求

- 如果题目要求**空间复杂度是 $O(1)$** ，这就意味着不能额外设置数组，不能将链表先用数组保存然后再进行处理。
- 如果题目要求**不可修改链表结构**，意思是我们不能随意的对链表元素顺序进行修改以及随意的插入和删除。

3.1.3 链表定义

题目中经常会有如下描述：

(2) 使用 C 或 C++语言，给出单链表结点的数据类型定义。

对这类问题，我们可以提前记忆模板，然后考试中根据题目要求进行改动（**仔细审题**）：

单链表（data 是该结点的权值，一般可以定义成 int；next 是指向下一个结点的指针，保存的是下一个结点的位置）：

```
typedef struct LNode
{
    Elemtype data;    // 该结点的权值
    struct LNode *next; // next指向下一个结点
}LNode;
```

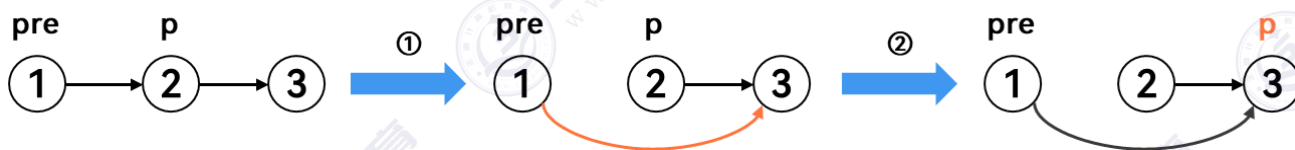
双链表（不需要特别记忆，只是比单链表多一个 prior 指针而已）：

```
typedef struct LNode
{
    Elemtype data;    // 该结点的权值
    struct LNode *prior, *next; // prior指向上一个结点
}LNode;
```

3.2 基本操作

链表的基本操作是实现所有算法的基础。

3.2.1 删除 p



在考试中删除 p 结点之后一般还需要将 p 移动到下一个结点，所以删除时需要注意两个问题：

- 删除操作不要导致断链。
- 要能够找到下一次待处理的结点，即 p 要更新所指结点。

```
pre->next = p->next;    // ① 将pre的next域指向p的下一个结点
p = p->next;           // ② 将p指向p的下一个结点
```

释放内存：在删除结点时，除了修改前后结点的指针，还需要考虑释放结点所占的内存。如果是 C 语言，需要调用 `free(p)`。但在考试中为了方便不写也没关系。

这是加上了 `free` 的写法，仅供参考：

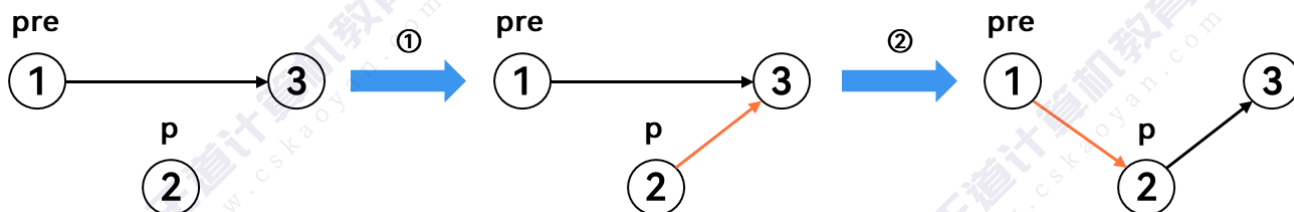
```
pre->next = p->next;    // ① 将pre的next域指向p的下一个结点
Node* temp = p;
p = p->next;           // ② 将p指向p的下一个结点
free(temp);
```

3.2.2 p 插入到 pre 后面

p 插入 pre 后面的结点:

```
p->next = pre->next;    // ① 将p的next域指向pre的下一个结点  
pre->next = p;           // ② 将pre的next域指向p对应结点
```

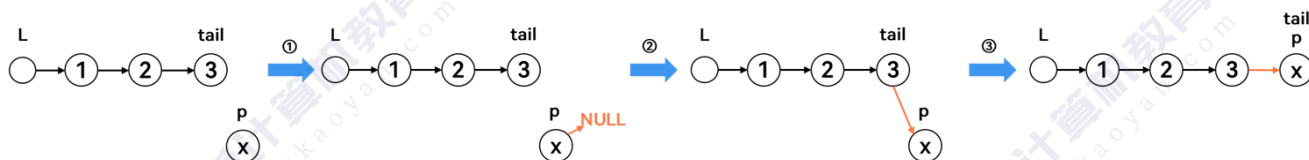
将 p 插入到 pre 后面要注意先处理 p 的 next, 后处理 pre 的 next, 否则可能会因为先修改了 pre 的 next 而导致找不到 pre 的下一个结点。



3.2.3 尾插法和头插法

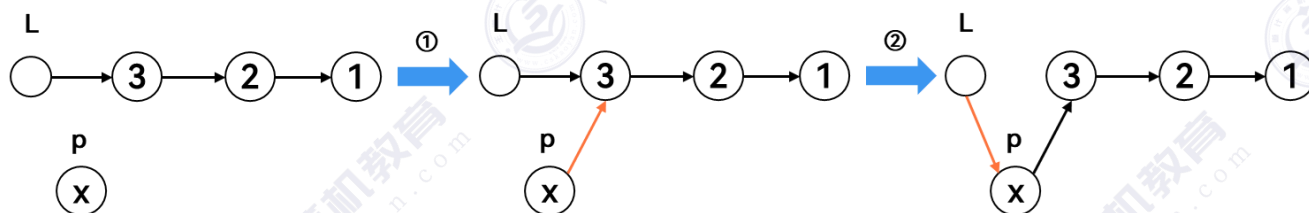
当我需要插入很多个元素到空链表中，而且要保证插入元素的顺序就是最终链表中元素的顺序，可以使用尾插法，即每次都是在链表末尾插入 p ，需要设置一个指针 $tail$ 指向链表末尾的元素：

```
p->next=NULL;    // ① 将p的next域指向NULL
tail->next=p;     // ② 将末尾结点的next指向p
tail=p;          // ③ p是新的末尾结点
```



而如果想让插入顺序和链表最终顺序相反，则不应使用尾插法而使用头插法，是将一个结点 p 插入到头结点 L 和其原来的第一个数据结点之间。这种方法常用于实现链表的逆置，具体使用请看 3.5.3。

```
void Head_Insert(Node* L, Node* p)
{
    p->next=L->next;
    L->next=p;
}
```



所有的单链表插入都应该是**先连后，再连前**，这样一定不会断链。

假如插入顺序是 1、2、3、 x ，如果使用**尾插法**插入则链表最终顺序为 1、2、3、 x （和插入顺序相同），如果使用**头插法**插入则链表最终顺序为 x 、3、2、1（和插入顺序相反）。

3.2.4 遍历链表

对于线性表（数组+链表）的遍历，如果表中元素个数确定就用 for 循环，否则用 while 循环。在遍历过程中访问指针 p 每次都需要后移即 $p=p->next$ ，不要忘记。链表一般是带头结点的，所以遍历时应从 $L->next$ 开始。

元素个数 n 已知

```
Node* p = L->next;
for (int i = 0; i < n; i++)
{
    visit(p); // 访问p结点
    p = p->next;
}
```

元素个数未知

```
Node* p = L->next;
while (p != NULL)
{
    visit(p); // 访问p结点
    p = p->next;
}
```

在此基础上，还可以统计元素个数 n:

```
Node* p = L->next;
int n = 0;
while (p != NULL)
{
    n++;
    visit(p); // 访问p结点
    p = p->next;
}
```

其实所有的循环都可以写成 for 循环，这种写法和 while 循环是等价的：

```
for (Node* p = L->next; p != NULL; p = p->next) visit(p);
```

3.3 暴力解法

练习：请在 2 分钟内想出这道例题做法（不考虑复杂度）

例题 3-1：请将链表 L 按照升序重新排序。结点结构为

data	next
------	------

3.3.1 枚举法，直接处理

链表的一些题目可以先采用最直接易想的方法，比如要改变链表元素顺序的话，可以将需要重排的元素一个一个拆下来重新插入。这类方法看起来比较憨，但是容易想到。

例题 3-1 解法如下：

```
// 查找并插入p
void Find_Insert(ListNode *pre, ListNode *p)
{
    while (pre->next!=NULL && pre->next->data<p->data) // 查找插入位置
        pre=pre->next;
    p->next=pre->next;
    pre->next=p; // 将p插入到pre之后
}

Node* ans(Node *L)
{
    Node *p=L->next;
    L->next=NULL;
    while (p!=NULL)
    {
        Node *temp=p->next;
        Find_Insert(L,p);
        p=temp;
    }
    return L;
}
```

3.3.2 链表转数组，再处理

可以使用数组保存链表中的结点地址或者直接保存数据，然后再按照数组的做题方法操作即可。

注意

以下情况不能使用数组保存链表的结点：

- 题目要求空间复杂度为 $O(1)$ ；

例题 3-1 解法如下：

```
// 快速排序
void Qsort(int A[], int L, int R)
{
    if (L >= R)
        return;
    int pivot, i = L, j = R;
    pivot = A[L];
    while (i < j)
    {
        while (i < j && A[j] >= pivot) j--;
        while (i < j && A[i] <= pivot) i++;
        if (i < j)
            swap(A[i], A[j]); // A[j] < pivot < A[i], 交换
    }
    swap(A[L], A[i]); // 基准归位
    Qsort(A, L, i - 1);
    Qsort(A, i + 1, R);
}

Node* ans(Node *L)
{
    int a[maxn], n = 0;
    Node *p = L->next;

    // 链表转数组
    while (p != NULL)
    {
        // visit(p);
        a[n] = p->data;
        n++;
        p = p->next;
    }
    Qsort(a, 0, n - 1);

    // 数组转链表
    p = L->next;
    for (int i = 0; i < n; i++)
    {
        p->data = a[i];
    }
}
```

```
p=p->next;  
}  
return L;  
}
```

3.4 判断能否优化

判断方法见 2.3。对于链表来说，如果算法的时间复杂度是 $O(n\log n)$ 或以上，则可以优化，否则就不需要。

3.5 适用于链表的优化算法

3.5.1 多指针法

回忆上一章内容，多指针法可以用于顺序表和链表，顺序表的多指针法分为三种：多个有序顺序表、单个有序顺序表、单个无序顺序表，后两种都会涉及从后往前访问，而在单链表中是无法实现从后往前访问的，因此只考虑第一种（多个有序链表）。

除此之外，链表自身有一些特殊用法：**快慢指针**、**前后指针**。

类型1：多个有序链表

请大家回顾顺序表的多指针法，对于多个有序顺序表的情况，需要对每一个顺序表都设置一个指针（从前往后或从后往前）。

以下是顺序表的写法，它和链表写法的思路、每个模块的功能都是一样的，只是每个模块具体代码不一样，请大家尝试根据顺序表写法改写成链表写法。

```
void Merge(int A[], int n, int B[], int m)
{
    int C[n + m];
    int i = 0, j = 0, k = 0;    // i, j, k 分别是 A, B, C 的指针

    // 直到有一个数组遍历完退出循环
    while (i < n && j < m)
        if (A[i] < B[j])
            C[k++] = A[i++];
        else
            C[k++] = B[j++];

    // 复制 A 中剩余的元素
    while (i < n)
        C[k++] = A[i++];

    // 复制 B 中剩余的元素
    while (j < m)
        C[k++] = B[j++];

    从左到右输出数组 C 中的元素
}
```

以下是链表写法。因为单链表只允许从前往后，每个有序链表都设置一个指针从头结点开始从后往前进行访问。在数组中每次后移指针下标 i 是 $i++$ ，而链表中指针 p 则是 $p=p->next$ ，本质一样。

归并两个带头结点的升序链表（理解+应用），代码如下：

```

// 归并操作
LNode* Merge(LNode *L1, LNode *L2)
{
    // p 和 q 分别指向两个链表的第一个数据结点
    LNode *p = L1->next;
    LNode *q = L2->next;

    // 创建一个新的头结点 L3，并用 tail 指向它
    LNode *L3 = (LNode *)malloc(sizeof(LNode));
    LNode *tail = L3;

    // 遍历两个链表，直到其中一个为空
    while (p != NULL && q != NULL)
    {
        if (p->data <= q->data)
        {
            tail->next = p;
            p = p->next;
        }
        else
        {
            tail->next = q;
            q = q->next;
        }
        tail = tail->next; // 移动 tail 指针到新链表的末尾
    }

    // 将未处理完的链表剩余部分直接链接到 tail 之后
    if (p != NULL) tail->next = p;
    else tail->next = q;

    // 释放原始链表的头结点，可以省略
    free(L1);
    free(L2);

    return L3;
}

```

请大家思考：多指针法插入的时候使用头插法，会发生什么事情？

类型2：快慢指针

如果需要判断一个单链表是否有环，可以通过设置两个指针 p 和 q，初始时都指向第 1 个点，然后每次 p 往后移动 1 次，而 q 往后移动 2 次，保证 q 往后移动的次数是 p 的两倍，如果 q 能从 p 后面追上 q 即 $p == q$ ，说明该单链表有环。

```
Node *p, *q;
p = q = L->next;
while (p != NULL && q != NULL)
{
    p = p->next;
    q = q->next;
    if (q == NULL)
        break;
    q = q->next;
    if (p == q)
        有环 // p和q相遇
}
```

而如果我们h需要返回中点，可以这样写

```
Node *p, *q;
p = q = L->next;
while (p != NULL && q != NULL)
{
    q = q->next;
    if (q == NULL || q->next == NULL)
        break;
    q = q->next;
    p = p->next;
    if (p == q)
        有环 // p和q相遇
}
```

类型3：前后指针

【2009 年真题，15 分】已知一个带有表头结点的单链表，结点结构为

data	link
------	------

假设该链表只给出了头指针 list。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第 k 个位置上的结点 (k 为正整数)。若查找成功，算法输出该结点的 data 域的值，并返回 1；否则，只返回 0。

因为单链表只能按照一个方向处理，无法从后往前回头。所以设置两个指针 p 和 q ，初始时 p 指向 L ， q 指向第 k 结点，然后 p 和 q 每次同时后移，这样 p 和 q 之间一直保持 k 的距离，当 q 指向 NULL 时，此时 p 指向倒数第 k 个元素。

```
Node *p, *q;
p = q = L;
for (int i = 0; i < k; i++)
    q = q->next;    // 假设链表长度 >= k
while (q != NULL)
{
    p = p->next;
    q = q->next;
}
输出 p
```

3.5.2 散列表

和数组中一样，都是额外定义数组保存中间过程，比如 $\text{count}[n]$ 表示链表中权值为 n 的元素出现的次数。

例题 2-3：数组 A 中有 n 个非负元素，每个元素绝对值 $< n$ ，其中只有一个数重复出现过，请输出这个数。如： $n=6$ ， $A=\{1, 2, 5, 2, 4, 3\}$

暴力解：枚举 优化解：散列表 解析：关键信息“绝对值 $< n$ ”，可使用散列表

```
// 链表做法
void Hash_Linklist(Node* L)
{
    int count[n] = {0};
    Node* p = L->next;
    while (p != NULL)
    {
        // visit(p);
        if (count[p->data] > 0)
            cout << p->data;
        count[p->data]++; // 要注意p->data是否可能超过数组下标
        p = p->next;
    }
}

// 顺序表做法
void Hash_Sequence(int a[], int n)
{
    int count[n] = {0};
    for (int i = 0; i < n; i++)
    {
        if (count[a[i]] > 0)
            cout << a[i];
        count[a[i]]++; // 要注意a[i]是否可能超过数组下标
    }
}
```

在使用时一定要考虑 $a[i]$ 的大小是否会超出 count 下标的范围

3.5.3 逆置

逆置就是要将线性表反转，顺序完全反过来。接下来我们会依次讨论顺序表的逆置、链表的逆置。

顺序表的逆置

顺序表的逆置很简单，依次将左侧第 1 个元素 ($A[0]$) 和右侧第 1 个元素 ($A[n-1]$) 交换、左侧第 2 个元素 ($A[1]$) 和右侧第 2 个元素 ($A[n-2]$) 交换.....

```
for (int i = 0; i < n/2; i++)
    swap(A[i], A[n-i-1]);
```

链表的逆置

链表因为无法实现随机访问，所以逆置问题无法采用顺序表相同的方式，但可以借助头插法重新插入所有结点。头插法的特点是后插入的结点先访问，这样重新插入一遍就实现了逆置的功能。

设置两个指针：p 指向当前正在处理的结点，temp 指向下一个处理的结点，每次把 p 头插法插入到头结点 L 后面，然后处理下一个结点，更新 p、temp。具体的写法有以下两种，我更推荐大家使用第二种。

第一种写法：

```
// 逆置操作
void Head_Insert(ListNode* L, ListNode* p)
{
    p->next = L->next;
    L->next = p;
}

void Reverse(ListNode* L)
{
    ListNode* p = L->next;
    L->next = NULL;
    while (p != NULL)
    {
        ListNode* temp = p->next;
        Head_Insert(L, p);
        p = temp;    // 更新p，指向下一个待处理结点
    }
}
```

如果让 Head_Insert() 过程能够返回下一个待处理结点地址，就有了第二种写法：

```

// 逆置操作
ListNode* Head_Insert(ListNode* L, ListNode* p)
{
    ListNode* temp = p->next;
    p->next = L->next;
    L->next = p;
    return temp;           // 返回原来p的下一个元素
}

void Reverse(ListNode* L)   // 主代码段
{
    ListNode* p = L->next;
    L->next = NULL;
    while (p != NULL)
        p = Head_Insert(L, p); // 更新p, 指向下一个待处理结点
}

```

3.6 章节练习

3.6.1 习题

【习题 3-1】（13分）已知两个长度分别为 m 和 n 带头结点的升序链表 L_1 和 L_2 ，若将它们合并为一个长度为 $m + n$ 的升序链表。请设计一个时间和空间上尽可能高效的算法，返回合并后的带头结点的升序链表，其头结点为 L_3 。结点的结构定义如下：

```
typedef struct ListNode {  
    int val;  
    struct ListNode* next;  
} ListNode;
```

要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用C或C++语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。

【习题 3-2】（13分）已知两个长度分别为m和n带头结点的升序链表L1和L2，若将它们合并为一个长度为m + n的降序链表，头结点为L3。请设计一个时间和空间上尽可能高效的算法，返回合并后的带头结点的降序链表，其头结点为 L3。结点的结构定义如下：

```
typedef struct ListNode {  
    int val;  
    struct ListNode* next;  
} ListNode;
```

要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用C或C++语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。

【习题 3-3】 (13分) 设计一个空间复杂度为 $O(1)$ 且时间上尽可能高效的算法int IsPalindrome(linklist L), 判断一个带头结点的单链表(表头指针为L, 链表长度为n) 除头结点外是否对称, 若对称, 则返回1, 否则返回0。例如, 表(25, 34, 34, 25)和表(25, 3, 25)都是对称的, 表(25, 3, 30)不是对称的。数据类型定义如下:

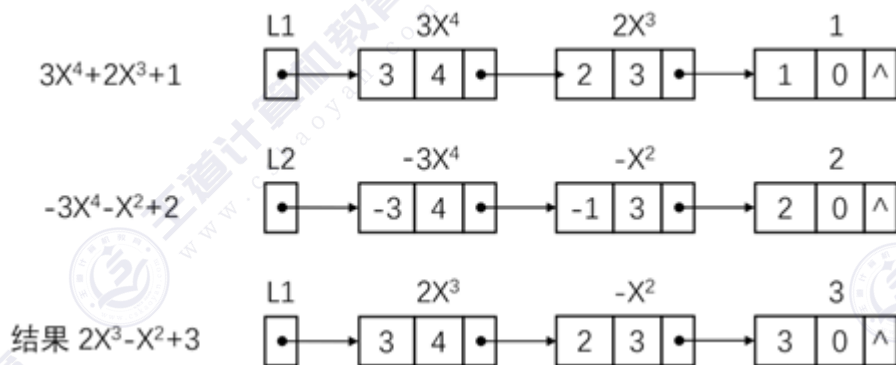
```
typedef struct node{
    int data;
    struct node *next;
}node, *linklist;
```

(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用C或C++语言描述算法, 关键之处给出注释。(3) 说明所设计算法的时间复杂度。

【习题 3-4】 (13分) 一元稀疏多项式用带头结点的单链表L1、L2按指数降序排列（其中L1的最高次项系数 $\geq$ L2的最高次项指数），请设计一个时间上尽可能高效的算法，将这两个多项式相加，结果仍按降序保存在链表中（多项式为0的时候只有头结点），返回头指针L1。结点结构（data表示系数，exp表示指数，next指向下一个结点）：

exp	data	next
-----	------	------

例：多项式 $f_1(x) = 3x^4 + 2x^3 + 1$ ， $f_2(x) = 3x^4 - 2x^2 + 2$ ，
 $f_3(x) = f_1(x) + f_2(x) = 2x^3 - x^2 + 3$ ，链表结构如下：



(1) 写出链表结点的数据

类型定义 (2) 采用C或C++语言描述算法，关键之处给出注释。 (3) 说明所设计算法的时间复杂度。

【习题 3-5】（13分）设有两个从小到大排序（无重复元素）的带头结点的有序链表L1和L2。设计一个空间复杂度为O(1)且时间上尽可能高效的算法，求这两个链表交运算的算法(即 $L1 \cap L2$)。要求结果链表仍是从小到大排序，但无重复元素。

```
typedef struct node{
    int data;
    struct node *next;
}node, *LinkList;
```

(1) 给出算法的基本设计思想。(2) 根据设计思想，采用C或C++语言描述算法，关键之处给出注释。(3) 说明所设计算法的时间复杂度。

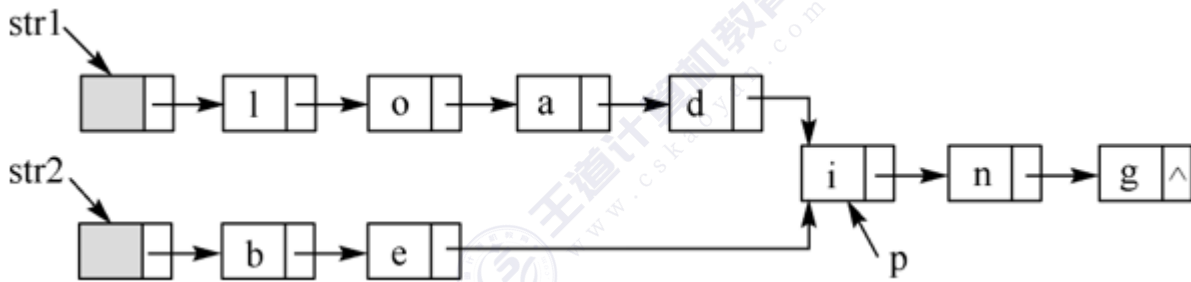
3.6.2 真题

【2009 年 42 题】（15 分）已知一个带有表头结点的单链表，结点结构为

data	link
------	------

假设该链表只给出了头指针 list。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第 k 个位置上的结点（k 为正整数）。若查找成功，算法输出该结点的 data 域的值，并返回 1；否则，只返回 0。要求：（1）描述算法的基本设计思想。（2）描述算法的详细实现步骤。（3）根据设计思想和实现步骤，采用程序设计语言描述算法（使用 C、C++ 或 Java 语言实现），关键之处请给出简要注释。

【2012 年 42 题】 42. 假定采用带头结点的单链表保存单词，当两个单词有相同的后缀时，则可共享相同的后缀存储空间，例如，“loading”和“being”的存储映像如下图所示。



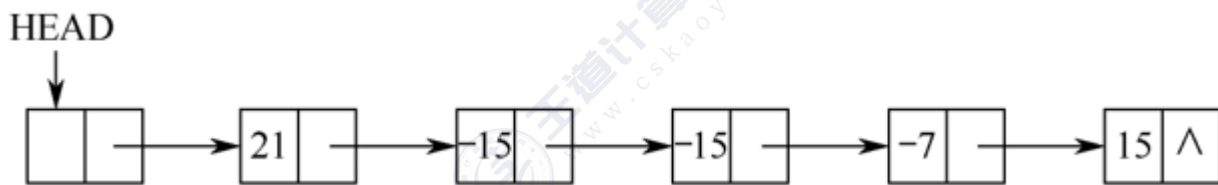
设

str1 和 str2 分别指向两个单词所在单链表的头结点，链表结点结构为：

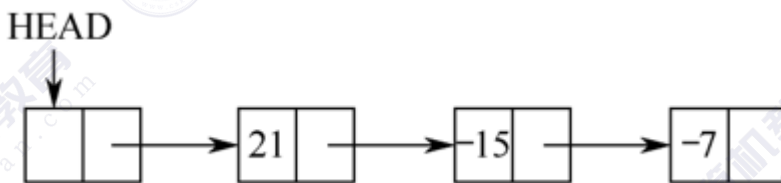
data	next
------	------

请设计一个时间上尽可能高效的算法，找出由 str1 和 str2 所指向两个链表共同后缀的起始位置（如图中字符 i 所在结点的位置 p）。要求：1) 给出算法的基本设计思想。2) 根据设计思想，采用 C 或 C++ 或 Java 语言描述算法，关键之处给出注释。3) 说明你所设计算法的时间复杂度。

【2015 年 41 题】 (15 分) 用单链表保存 m 个整数，结点的结构为 $[data, link]$ ，且 $|data| \leq n$ (n 为正整数)。现要求设计一个时间复杂度尽可能高效的算法，对于链表中 $data$ 的绝对值相等的结点，仅保留第一次出现的结点而删除其余绝对值相等的结点。例如，若给定的单链表 HEAD 如下：



则



删除结点后的 HEAD 为

要求：

- (1) 给出算法的基本设计思想。
- (2) 使用 C 或 C++ 语言，给出单链表结点的数据类型定义。
- (3) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
- (4) 说明你所设计算法的时间复杂度和空间复杂度。

【2019 年 41 题】 (13 分) 设线性表 采用带头结点的单链表保存, 链表中的结点定义如下:

```
typedef struct node {  
    int data;  
    struct node* next;  
} NODE;
```

请设计一个空间复杂度为 $O(1)$ 且时间上尽可能高效的算法, 重新排列 L 中的各结点, 得到线性表 $L = (a_1, a_n, a_2, a_{n-1}, a_3, a_{n-2} \dots)$ 。要求: (1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(3) 说明你所设计的算法的时间复杂度。

3.7 章节答案

3.7.1 习题

习题 3-1

(13分) 已知两个长度分别为 m 和 n 带头结点的升序链表 L_1 和 L_2 ，若将它们合并为一个长度为 $m + n$ 的升序链表。请设计一个时间和空间上尽可能高效的算法，返回合并后的带头结点的升序链表，其头结点为 L_3 。结点的结构定义如下：

```
typedef struct ListNode {  
    int val;  
    struct ListNode* next;  
} ListNode;
```

要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用C或C++语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举法	$O((m+n)n)$	$O(1)$	9
优化解	多指针法	$O(m+n)$	$O(1)$	13

【解法1】枚举法

（1）枚举 L_2 中的每个结点，将其插入到 L_1 中正确的位置，始终保持 L_1 的升序特性。每次插入时从 L_1 头部开始线性查找插入位置。（2）代码如下：

```
// 查找并插入p  
void Find_Insert(ListNode *pre, ListNode *p)  
{  
    while (pre->next!=NULL && pre->next->val<p->val)    // 查找插入位置  
        pre=pre->next;  
    p->next=pre->next;  
    pre->next=p;    // 将p插入到pre之后  
}  
  
ListNode* ans(ListNode *L1, ListNode *L2)  
{  
    ListNode* p=L2->next;    // L2跳过头结点  
    while (p!=NULL){  
        ListNode* temp=p->next;    // temp暂存下一个结点  
        Find_Insert(L1, p);  
        p=temp;  
    }  
}
```

```
    return L1;
}
```

(3) 时间复杂度: $O(mn)$ 空间复杂度: $O(1)$

【解法2】使用多指针法, 利用链表的升序特性合并。

(1) 同时遍历两个链表, 逐个比较结点的值, 将较小的结点尾插法插入到新链表L3中, 直到其中一个链表遍历完为止。然后将未遍历完的链表的剩余部分直接连接到新链表L3的尾部, 返回L3就可以得到合并后的升序序列。(2) 代码如下:

```
ListNode* ans(ListNode *L1, ListNode *L2)
{
    ListNode* L3=(ListNode*)malloc(sizeof(ListNode)); // L3作为新链表的起点
    L3->next=NULL;
    L1=L1->next; // L1跳过头结点
    L2=L2->next; // L2跳过头结点
    ListNode* p=L3;
    while (L1!=NULL && L2!=NULL){
        if (L1->val<=L2->val){
            p->next=L1;
            L1=L1->next;
        }
        else{
            p->next=L2;
            L2=L2->next;
        }
        p=p->next;
    }
    /* 将链表剩余部分直接链接到新链表L3的尾部 */
    if (L1!=NULL)
        p->next=L1;
    else
        p->next=L2;
    return L3;
}
```

(3) 时间复杂度: $O(m + n)$ 空间复杂度: $O(1)$

习题 3-2

(13分) 已知两个长度分别为 m 和 n 带头结点的升序链表 $L1$ 和 $L2$, 若将它们合并为一个长度为 $m + n$ 的降序链表, 头结点为 $L3$ 。请设计一个时间和空间上尽可能高效的算法, 返回合并后的带头结点的降序链表, 其头结点为 $L3$ 。

结点的结构定义如下:

```
typedef struct ListNode {
    int val;
    struct ListNode* next;
} ListNode;
```

要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用C或C++语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举法+转数组	$O((m+n)^2)$	$O(m+n)$	9
优化解	多指针法+头插法	$O(m+n)$	$O(1)$	13

本题的枚举法暴力解和上一题类似，不再给出

【解法1】枚举法+转数组

（1）首先将链表L2的结点依次取下来，按照升序查找并插入到链表L1中得到升序链表。然后将升序链表L1转入数组中，再将数组逆置，然后存回链表中。（2）代码如下：

```
// 查找并插入p
void Find_Insert(ListNode *pre, ListNode *p)
{
    while (pre->next!=NULL && pre->next->val<p->val) // 查找插入位置
        pre=pre->next;
    p->next=pre->next;
    pre->next=p; // 将p插入到pre之后
}

ListNode* ans(ListNode *L1, ListNode *L2)
{
    ListNode* p=L2->next; // L2跳过头结点
    while (p!=NULL)
    {
        ListNode* temp=p->next; // temp暂存下一个结点
        Find_Insert(L1, p);
        p=temp;
    }
    p=L1->next;
    int len=0;
    int a[max_nm]; // max_nm是m+n之和的最大值
    while (p!=NULL)
    {
        a[len]=p->val;
        len++;
        p=p->next;
    }
    p=L1->next;
    for (int i=len-1;i>=0;i--) // 逆序写回链表
```

```

{
    p->val=a[i];
    p=p->next;
}
return L1;
}

```

(3) 时间复杂度: $O((m+n)n)$ 空间复杂度: $O(m+n)$

【解法2】多指针法+头插法：利用链表的升序特性合并，插入时使用头插法得到降序序列。

(1) 采用归并思想，每次比较L1与L2当前元素值，将较小的结点直接头插到新链表L3中。头插法本身天然实现了逆序，无需额外逆置操作，最后即可得到降序链表。

(2) 代码如下：

```

ListNode* Head_Insert(ListNode* L, ListNode* p)
{
    ListNode* temp=p->next;
    p->next=L->next;
    L->next=p;
    return temp;
}

ListNode* Merge(ListNode *L1, ListNode *L2)
{
    ListNode* L3=(ListNode*)malloc(sizeof(ListNode)); // L3是新链表的起点
    L3->next=NULL;
    L1=L1->next; // L1跳过头结点
    L2=L2->next; // L2跳过头结点
    while (L1!=NULL && L2!=NULL)
        if (L1->val<=L2->val)
            L1=Head_Insert(L3, L1);
        else
            L2=Head_Insert(L3, L2);
    // 将链表剩余结点依次头插法插入新链表L3中
    while (L1!=NULL)
        L1=Head_Insert(L3, L1);
    while (L2!=NULL)
        L2=Head_Insert(L3, L2);
    return L3;
}

```

(3) 时间复杂度: $O(m+n)$ 空间复杂度: $O(1)$

习题 3-3

(13分) 设计一个空间复杂度为 $O(1)$ 且时间上尽可能高效的算法int IsPalindrome(linklist L), 判断一个带头结点的单链表（表头指针为L，链表长度为n）除头结点外是否对称，若对称，

则返回1，否则返回0。例如，表(25, 34, 34, 25)和表(25, 3, 25)都是对称的，表(25, 3, 30)不是对称的。数据类型定义如下：

```
typedef struct node{
    int data;
    struct node *next;
}node, *linklist;
```

(1) 给出算法的基本设计思想。(2) 根据设计思想，采用C或C++语言描述算法，关键之处给出注释。(3) 说明所设计算法的时间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举法	$O(n^2)$	$O(1)$	9
优化解	多指针法+逆置	$O(n)$	$O(1)$	13

本题要求空间复杂度为 $O(1)$ ，所以不能使用转数组的方法。

【解法1】枚举法

- (1) 先遍历链表统计总共有多少个元素，然后两层循环依次枚举前后每一对元素比较。
(2) 代码如下：

```
int IsPalindrome(linklist L)
{
    if (L==NULL || L->next==NULL) // 链表为空或一个结点时
        return 1;
    linklist p=L->next;
    int len=0;
    while (p!=NULL)
    {
        len++;
        p = p->next;
    }
    linklist left=L, right;
    for (int i=1;i<=len/2;i++)
    {
        left=left->next;
        right=L;
        for (int j=1; j<=len-i+1; j++)
            right=right->next;
        if (left->data!=right->data)
            return 0;
    }
    return 1;
}
```

(3) 时间复杂度： $O(n^2)$

【解法2】多指针法+逆置

(1) ①使用快慢指针找到链表的中点 (slow指向后半段的前一个位置)。②翻转链表的后半段。③同时遍历前半段和翻转后的后半段，逐个比较结点的值。④如果所有结点的值都相等，则链表是对称的，返回1；否则，链表不是对称的，返回0。(2) 代码如下：

```
// 头插法: 将 p 插入到以 L 为头结点的链表开头
linklist Head_Insert(linklist L, linklist p)
{
    ListNode* temp=p->next;
    p->next=L->next;
    L->next=p;
    return temp;
}

int IsPalindrome(linklist L)
{
    if (L==NULL || L->next==NULL) // 链表为空或一个结点时
        return 1;
    linklist fast=L->next, slow=L->next;
    // 使用快慢指针找到中点 slow, 使得 slow->next 是后半段起点
    while (fast->next!=NULL && fast->next->next!=NULL)
    {
        fast=fast->next->next;
        slow=slow->next;
    }
    linklist p2=slow->next; // 后半段从slow->next开始
    linklist p1=L->next; // 前半段从开始, 到slow结束
    slow->next=NULL;
    // 翻转链表的后半段
    linklist L2=(linklist)malloc(sizeof(node)); // 后半段链表头
    L2->next=NULL;
    while (p2!=NULL)
        p2=Head_Insert(L2,p2);
    p2=L2->next; // p2指向L2的第一个元素
    // 比较前半段和翻转后的后半段的结点值
    while (p1!=NULL && p2!=NULL)
    {
        if (p1->data!=p2->data)
            return 0;
        p1=p1->next;
        p2=p2->next;
    }
    return 1;
}
```

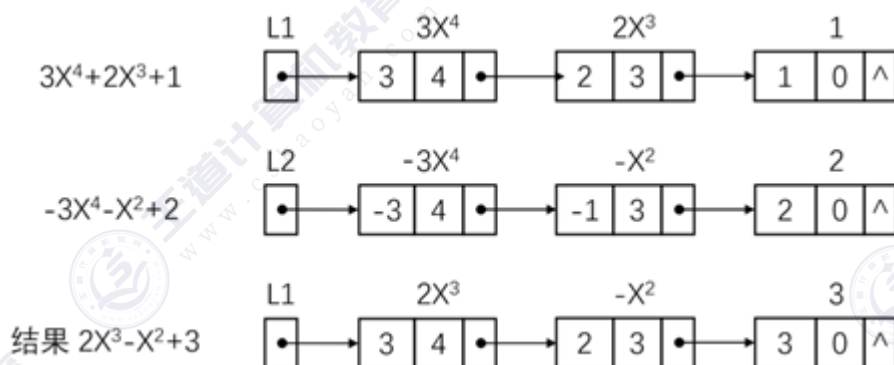
(3) 时间复杂度: $O(n)$

习题 3-4

(13分) 一元稀疏多项式用带头结点的单链表L1、L2按指数降序排列（其中L1的最高次项系数 $\geq$ L2的最高次项指数），请设计一个时间上尽可能高效的算法，将这两个多项式相加，结果仍按降序保存在链表中（多项式为0的时候只有头结点），返回头指针L1。结点结构（data表示系数，exp表示指数，next指向下一个结点）：

exp	data	next
-----	------	------

例：多项式 $f_1(x) = 3x^4 + 2x^3 + 1$ ， $f_2(x) = 3x^4 - 2x^2 + 2$ ， $f_3(x) = f_1(x) + f_2(x) = 2x^3 - x^2 + 3$ ，链表结构如下：



(1) 写出链表结点的数据

类型定义 (2) 采用C或C++语言描述算法，关键之处给出注释。 (3) 说明所设计算法的时间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举法	$O(mn)$	$O(1)$	9
优化解	多指针法	$O(m + n)$	$O(1)$	13

【解法1】枚举法

对 L2 的每一项从头遍历，尝试将其插入到 L1 中合适的位置；

(1) 定义：

```
typedef struct Node{
    int data;
    int exp;
    struct Node *next;
}Node, *LinkList;
```

(2) 代码如下：

```
void Find_Insert(LinkList pre, LinkList p) // 查找并插入p
{
```

```

while (pre->next!=NULL && pre->next->exp>p->exp)    // 查找插入位置
    pre=pre->next;
LinkedList temp=pre->next;                          // temp是pre后一个位置
// 将p插入到pre后面
if (temp==NULL || temp->exp<p->exp)
{
    p->next=temp;
    pre->next=p;
    return;
}
// 次数相等，合并同类型
temp->data+=p->data;
if (temp->data==0)                                // 如果相加后系数=0则删除temp结点
    pre->next=temp->next;
}
LinkedList ans(LinkedList L1, LinkedList L2)
{
    LinkedList p=L2->next;                          // p2遍历链表L2
    while (p!=NULL)
    {
        LinkedList temp=p->next;
        Find_Insert(L1, p);
        p=temp;
    }
    return L1;
}

```

(3) 时间复杂度: $O(mn)$

【解法2】多指针法

设两个指针 $p1$ 和 $p2$ 分别遍历两个链表 $L1$ 和 $L2$ ，另设一个 $pre1$ 作为 $p1$ 的前驱指针，通过同时比较两边结点的指数大小来进行归并操作，具体策略如下：

- 若 $p1->exp > p2->exp$ ，说明 $L1$ 当前项指数更高， $p1$ 向后推进；
- 若 $p1->exp < p2->exp$ ，说明 $L2$ 当前项指数更高，将 $p2$ 插入到 $pre1$ 与 $p1$ 之间；
- 若指数相等，直接合并系数，若合并后系数为 0，删除该项。

剩余的 $L2$ 结点（未合并完）可直接接到 $L1$ 尾部。

(1) 定义：

```

typedef struct Node{
    int data;
    int exp;
    struct Node *next;
}Node, *LinkedList;

```

(2) 代码如下：

```

void Insert(Linklist pre, Linklist p)
{
    p->next=pre->next;
    pre->next=p;
}

Linklist ans(Linklist L1, Linklist L2)
{
    Linklist p1=L1->next;          // p1遍历L1表
    Linklist p2=L2->next;          // p2遍历L2表
    Linklist pre1=L1;              // pre1是p1前一个结点
    while (p1!=NULL && p2!=NULL)
    {
        if (p1->exp>p2->exp)        // p1的指数更高则继续遍历L1
        {
            pre1=p1;
            p1=p1->next;
        }
        else if (p1->exp<p2->exp)    // p1的指数更低则将p2插入pre1后
        {
            Linklist temp=p2->next;
            Insert(pre1,p2);
            pre1=p2;
            p2=temp;
        }
        else                        // p1、p2指数相同则相加并判断是否删除
        {
            p1->data+=p2->data;
            if (p1->data==0)          // 若为0则删除p1结点
                pre1->next=p1->next;
            else
                pre1=p1;              // 若不为0则更新pre1
            p1 = p1->next;
            p2 = p2->next;
        }
    }
    if (p2!=NULL)
        pre1->next=p2;              // 如果链表L2还未访问完则接到pre1后面
    return L1;
}

```

(3) 时间复杂度: $O(m + n)$

习题 3-5

(13分) 设有两个从小到大排序(无重复元素)的带头结点的有序链表L1和L2。设计一个空间复杂度为 $O(1)$ 且时间上尽可能高效的算法, 求这两个链表交运算的算法(即 $L1 \cap L2$)。要求结果链表仍是从小到大排序, 但无重复元素。

```

typedef struct node{
    int data;

```

```
struct node *next;
}node, *LinkList;
```

(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用C或C++语言描述算法, 关键之处给出注释。(3) 说明所设计算法的时间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举法	$O(n^2)$	$O(1)$	9
优化解	多指针法	$O(n)$	$O(1)$	13

本题要求空间复杂度为 $O(1)$, 所以不能使用转数组的方法。

【解法1】枚举法

(1) 两层循环, 外层循环遍历链表L1每一个结点, 内层枚举链表L2来判断L1中的每个结点是否出现过, 如果L1中的某个结点未出现过则删除。(2) 代码如下:

```
LinkList ans(LinkList L1, LinkList L2)
{
    LinkList p1=L1->next;          // p1遍历L1表
    LinkList pre1=L1;              // pre1是p1前一个结点
    while (p1!=NULL)
    {
        LinkList p2=L2->next;      // p2遍历L2表
        while (p2!=NULL)
        {
            if (p1->data==p2->data) break;
            p2=p2->next;
        }
        if (p2==NULL)
            pre1->next=p1->next;    // 没出现过则删除结点p1, pre1不需要更新
        else
            pre1=pre1->next;        // 出现过则不删除结点p1, pre1后移
        p1=p1->next;
    }
    return L1;
}
```

(3) 时间复杂度: $O(n^2)$

【解法2】多指针法

(1) 指针p1、p2分别遍历链表L1和L2, pre保存p1前一个结点, 每次比较p1和p2: ①如果 $p1->data == p2->data$, 说明找到了相同的结点, 保留p1结点, 然后p1、p2同时后移; ②如果

$p1 \rightarrow data > p2 \rightarrow data$, 说明 $p2 \rightarrow data$ 在 $p1$ 中不会出现, 并将 $p2$ 后移; ③如果 $p1 \rightarrow data < p2 \rightarrow data$, 说明 $p1 \rightarrow data$ 在 $p2$ 中不会出现, 删除 $p1$ 结点, 并将 $p1$ 后移。继续按照上述步骤遍历, 直到遍历完一个链表。如果链表 $L1$ 还未遍历完, 则剩下的元素也不需要, 删除 $p1$ 及之后的元素, 最后返回 $L1$ 。

(2) 代码如下:

```
LinkList ans(LinkList L1, LinkList L2)
{
    LinkList p1=L1->next;          // p1遍历L1表
    LinkList p2=L2->next;          // p2遍历L2表
    LinkList pre1=L1;              // pre1是p1前一个结点
    while (p1!=NULL && p2!=NULL)
    {
        if (p1->data<p2->data)      // p1->data小则删除p1结点, p1前移继续寻找
        {
            pre1->next=p1->next;
            p1=p1->next;
        }
        else if (p1->data>p2->data) // p2->data 小, p2前移继续寻找
            p2=p2->next;
        else
            // p1、p2的data相同则保留p1结点, p1、p2都前移继续寻找
            {
                pre1=p1;            // 更新pre1
                p1=p1->next;
                p2=p2->next;
            }
    }
    while (p1 != NULL)
    {
        pre1->next = p1->next;
        p1 = pre1->next;
    }
    return L1;
}
```

(3) 时间复杂度: $O(m + n)$

3.7.2 真题

2009 年 42 题

(15分) 已知一个带有表头结点的单链表，结点结构为

data	link
------	------

假设该链表只给出了头指针list。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第k个位置上的结点（k为正整数）。若查找成功，算法输出该结点的data域的值，并返回1；否则，只返回0。要求：（1）描述算法的基本设计思想。（2）描述算法的详细实现步骤。（3）根据设计思想和实现步骤，采用程序设计语言描述算法（使用C、C++或Java语言实现），关键之处请给出简要注释。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举	$O(n^2)$	$O(1)$	11
暴力解	链表转数组	$O(n)$	$O(n)$	13
优化解	遍历链表两次	$O(n)$	$O(1)$	15
优化解	链表操作，前后指针	$O(n)$	$O(1)$	15

【解法1】暴力解：枚举

（1）两层循环，外层是遍历i，内层是找到倒数第i个元素，比如上一轮查找到倒数第i-1个结点是q，那下一轮当p->link == q时p就是倒数第i个结点。（2）算法如下：

```
int ans(node* list, int k)
{
    node* p;
    node* q=NULL;          // q=NULL是倒数第0个结点
    for (int i=1; i<=k; i++)
    {
        p=list->link;
        if (q==p) return 0; // 找不到倒数第k个结点
        while (p->link!=q)
            p=p->link;      // p最终是倒数第i个结点
        q=p;
    }
    printf("%d\n", p->data); // 输出倒数第k个结点的值
}
```

```
    return 1;
}
```

(3) 时间复杂度: $O(n^2)$ 空间复杂度: $O(1)$

【解法2】优化解：链表转数组 (1) 遍历链表，将所有结点都存放在数组中，最后输出数组中倒数第k个元素即可。 (2) 算法如下：

```
int ans(node* list, int k)
{
    node* A[maxn];           // 结点数组
    node* p=list->link;       // 正在遍历的结点
    int n=0;                  // 数组中结点个数
    while (p!=NULL)
    {
        A[n++]=p;
        p=p->link;
    }
    if (n<k) return 0;
    printf("%d\n", A[n-k]->data); // 输出倒数第k个元素的值
    return 1;
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(n)$

【解法3】优化解：遍历链表两次

(1) 要找到倒数第k个结点，但链表不能回退，所以需要知道他是从前往后数第t个结点（不算头结点）， $t=n-k+1$ ，n是总结点数，那我们只需要先遍历一次链表求出n，然后再遍历一次链表找到第t个结点。 (2) 算法如下：

```
int ans(node* list, int k)
{
    node* p=list->link;
    int t, n=0;
    while (p!=NULL)
    {
        n++;
        p=p->link;
    }
    t=n-k+1;
    if (t<=0) return 0;
    p=list; // 从头结点开始往后移t次
    for (int i=0; i<t; i++)
        p=p->link;
    printf("%d\n", p->data); // 输出倒数第k个元素的值
}
```

```
    return 1;
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(1)$

【解法4】链表操作，前后指针

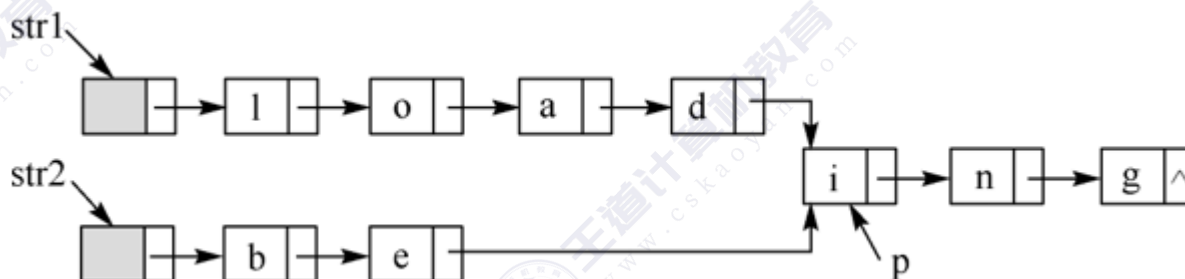
(1) 初始时设置两个指针left和right都指向头结点，right指针先后移k次，若此过程中left指向了NULL则查找失败，返回0。然后将left和right同时后移，直到right指向NULL为止，因为left和right直接相差k次移动，所以此时left一定指向倒数第k个结点，输出left，返回1。(2) 算法如下：

```
int ans(node* list, int k)
{
    node* left=list;
    node* right=list;
    for (int i=0; i<k; i++)
    {
        right=right->link;
        if (right==NULL) return 0; // 链表长度不足k
    }
    while (right!=NULL)
    {
        left=left->link;
        right=right->link;
    }
    printf("%d\n", left->data); // 输出倒数第k个元素的值
    return 1;
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(1)$

2012 年 42 题

假定采用带头结点的单链表保存单词，当两个单词有相同的后缀时，则可共享相同的后缀存储空间，例如，“loading”和“being”的存储映像如下图所示。



设str1

和str2分别指向两个单词所在单链表的头结点，链表结点结构为：

data

next

请设计一个时间上尽可能高效的算法，找出由str1和str2所指向两个链表共同后缀的起始位置（如图中字符i所在结点的位置p）。要求：1) 给出算法的基本设计思想。2) 根据设计思想，采用C或C++或Java语言描述算法，关键之处给出注释。3) 说明你所设计算法的时间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举	$O(n^2)$	$O(1)$	11
暴力解	链表转数组	$O(n)$	$O(n)$	13
优化解	多指针法	$O(n)$	$O(1)$	15

请注意本题是返回地址，而对于相同后缀来说，只要发现两个链指向了同一个结点，后面的部分也会相同。即地址相同等价于后缀完全相同。

【解法1】暴力解：枚举str1和str2起始位置

- (1) 设str1和str2的长度较大的值是n，两层for循环枚举str1和str2相同后缀的起始位置。
- (2) 算法如下：

```
Node* ans(Node *str1, Node *str2)
{
    Node* p1=str1->next;    // str1的后缀起始位置
    Node* p2;                // 之后循环中会赋初值
    while (p1!=NULL)
    {
        p2=str2->next;
        while (p2!=NULL)
        {
            if (p1==p2)
                return p1; // 后缀完全匹配，返回p1
            p2=p2->next;
        }
        p1=p1->next;
    }
    return NULL;
}
```

- (3) 时间复杂度： $O(n^2)$ 空间复杂度： $O(1)$

【解法2】暴力解：链表转数组，数组保存结点地址

(1) 设 str1 和 str2 的长度较大的值是 n，设置两个新数组 S1 和 S2 分别保存 str1 和 str2 链中的每个结点的地址。(2) 算法如下：

```
Node* ans(Node *str1, Node *str2)
{
    Node* S1[maxn], S2[maxn]; // 保存str1和str2各结点地址, maxn是最大可能长度
    Node* p1=str1->next;      // str1的后缀起始位置
    Node* p2=str2->next;      // str2的后缀起始位置
    int Len1=0, Len2=0;
    while (p1!=NULL)
    {
        S1[Len1]=p1;
        Len1++;
        p1=p1->next;
    }
    while (p2!=NULL)
    {
        S2[Len2]=p2;
        Len2++;
        p2=p2->next;
    }
    // 从尾到头同步比较
    int i = Len1 - 1, j = Len2 - 1;
    while (i >= 0 && j >= 0 && S1[i] == S2[j])
    {
        i--;
        j--;
    }
    if (i == Len1 - 1) return NULL; // 尾部结点都不匹配
    return S1[i + 1];              // 共同后缀起始地址
}
```

(3) 时间复杂度： $O(n)$ 空间复杂度： $O(n)$

【解法3】优化解：多指针法，前后指针

(1) 每个链表设置一个指针，先求出两个链的长度，然后长的链指针往后移动x次（x是长度差的绝对值），使得两链剩余未访问结点数相同，然后两链表的指针同步往后移动比较，直到指向的是同一个结点。(2) 算法如下：

```
Node* ans(Node *str1, Node *str2)
{
    Node* p1=str1->next; // 指针p1用来遍历str1
    Node* p2=str2->next; // 指针p2用来遍历str2
    int Len1=0, Len2=0;
    while (p1!=NULL)
    {
        Len1++;
        p1=p1->next;
    }
    while (p2!=NULL)
    {
```

```

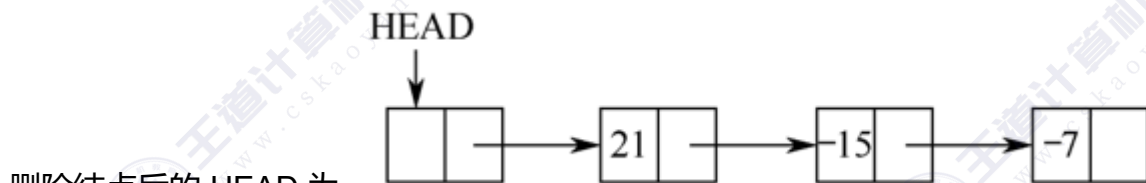
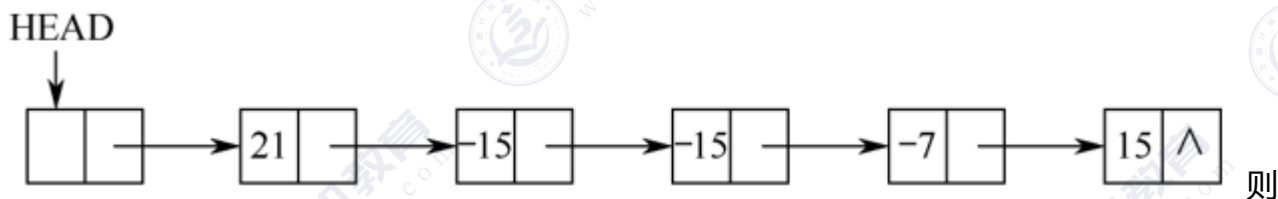
    Len2++;
    p2=p2->next;
}
p1=str1->next;
p2=str2->next;
if (Len1>Len2)
    for (int i=0; i<Len1-Len2; i++) // str1更长, p1先移动一段
        p1=p1->next;
else
    for (int i=0; i<Len2-Len1; i++) // str2更长, p2先移动一段
        p2=p2->next;
while (p1!=NULL && p2!=NULL)
{
    if (p1==p2) return p1; // 返回相同后缀起始地址
    p1=p1->next;
    p2=p2->next;
}
return NULL; // 无相同后缀, 可不写
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(1)$

2015 年 41 题

(15 分) 用单链表保存 m 个整数, 结点的结构为 $[data, link]$, 且 $|data| \leq n$ (n 为正整数)。现要求设计一个时间复杂度尽可能高效的算法, 对于链表中 $data$ 的绝对值相等的结点, 仅保留第一次出现的结点而删除其余绝对值相等的结点。例如, 若给定的单链表 HEAD 如下:



删除结点后的 HEAD 为

要求:

(1) 给出算法的基本设计思想。(2) 使用 C 或 C++ 语言, 给出单链表结点的数据类型定义。(3) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(4) 说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举	$O(m^2)$	$O(1)$	11
优化解	散列表	$O(m)$	$O(n)$	15

【解法1】暴力解：枚举，每一个结点都和其他结点比较

(1) 两层循环枚举所有结点的两两比较，外层遍历链表中的结点p，内层q遍历p之后的结点，将和p绝对值相等的结点删除（只保留第一个出现的p）。

(2) 结点数据结构定义：

```
typedef struct Node{
    int data;           // 该结点权值
    struct Node *link;  // 下一个结点
}Node;
```

(3) 算法如下：

```
void ans(Node* HEAD)
{
    Node* p=HEAD->link;    // 外层遍历结点p
    Node *pre, *r;          // pre是q前一个结点
    // 外层循环固定p，pre从p的下一个结点开始，遍历对比p的数据
    while (p!=NULL)
    {
        pre=p;              // 每次都需要更新pre，从p开始
        while (pre->link!=NULL)
        {
            q=pre->link;    // q表示待比较结点
            if (abs(p->data)==abs(q->data))
            {
                pre->link=q->link;
                free(q);
            }
            else
                pre=pre->link;
        }
        p=p->link;
    }
}
```

(3) 时间复杂度： $O(m^2)$ 空间复杂度： $O(1)$

【解法2】优化解：散列表

(1) data的绝对值 $\leq n$ ，那可以用数组count[0~n]来保存对于每一个绝对值是否出现过，比如count[1]>0表示该绝对值已经出现过了，之后再出现就可以删除当前访问的结点。遍历一次链表，更新count数组以及删除后面重复出现的结点。

(2) 结点数据结构定义：

```
typedef struct Node{
    int data;           // 该结点权值
```

```

    struct Node *link; // 下一个结点
}Node;

```

(3) 算法如下:

```

void ans(Node* HEAD, int m, int n)
{
    Node* p=HEAD->link; // 当前访问结点p
    Node* pre=HEAD; // p的前一个结点pre
    int count[n+1]={0};
    while (p!=NULL)
    {
        if (count[abs(p->data)]==0)
        {
            count[abs(p->data)]=1; // p的权值第一次出现
            pre=p;
            p=p->next;
        }
        else
        {
            // p的权值前面已经出现过,删除p结点
            pre->link=p->link;
            free(p);
            p=pre->link;
        }
    }
}

```

(4) 时间复杂度: $O(m)$ 空间复杂度: $O(n)$

2019 年 41 题

(13 分) 设线性表 采用带头结点的单链表保存, 链表中的结点定义如下:

```

typedef struct node {
    int data;
    struct node* next;
} NODE;

```

请设计一个空间复杂度为 $O(1)$ 且时间上尽可能高效的算法, 重新排列 L 中的各结点, 得到线性表 $L = (a_1, a_n, a_2, a_{n-1}, a_3, a_{n-2} \dots)$ 。要求: (1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(3) 说明你所设计的算法的时间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	枚举, 直接插入	$O(n^2)$	$O(1)$	12
优化解	逆置+多指针法	$O(n)$	$O(1)$	15

【解法1】暴力解（两种写法）

写法一：反复提尾结点插入

(1) p 指向头结点后的第一个结点。每次循环找到链表的尾结点 q 及其前驱 pre ，将 q 摘下插入到 p 之后，然后 p 后移两格。当 p 后面只剩一个结点时（即 $pre == p$ ），该结点已在正确位置，循环结束。(2) 算法如下：

```
void ans(Node* L)
{
    Node *p = L->next;           // p 从头结点开始
    while (p != NULL && p->next != NULL)
    {
        // 找尾结点 q 及其前驱 pre
        Node *pre = p;
        while (pre->next->next != NULL) // pre 走到倒数第二个
            pre = pre->next;
        Node *q = pre->next;           // q 是尾结点
        if (pre == p) break;           // p 后只剩 q，已就位

        // 将尾结点 q 插入到 p 之后
        pre->next = NULL;
        q->next = p->next;
        p->next = q;
        p = q->next;                   // p 后移两格
    }
}
```

(3) 时间复杂度： $O(n^2)$ ，空间复杂度： $O(1)$

写法二：统计长度、折半插入

(1) 先遍历链表统计结点数 n ，将链表分成前半 $(n+1)/2$ 个和后半 $n/2$ 个。再将后半结点依次插入到前半对应位置。(2) 算法如下：

```
void ans(Node* L)
{
    // 统计链表结点数
    Node *p = L->next;
    int n = 0;
    while (p != NULL) { n++; p = p->next; }
    // 链表分成两半，p 指向后半第一个结点
    p = L;
    for (int i = 0; i < (n+1)/2; i++)
        p = p->next;
    Node *temp = p;
    p = p->next;
    temp->next = NULL;
    // 将后半结点依次插入到前半
    for (int i = 0; i < n/2; i++)
    {
        temp = p->next;
```

```

Node *pre = L;
for (int j = 0; j < n/2-i; j++)
    pre = pre->next;
p->next = pre->next;
pre->next = p;
p = temp;
}
}

```

(3) 时间复杂度: $O(n^2)$, 空间复杂度: $O(1)$

【解法2】优化解：逆置 + 多指针法

(1) 步骤如下：

1. 前后指针找到中间结点。
 2. 从中间结点处将链表分成两半，前半是 $(n+1)/2$ 个结点，后半 $n/2$ 个结点。
 3. 使用头插法将链表后半的所有结点逆置。
 4. 将前半链表和逆置后的后半链表交叉排列合并成一个链表。
- (2) 算法如下：

```

// 头插法
void Head_Insert(Node* L, Node* p)
{
    p->next=L->next;
    L->next=p;
}

void ans(Node* L)
{
    // 找中间结点 p
    Node *p, *q;
    p = q = L->next;
    while (p != NULL && q != NULL)
    {
        q = q->next;
        if (q == NULL || q->next == NULL)
            break;
        q = q->next;
        p = p->next;
    }

    // 链表从 p 后分成两半
    Node *temp = p;
    p = p->next;
    temp->next = NULL;

    // 为链表后半设置新的头结点
    Node *L2 = (Node*)malloc(sizeof(Node));
    L2->next = NULL;

    // 将后半结点逆置
    while (p != NULL)

```

```

{
    temp = p->next;
    Head_Insert(L2, p);
    p = temp;
}

Node *p1 = L->next;
Node *p2 = L2->next;

// 将前半链表和逆置后的后半链表交叉合并
while (p2 != NULL)
{
    暂存 p1 和 p2 的下一个结点
    Node* temp1 = p1->next;
    Node* temp2 = p2->next;
    // p2 插入到 p1 后面
    p2->next = p1->next;
    p1->next = p2;
    // 更新 p1 和 p2
    p1 = temp1;
    p2 = temp2;
}
}

```

(3) 时间复杂度: $O(n)$

4 树

由 2014 年真题引入

【2014 年真题，15 分】二叉树的带权路径长度（WPL）是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T ，采用二叉链表存储，结点结构如下：

left	weight	right
------	--------	-------

其中叶结点的 `weight` 域保存该结点的非负权值。设 `root` 为指向 T 的根结点的指针，请设计求 T 的 WPL 的算法，要求：

- 1) 给出算法的基本设计思想。
- 2) 使用 C 或 C++ 语言，给出二叉树结点的数据类型定义。
- 3) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

4.1 树概述

4.1.1 思考流程

树和图的题目一般不会特别要求复杂度，如果没有特别要求的话，考试中只要做出来就可以直接写，不用考虑优化问题。一般都会考察树的遍历，除非特别说明，否则遍历就用递归写，不要写非递归。

4.1.2 考察结构

树和森林

树和森林（孩子兄弟表示法）

firstchild 是指向该结点第一个儿子结点的指针，nextbro 是指向下一个兄弟结点的指针

```
typedef struct TNode
{
    Elemtype data;    // 每个结点的权值
    struct TNode *firstchild, *nextbro;
}TNode;              // 树结点
```

树和森林（结点用双亲表示法存储，常用于并查集）

一般采用数组表示，father 数组保存每个结点的父亲结点在数组中的下标：

```
int father[MAX_SIZE];    // 保存每个结点的父亲结点在数组中的下标
```

或者

```
typedef struct TNode
{
    Elemtype data;          // 每个结点的权值
    struct TNode *father;    // father是指向该结点父亲结点的指针
}TNode;                    // 树结点
```

二叉树

二叉树（二叉链表）

```
typedef struct BTreeNode
{
    Elemtype data;          // 每个结点的权值
    struct BTreeNode *lchild, *rchild; // 指向左右孩子结点的指针
}BTreeNode;                // 二叉树结点
```

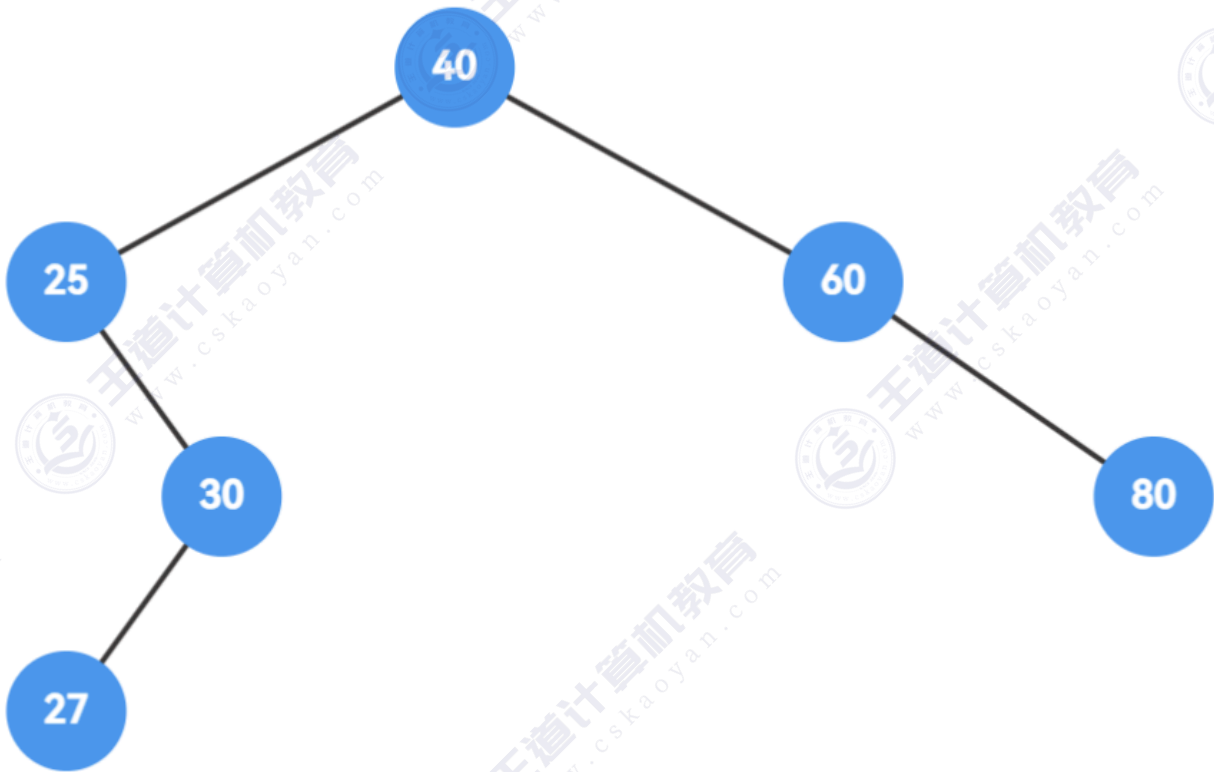
二叉树（顺序）

```
typedef struct
{
    int SqBiTNode[MAX_SIZE]; // 保存二叉树结点值的数组
    int ElemNum;              // 实际占用的数组元素个数
} SqBiTree;
```

对于完全二叉树，只要从根起按层序存储即可，依次自上而下、自左至右存储结点元素，即将完全二叉树上编号为 i 的结点元素存储在如上定义的一维数组中下标为 $i-1$ 的位置。可以用 -1 或者 0 表示对应结点不存在。

T1.SqBiTNode	40	25	60	-1	30	-1	80	-1	-1	27		
--------------	----	----	----	----	----	----	----	----	----	----	--	--

T1.ElemNum = 10

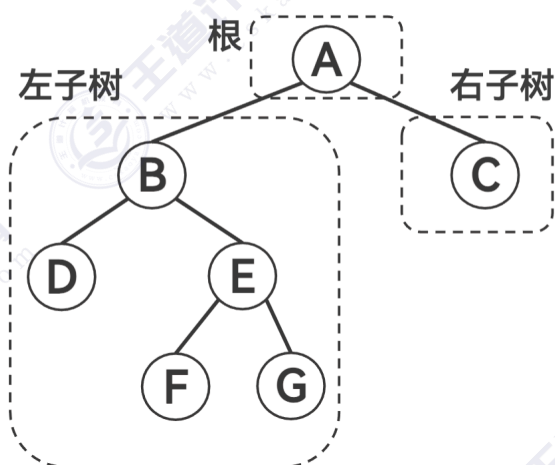


4.2 树的递归遍历

树的遍历是 408 代码题的常考点，包括 2014、2017、2022 年，只要考察树必然需要用到遍历，一般是考察树的遍历+某种树的定义（如二叉搜索树），而二叉树的遍历包括先序中序后序遍历，他们又分为递归和非递归两种方式，每一种遍历的非递归逻辑都完全不一样，复习起来很痛苦也没有必要，而递归的代码逻辑非常简单，所以考试中除了特殊情况，树的遍历请一定使用递归（递归简单好用）。

4.2.1 二叉树的递归遍历

递归，从本质上讲，是 **分治** 思想的一种完美体现。分治策略就是“分而治之”，它将一个复杂的大问题分解成若干个规模更小、但结构与原问题相同的子问题，然后逐个解决。



在二叉树的遍历中，**分**：将“遍历整棵树”这个大问

题，分解为三个更小的问题：

- 访问根结点本身。
- 遍历左子树（一个规模更小的相同问题）。
- 遍历右子树（另一个规模更小的相同问题）。

因此，我们可以把递归想象成一个“任务分包”的过程。你作为“总指挥”，只需要应用分治思想，关注当前结点，并把子树的任务“外包”出去。

处理二叉树我们总共需要完成三个工作：

- 处理好你当前手头的工作（即访问结点 `p` 本身）。
- 把遍历左子树的 **整个任务**，完全甩给你的“左下属”（处理左子树 `order(p->lchild)`）。
- 把遍历右子树的 **整个任务**，完全甩给你的“右下属”（处理右子树 `order(p->rchild)`）。

核心信念：当你调用 `order(p->lchild)` 时，你不需要关心它内部是如何一步步执行的。你只需要**完全信任**这个调用，相信它一旦执行，就一定能帮你把**整个左子树**完美地遍历一遍。同理，`order(p->rchild)` 也一样。

有了这个“信任”，你的工作就变得极其简单：你只需要考虑三件事（访问自己、调用左下属、调用右下属）的执行顺序即可。把复杂的大问题，分解成了在当前结点的一个极小问题。

遍历 order

- 先序遍历 PreOrder:
 1. 访问自己 `visit(p);`
 2. 调用左 `PreOrder(p->lchild);`
 3. 调用右 `PreOrder(p->rchild);`
- 中序遍历 InOrder:
 1. 调用左 `InOrder(p->lchild);`
 2. 访问自己 `visit(p);`
 3. 调用右 `InOrder(p->rchild);`
- 后序遍历 PostOrder:
 1. 调用左 `PostOrder(p->lchild);`
 2. 调用右 `PostOrder(p->rchild);`
 3. 访问自己 `visit(p);`

先访问自己是先序，在中间访问自己是中序，最后访问自己是后序

先中后序的递归遍历时间复杂度为 $O(n)$ ，空间复杂度为 $O(h)$ ， n 是总结点数， h 是树高，如果不确定树高则空间复杂度为 $O(n)$ 。

```
// 先序（根左右）
void PreOrder(BTNode *p)
{
    if (p==NULL) return;
    visit(p); // 对p结点的访问等
    PreOrder(p->lchild);
    PreOrder(p->rchild);
}

// 中序（左根右）
void InOrder(BTNode *p)
{
    if (p==NULL) return;
    InOrder(p->lchild);
    visit(p); // 对p结点的访问等
    InOrder(p->rchild);
}

// 后序（左右根）
void PostOrder(BTNode *p)
{
    if (p==NULL) return;
    PostOrder(p->lchild);
    PostOrder(p->rchild);
    visit(p); // 对p结点的访问等
}
```

4.2.2 树和森林的递归遍历

树和森林的递归遍历本质上是对**孩子兄弟表示法**结构的递归操作，重点在于掌握：

- **firstchild** 表示第一个孩子（相当于“左子树”）。
- **nextbro** 表示兄弟结点（相当于“右子树”）。

二叉树的遍历：

先序遍历：根左右 → 根、子树、下一个兄弟树

中序遍历：左根右 → 子树、根、下一个兄弟树

后序遍历：左右根 → 子树、下一个兄弟树、根

层序遍历：从上到下，从左到右，层层遍历



二叉树转树的口诀：左孩子，右兄弟

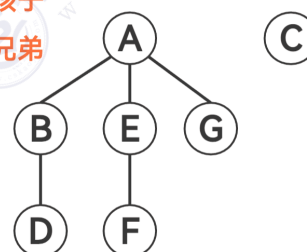
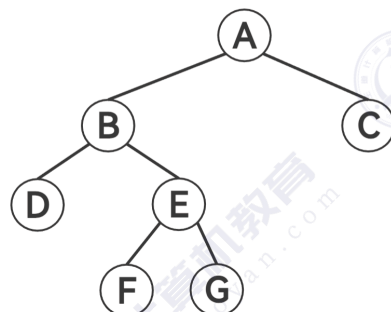
二叉树中x的左孩子是原树中x的第一个孩子

二叉树中x的右孩子是原树中x的下一个兄弟

树或森林的遍历：

先根遍历：先访问根，然后依次访问子树

后根遍历：先依次访问子树，然后访问根



即“左孩子右兄弟”。

也就是说，这种结构可以转化为“二叉树”模型来理解递归逻辑。

- 树和森林的先根遍历等价于二叉树的先序遍历。
- 树和森林的后根遍历等价于二叉树的中序遍历。

先根遍历（等价于先序遍历）：

```
void TreePreOrder(TNode *T)
{
    if (T == NULL) return;
    visit(T);           // 访问当前结点
    TreePreOrder(T->firstchild); // 递归访问第一个孩子
    TreePreOrder(T->nextbro);   // 递归访问兄弟结点
}
```

后根遍历（孩子都访问完再访问根，等价于中序遍历）：

```
void TreePostOrder(TNode *T)
{
    if (T == NULL) return;
    TreePostOrder(T->firstchild); // 先访问所有孩子
    visit(T);
```

```
visit(T);           // 最后访问根
TreePostOrder(T->nextbro); // 然后访问兄弟结点
}
```

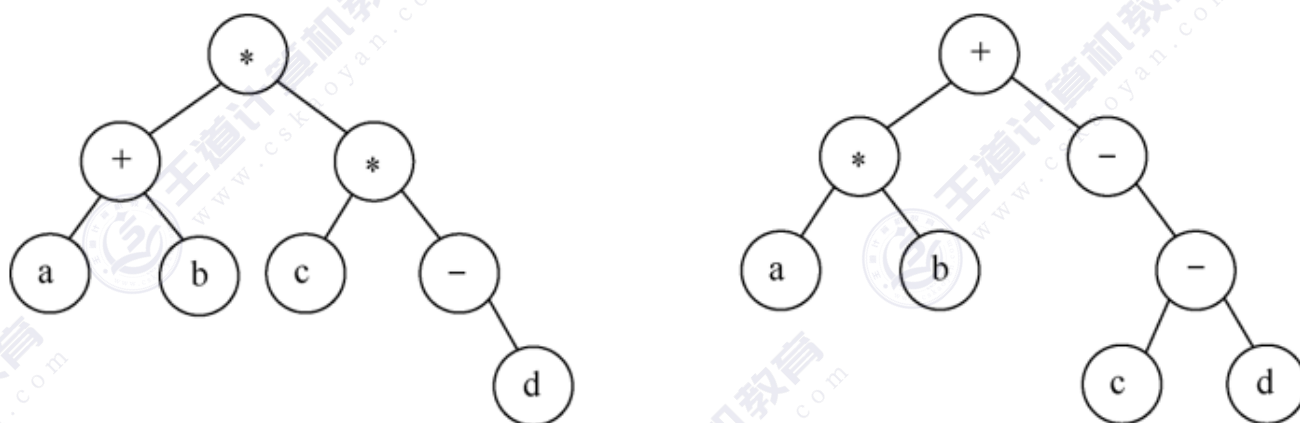
关键还是掌握二叉树的三种遍历，在二叉树遍历的基础上稍加修改就得到树和森林的遍历

4.3 树的做题方法

408 中树算法考察的核心是树的遍历，所以在做题应确定遍历，以对应遍历作为框架做题。建议分成四步：审题——写框架——填充——调整，其中第一步是最重要的。

请阅读 2017 年 41 题

(15分) 请设计一个算法，将给定的表达式树（二叉树）转换为等价的中缀表达式（通过括号反映操作符的计算次序）并输出。例如，当下列两棵表达式树作为算法的输入时，输出的等价中缀表达式分别为 $(a+b) \times (c \times (-d))$ 和 $(a \times b) + (-(c-d))$ 。



二叉树结点定义如下：

```
typedef struct node{
    char data[10]; // 存储操作数或操作符
    struct node *left, *right;
}BTree;
```

4.3.1 审题并分析

根据题目的要求，从根结点开始想一下按递归的方式需要当前结点做什么？然后根据访问顺序确定 x 序遍历。确定思路以后再回头看题检查是否满足了题目要求。

选择遍历方式的依据：处理 p 时需要子树的结果吗？不需要则先序；需要则后序；需要有序则中序；需要按层则层序。

- **先序（根→左→右）**：不依赖子树。如统计叶子、复制树、序列化、查找结点。
- **中序（左→根→右）**：利用有序性。如 BST 判合法、求第 k 小、输出中缀表达式。
- **后序（左→右→根）**：依赖子树结果。如求深度、判平衡、删树、求直径。
- **层序（逐层扫描）**：需要层级信息。如最短路径、判完全二叉树、求宽度、按层输出。需队列。

2017 年 41 题 分析：本题需要输出中缀表达式，因此考虑使用中序遍历。在访问每一个结点的时候需要输出该结点的 data 域。对于所有的不为原树也不为叶结点的子树，这种子树对应

的式子两边都需要加括号。

4.3.2 草稿纸上书写框架

写法 1：只写递归过程部分（递归实现遍历，确定是什么序遍历），输出靠递归返回的参数。框架以中序为例：

```
// 需要返回值
int InOrder(BTNode *p) // 返回类型不一定是int
{
    if (p==NULL) return 返回值;
    InOrder(p->lchild); // 调用根据题目修改
    visit(p);           // 是最重要的部分，根据题目要求决定
    InOrder(p->rchild);
    return xx;
}

// 不需要返回值
void InOrder(BTNode *p)
{
    if (p==NULL) return;
    InOrder(p->lchild); // 调用根据题目修改
    visit(p);           // 是最重要的部分，根据题目要求决定
    InOrder(p->rchild);
}
```

如果递归过程即可完成所有功能则使用写法 1，大部分问题都可以使用，建议优先使用。

写法 2：程序分为两个部分：本题主过程部分 ans（负责调用递归过程以及输入输出）、和对应的递归过程部分（递归实现遍历，确定是什么序遍历）。框架以中序为例：

```
int InOrder(BTNode *p)
{
    if (p==NULL) return 返回值;
    InOrder(p->lchild); // 调用根据题目修改
    visit(p);           // 是最重要的部分，根据题目要求决定
    InOrder(p->rchild);
    return 返回值;
}

void ans(BTNode *T)
{
    输出InOrder(T);
    完成其他工作
}
```

当写法 1 无法完全实现功能时，就需要补充主体过程部分，即此处的 ans()。

2017 年 41 题 分析：本题不需要返回参数，框架如下：

```
void InOrder(BTNode *p)
{
    if (p==NULL) return;    // p是空结点
    InOrder(p->lchild);      // 递归调用左子树
    visit(p);
    InOrder(p->rchild);      // 递归调用右子树
}
```

4.3.3 填充 visit(p)

确定对于 p 需要处理什么，并补充 visit(p) 部分

2017 年 41 题 分析：visit(p) 需要输出该结点的 data 域（注意是输出字符数组）。

```
printf("%s", p->data);    // 输出操作数
```

或

```
输出 p->data
```

4.3.4 再次读题并调整

把前面几步得到的基本代码写出来，并比对题目要求，补充或修改对应的语句。

```
void InOrder(BTNode *p)
{
    if (p==NULL) return;    // p是空结点
    InOrder(p->lchild);      // 递归调用左子树
    printf("%s", p->data);    // 输出操作数
    InOrder(p->right, root);  // 递归调用右子树
}
```

对于括号还未处理，需要判断以 p 为根的树是否为原树或者叶结点。如果满足要求则在开头输出左括号，在末尾输出右括号。

- 不为叶结点：if (p->left!=NULL || p->right!=NULL)。
- 不为根：需要主要过程调用时增加一个参数——根 root。if (p!=root)。

调整后：

```
void InOrder(BTree* p, BTree* root) // root是原二叉树的根
{
    if (p==NULL) return;    // p是空结点
    if ((p->left!=NULL || p->right!=NULL) && (p!=root))
        printf("(");
    InOrder(p->left, root);    // 递归调用左子树
    printf("%s", p->data);    // 输出操作数
    InOrder(p->right, root);  // 递归调用右子树
}
```

```

if ((p->left!=NULL || p->right!=NULL) && (p!=root))
    printf("%d");
}

```

4.3.5 返回值或全局变量

很多时候题目需要我们统计一些信息，可以采用函数返回值或者全局变量的方式来统计，有时候两种方法都可以。

以计算二叉树的带权路径长度 WPL 为例：**WPL = 所有（叶结点的权值 W_i ×该结点深度 D_i ）求和。**

全局变量

这种方式更像是在遍历过程中“收集”信息。

- **核心理想**：定义一个在所有递归调用中都共享的变量，在遍历的任何一步，只要满足条件，就直接修改这个共享变量。
- **适用场景**：全局变量是一个全局的累加、计数或状态标记。

```

int WPL = 0; // WPL是全局变量
void PreOrder(BTNode *p, int depth) // 当前结点p，深度depth
{
    if (p == NULL) return; // p是空结点

    // visit(p);
    if (p->left == NULL && p->right == NULL) // p是叶结点
        WPL += depth*p->weight;

    PreOrder(p->left, depth+1); // 递归调用左孩子
    PreOrder(p->right, depth+1); // 递归调用右孩子
}

```

函数返回值

这种方式是“自底向上”传递信息的核心。

- **核心理想**：子问题的解（即对子树的递归调用）通过 `return` 语句返回给父问题，父问题利用这些返回的结果来计算自己的解。
- **适用场景**：某过程的返回值表示的是以该子树为核心的二叉树的对应值。

```

int postOrder(BTNode *p, int depth) // 当前结点p，深度depth
{
    if (p == NULL) return 0; // p是空结点
    int L_WPL = postOrder(p->left, depth+1);
    int R_WPL = postOrder(p->right, depth+1);

    // visit(p);
}

```

```
if (p->left==NULL && p->right==NULL) // p是叶结点
    return depth*p->weight;
else return L_WPL + R_WPL;
}
```

4.3.6 课堂练习

对于下列问题，请思考他们分别应该使用什么序遍历、`visit(p)` 实现什么功能？

例题 4-1：求二叉树的叶子结点个数。

例题 4-2：求二叉树的深度。

例题 4-3：判定二叉树（顺序存储）是否为完全二叉树。

例题 4-4：判定二叉树是否为二叉排序树。

例题 4-5：统计森林的叶结点数。

例题 4-6：判定二叉排序树是否为平衡二叉树。

请大家思考并写下来。

核心思想：如何确定遍历方式与 `visit()` 功能 在处理任何二叉树的算法问题时，你都需要回答两个核心问题：

1. **我应该用哪种遍历顺序？**（先序、中序、后序、层序）
2. **在遍历到每个结点 `p` 时，我需要做什么？**（这就是 `visit(p)` 的功能）

`visit(p)` 的功能： `visit(p)` 是你算法的核心逻辑。它定义了访问到结点 `p` 时，具体要执行的操作。这个操作可能很简单（比如打印结点值），也可能很复杂（比如比较、计算、更新全局变量等）。

例题分析

现在，我们用上面的核心思想来逐一分析每个例题。

例题 4-1：求二叉树的叶子结点个数

- **遍历方式选择：** 判断一个结点 `p` 是否为叶子结点（即 `p->left == NULL && p->right == NULL`），这个判断完全**不依赖于**它的子树信息。因此，先序、中序、后序遍历都可以完成任务。
- **`visit(p)` 功能实现：** 我们需要一个全局计数器（比如 `int leaf_count = 0;`）。
`visit(p)` 的逻辑非常简单：
 1. 检查当前结点 `p` 是否为叶子结点。
 2. 如果是，则 `leaf_count++`。

例题 4-2：求二叉树的深度

- **遍历方式选择：** 一个结点 `p` 的深度（或以此结点为根的子树的高度）是 `1 + max(左子树高度, 右子树高度)`。显然，为了计算 `p` 的深度，你**必须**先知道它左、右子树的深度。这是一个典型的“自底向上”的计算过程，因此必须使用**后序遍历**。

- **visit(p) 功能实现：** 这里的 `visit(p)` 不再是简单地修改全局变量，而是通过函数的返回值来传递信息。整个求深度的递归函数本身就体现了后序遍历的思想。

例题 4-3：判定二叉树（顺序存储）是否为完全二叉树

- **遍历方式选择：** 完全二叉树的定义是按层序从左到右填充的。这天然地指向了**层序遍历**。在遍历过程中，一旦我们遇到了一个空结点，那么之后的所有结点都必须是空结点，否则就不是完全二叉树。**特别注意：** 题目指定了“顺序存储”（即数组）。对于数组表示的二叉树，判断更简单。如果树有 n 个结点，那么它必须占据数组的前 n 个位置，中间不能有“空洞”。我们只需要找到最后一个非空结点的位置 k ，然后检查从 0 到 k 是否都有结点即可。这本质上是一种简化的线性扫描，可以看作层序思想的应用。
- **visit(p) 功能实现 (基于队列的层序遍历)：**
 1. 使用一个队列，根结点入队。
 2. 设置一个标志位 `bool leaf = false;`。
 3. 循环直到队列为空：
 - 结点 p 出队。
 - 如果 `leaf` 为 `true`，但 p 却有子结点（左或右），则该树不是完全二叉树，返回 `false`。
 - 如果 p 有右孩子但没有左孩子，直接返回 `false`。
 - 如果 p 只有左孩子或没有孩子，将 `leaf` 设为 `true`。
 - 将 p 的非空子结点入队。
 4. 循环正常结束，返回 `true`。

例题 4-4：判定二叉树是否为二叉排序树 (BST)

- **遍历方式选择：** 二叉排序树 (BST) 的一个核心性质是：它的**中序遍历**序列是一个严格递增的序列。这是判断 BST 的最直接、最简单的方法。
- **visit(p) 功能实现：** 我们需要一个变量 `pre_value` 来记录中序遍历中“上一个”被访问的结点的值。
 1. 初始化 `pre_value` 为一个极小值（比如 `INT_MIN`）。
 2. 进行中序遍历。
 3. `visit(p)` 的逻辑是：
 - 比较当前结点 `p->value` 和 `pre_value`。
 - 如果 `p->value <= pre_value`，则不满足递增性质，该树不是 BST，返回 `false`。
 - 如果满足，则更新 `pre_value = p->value`，然后继续遍历。
 4. 如果整个遍历都成功，返回 `true`。

例题 4-5：统计森林的叶结点数

- **遍历方式选择：** 森林是多棵树的集合。我们可以对森林中的每一棵树分别进行遍历。对于每一棵树，求其叶子结点数的方法和例题 4-1 完全一样。因此，对每棵树可以使用**任意遍历方式**（先序、中序、后序均可）。

- **visit(p) 功能实现：** 与例题 4-1 完全相同。
 1. 设置一个全局计数器 `leaf_count = 0;`。
 2. 对森林中的每一棵树 `T`，调用求叶子数的函数（该函数内部进行遍历）。
 3. `visit(p)` 的逻辑是：检查 `p` 是否为叶子结点，如果是，则 `leaf_count++`。

例题 4-6：判定二叉排序树是否为平衡二叉树 (AVL)

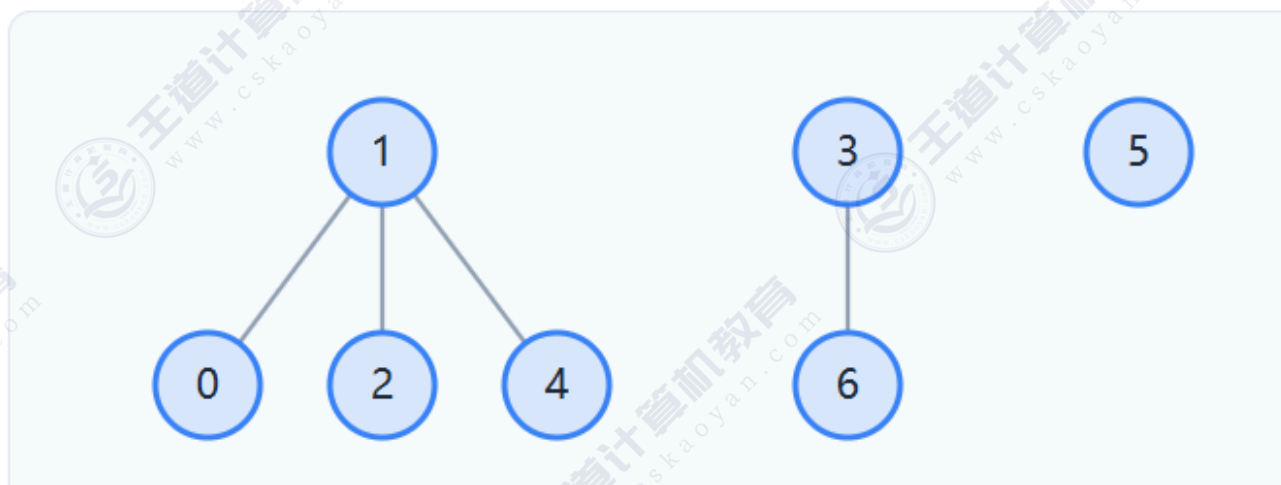
- **遍历方式选择：** 平衡二叉树的定义是：对于任意结点，其左、右子树的高度差的绝对值不超过 1。要判断结点 `p` 是否平衡，你**必须**先知道其左、右子树的高度。这又是一个典型的“自底向上”的计算，因此必须使用**后序遍历**。
- **visit(p) 功能实现：** 这个实现结合了求树的高度（例题 4-2）和判断。我们可以设计一个递归函数，它返回子树的高度；如果发现不平衡，则返回一个特殊值（如 -1）作为标记。

例题	核心问题	最佳遍历方式	visit(p) / 核心操作
4-1: 求叶子数	结点自身属性	先序/中序/后序遍历	判断 <code>p</code> 是否为叶子，是则计数器加一
4-2: 求深度	依赖子树结果	后序遍历	$1 + \max(\text{左子树深度}, \text{右子树深度})$
4-3: 是否完全二叉树	层次结构/无间隙	层序遍历	检查遇到第一个空位后，是否还有非空结点
4-4: 是否二叉排序树	结点值的有序性	中序遍历	比较 <code>p->value</code> 与前一个结点的值，判断是否递增
4-5: 森林叶子数	遍历多棵树	先根/后根遍历	同 4-1，对森林中每棵树分别统计
4-6: 是否平衡二叉树	依赖子树高度	后序遍历	计算左右子树高度差，判断是否 > 1

4.4 并查集

并查集是一种高效支持“集合合并”与“查找代表元”的数据结构，常用于处理图中**连通性判断**、**集合合并问题**，例如判断是否构成树、求连通块数量、判断环等问题。在图那一章中介绍其应用。

Father 数组: [1, 1, 1, 3, 1, 5, 3]



并查集使用数组 `father[]` 存储每个元素的“父结点”（其实是祖先结点），形成森林结构。为了方便使用，通常 `father` 数组定义成全局变量。

- `find(x)`：查找下标为 `x` 的结点所在树的树根下标，可路径压缩（把查找途中所有结点全部变成根结点的儿子，减少树的高度）。
- `Union(x, y)`：将下标为 `x`、`y` 的结点所在树合并。

```
int father[MAXN]; // father[i] 表示i所在集合的代表元（根）
```

```
// 并查集查找（带路径压缩）
```

```
int find(int x)
```

```
{
```

```
    if (father[x] != x)
```

```
        father[x] = find(father[x]);
```

```
    return father[x];
```

```
}
```

```
// 并查集合并
```

```
void Union(int x, int y)
```

```
{
```

```
    int fx = find(x);
```

```
    int fy = find(y);
```

```
    if (fx != fy)
```

```
father[fy] = fx;  
}
```

4.5 二叉搜索树

数据结构定义同二叉链表表示法（命名规则：二叉树是BTNode，二叉搜索树是BSTNode），只是保证权值：左<中<右。

4.5.1 数据结构定义

```
typedef struct BSTNode
{
    ElemType data;
    struct BSTNode *lchild, *rchild;
} BSTNode, *linklist;
```

4.5.2 查找

```
BTNode* BST_Search(BSTNode *T, ElemType x)
{
    BSTNode *p = T;
    while (p != NULL)
    {
        if (x == p->data)
            return p;
        if (x < p->data)
            p = p->lchild;
        else
            p = p->rchild;
    }
    return NULL;
}
```

查找的时间复杂度是 $O(h)$ ， h 是高度，最坏情况就是 $O(n)$ 。对于二叉搜索树，最多也就是考察查找了，但不一定只会查找一次，可能需要查找多次。如果我们实在不会写二叉搜索树的查找怎么办呢，回忆一下：是不是所有树的问题都可以用遍历来做，所以你也可以对二叉搜索树进行遍历查找，时间复杂度也是 $O(n)$ 。

4.6 章节练习

4.6.1 习题

【习题 4-1】 有 n 个结点的完全二叉树的权值存放在一维数组 $A[n]$ 中（从 $A[0]$ 开始存放），试设计一个算法，将该完全二叉树用二叉链表表示，算法返回指向根结点的指针。二叉树结点数据类型定义如下：

```
typedef struct node{
    int data;
```

```
struct node *lchild, *rchild;  
}BTNode;
```

(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(3) 说明所设计算法的时间复杂度。

【习题 4-2】 设计一个算法，将一棵以二叉链表存储的二叉树T按顺序方式存储到一维数组B[] 中（根结点下标为 1，结点的权值 ≥ 0 ，数组值等于 -1 表示该结点不存在）。二叉树结点的数据类型定义如下：

```
typedef struct node{
    int data;
    struct node *lchild, *rchild;
}BTNode;
```

(1) 给出算法的基本设计思想。(2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。(3) 说明所设计算法的时间复杂度。

【* 习题 4-3】 已知由一棵二叉链表表示的二叉树，设计算法，输出二叉树的高度（只有根结点的二叉树高度为 1）。二叉树结点数据类型定义如下：

```
typedef struct node{
    int data;
    struct node *lchild, *rchild;
}BTNode;
```

(1) 给出算法的基本设计思想，关键之处给出注释。(2) 根据设计思想，采用 C 或 C++ 语言描述算法。(3) 说明所设计算法的时间复杂度。

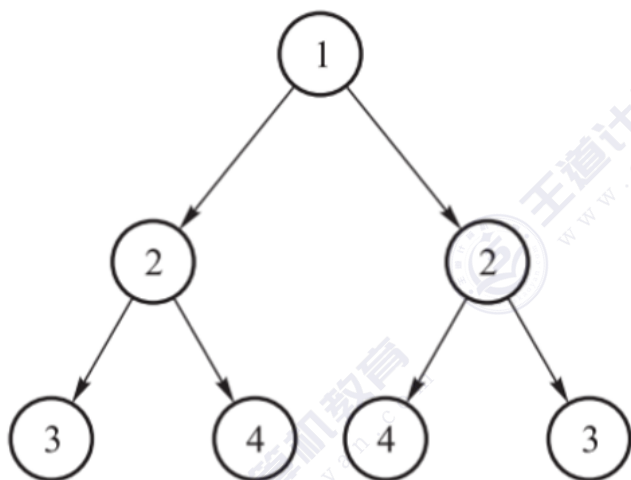
【习题 4-4】 设树T采用孩子兄弟链表表示（数据域、孩子域、兄弟域），设计算法计算树 T 的度。（1）写出树结点的数据结构定义。（2）给出算法的基本设计思想。（3）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释（4）说明所设计算法的时间复杂度。

【习题 4-5】 设二叉树以二叉链表形式存放，根结点的深度为 1，一棵二叉树的繁茂程度定义为各层结点数与该层深度的乘积之和。试设计算法，求二叉树的繁茂程度。数据类型定义如下：

```
typedef struct node{
    int data;
    struct node *lchild, *rchild;
}BTNode;
```

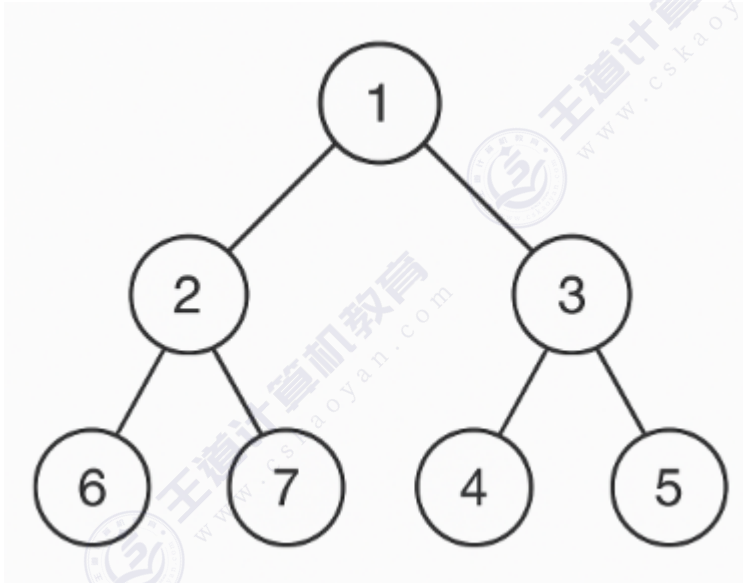
(1) 给出算法的基本设计思想。(2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。(3) 说明所设计算法的时间复杂度和空间复杂度。

【习题 4-6】 给定一棵二叉树，设计算法检查它是否是镜像对称的，如果是镜像对称则返回 1，否则返回 0。例如下图所示就是镜像对称的。



(1) 给出算法的基本设计思想。(2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。(3) 说明所设计算法的时间复杂度。

【习题 4-7】（13 分）二叉树中两个结点 p 和 q 的最低公共祖先是二叉树中离根结点尽可能远且同时是 p 和 q 祖先的结点，结点的祖先可以是自己。给定一棵非空二叉树 T，采用二叉链表存储。例如，对于下图所示的非空二叉树 T，结点 3 和结点 4 的最低公共祖先是结点 3，结点 2 和结点 4 的最低公共祖先是结点 1。



二叉树结点定义如下：

```
typedef struct node
{
    int data;
    struct node *left, *right;
}BTree;
```

root 为指向 T 的根结点的指针，请设计算法求出 p 和 q 的最低公共祖先的地址，要求： 1) 给出算法的基本设计思想。 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

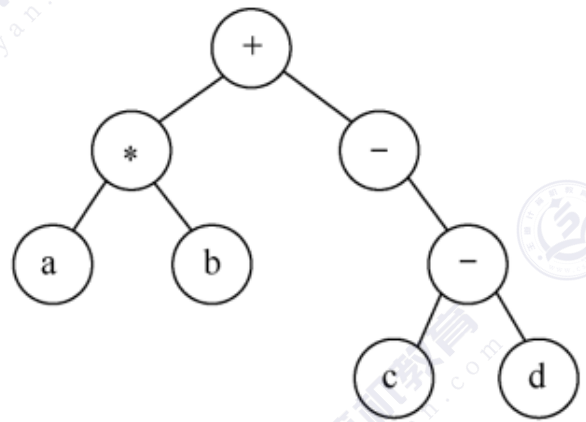
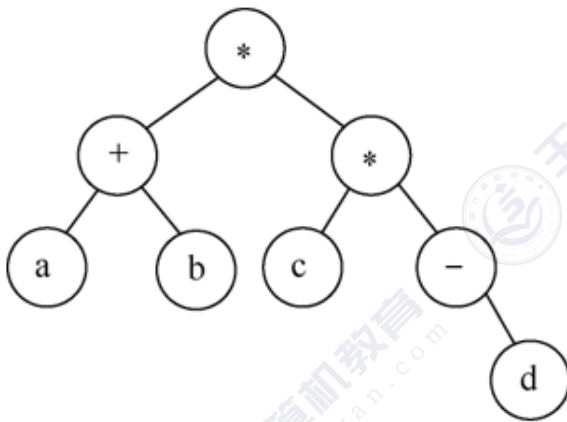
4.6.2 真题

【2014 年 41 题】（13分）二叉树的带权路径长度（WPL）是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T，采用二叉链表存储，结点结构如下：

left	weight	right
------	--------	-------

其中叶结点的 weight 域保存该结点的非负权值。设 root 为指向 T 的根结点的指针，请设计求 T 的 WPL 的算法，要求：（1）给出算法的基本设计思想。（2）使用 C 或 C++ 语言，给出二叉树结点的数据类型定义。（3）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

【2017 年 41 题】（15分）请设计一个算法，将给定的表达式树（二叉树）转换为等价的中缀表达式（通过括号反映操作符的计算次序）并输出。例如，当下列两棵表达式树作为算法的输入时，输出的等价中缀表达式分别为 $(a+b) \times (c \times (-d))$ 和 $(a \times b) + (-(c-d))$ 。



二叉树结点定义如下：

```
typedef struct node{
    char data[10]; // 存储操作数或操作符
    struct node *left, *right;
}BTree;
```

要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

【2022 年 41 题】（13分）已知非空二叉树 T 的结点值均为正整数，采用顺序存储方式保存，数据结构定义如下：

```
typedef struct {           // MAX_SIZE为已定义常量
    int SqBiTNode[MAX_SIZE]; // 保存二叉树结点值的数组
    int ElemNum;             // 实际占用的数组元素个数
} SqBiTree;
```

T中不存在的结点在数组 SqBiTNode 中用 -1 表示。例如，对于下图所示的两棵非空二叉树 T1 和 T2， T1 的存储结果如下：

T1.SqBiTNode	40	25	60	-1	30	-1	80	-1	-1	27	
--------------	----	----	----	----	----	----	----	----	----	----	--

T1.ElemNum = 10

T2 的存储结果如下：

T2.SqBiTNode	40	50	60	-1	30	-1	-1	-1	-1	-1	35
--------------	----	----	----	----	----	----	----	----	----	----	----

T2.ElemNum = 11

请设计一个尽可能高效的算法，判定一棵采用这种方式存储的二叉树是否为二叉搜索树，若是，则返回 true，否则，返回 false。要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

【2026 年 41 题】回忆版 （13分）假定二叉搜索树使用二叉链表存储，存储结构如下：

```
typedef struct BSTNode{
    int data;
    struct BSTNode *left,*right;
} BSTNode;
typedef BSTNode BTNode;
```

给了一个数k，求二叉树节点值和k的差的绝对值的最小值。设计尽可能高效的算法searchX (BTtree root,int k) 输出最小值并输出所有最小值节点的关键字值。（1）给出算法的基本设计思想。（2）根据设计思想，采用 C/C++ 描述算法，关键之处给出注释。

4.7 章节答案

4.7.1 习题

习题 4-1

有 n 个结点的完全二叉树的权值存放在一维数组 A[n] 中（从 A[0] 开始存放），试设计一个算法，将该完全二叉树用二叉链表表示，算法返回指向根结点的指针。二叉树结点数据类型定义如下：

```
typedef struct node{
    int data;
    struct node *lchild, *rchild;
}BTNode;
```

（1）给出算法的基本设计思想。（2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。（3）说明所设计算法的时间复杂度。

【解析】对这棵树采用先中后序皆可，下面采用先序遍历，注意是结点从 A[0] 还是 A[1] 开始存放。【答案】（1）对完全二叉树从根结点开始先序遍历构建树，下标为i的结点左子树下标为 $i \times 2 + 1$ ，他的右子树下标为 $i \times 2 + 2$ ，当访问的结点下标 i 超过 n-1 时说明该结点为空。

（2）代码如下：

```
BTNode* PreOrder(int A[], int i, int n) // i是当前访问下标
{
    if (i >= n || i < 0)
        return NULL; // 该结点不存在
    BTNode* p = (BTNode*)malloc(sizeof(BTNode));
    p->data = A[i]; // 访问当前结点visit(p)
    p->lchild = PreOrder(A, i * 2 + 1, n);
    p->rchild = PreOrder(A, i * 2 + 2, n);
}
```

```
    return p;
}
```

(3) 时间复杂度: $O(n)$

习题 4-2

设计一个算法, 将一棵以二叉链表存储的二叉树T按顺序方式存储到一维数组 B[] 中(根结点下标为 1, 结点的权值 ≥ 0 , 数组值等于 -1 表示该结点不存在)。二叉树结点的数据类型定义如下:

```
typedef struct node{
    int data;
    struct node *lchild, *rchild;
}BTNode;
```

(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(3) 说明所设计算法的时间复杂度。

【解析】本题先、中、后序皆可, 下面采用后序遍历。【答案】(1) 初始时对数组全部元素赋值为 -1, 对二叉树进行后序遍历, 根结点下标为 1, i 号结点左孩子下标为 $2 \times i$, 右孩子下标为 $2 \times i + 1$, 将对应结点的权值存入数组中。(2) 代码如下:

```
void PostOrder(BTNode* p, int B[], int i) // i是当前访问下标
{
    if (p==NULL)
    {
        B[i]=-1;
        return; // 该结点不存在
    }
    PostOrder(p->lchild, B, i*2);
    PostOrder(p->rchild, B, i*2+1);
    B[i]=p->data; // 访问当前结点visit(p)
    return;
}
void ans(BTNode* T, int B[])
{
    for (int i; i<maxn; i++) // maxn是数组的最大长度
        PostOrder(T, B, i);
}
```

(3) 时间复杂度: $O(n)$

请大家思考, 如果根结点下标为 0 应该怎么写?

这种情况下, 根结点下标为 0, i 号结点左孩子下标为 $2 \times i + 1$, 右孩子下标为 $2 \times i + 2$

* 习题 4-3

已知由一棵二叉链表表示的二叉树，设计算法，输出二叉树的高度（只有根结点的二叉树高度为 1）。二叉树结点数据类型定义如下：

```
typedef struct node{
    int data;
    struct node *lchild, *rchild;
}BTNode;
```

(1) 给出算法的基本设计思想，关键之处给出注释。(2) 根据设计思想，采用 C 或 C++ 语言描述算法。(3) 说明所设计算法的时间复杂度。

【答案】(1) 对二叉树进行先序/中序/后序遍历，遍历时设置变量 depth 保存当前子树的高度 = max(左子树高度, 右子树高度)+1，最后输出整棵树的高度即可。(2) 代码如下：

```
int PostOrder(BTNode* p)    // 后序遍历
{
    if (p==NULL)
        return 0;           // 该结点不存在
    int L=PostOrder(p->lchild);
    int R=PostOrder(p->rchild);
    int depth=max(L, R)+1;   // 访问当前结点visit(p)
    return depth;
}
```

(3) 时间复杂度： $O(n)$

习题 4-4

设树 T 采用孩子兄弟链表表示（数据域、孩子域、兄弟域），设计算法计算树 T 的度。

(1) 写出树结点的数据结构定义。(2) 给出算法的基本设计思想。(3) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释(4) 说明所设计算法的时间复杂度。

【答案】(1) 定义如下：

```
typedef struct node{
    int data;
    struct node *firstchild, *nextbro;
}TNode;
```

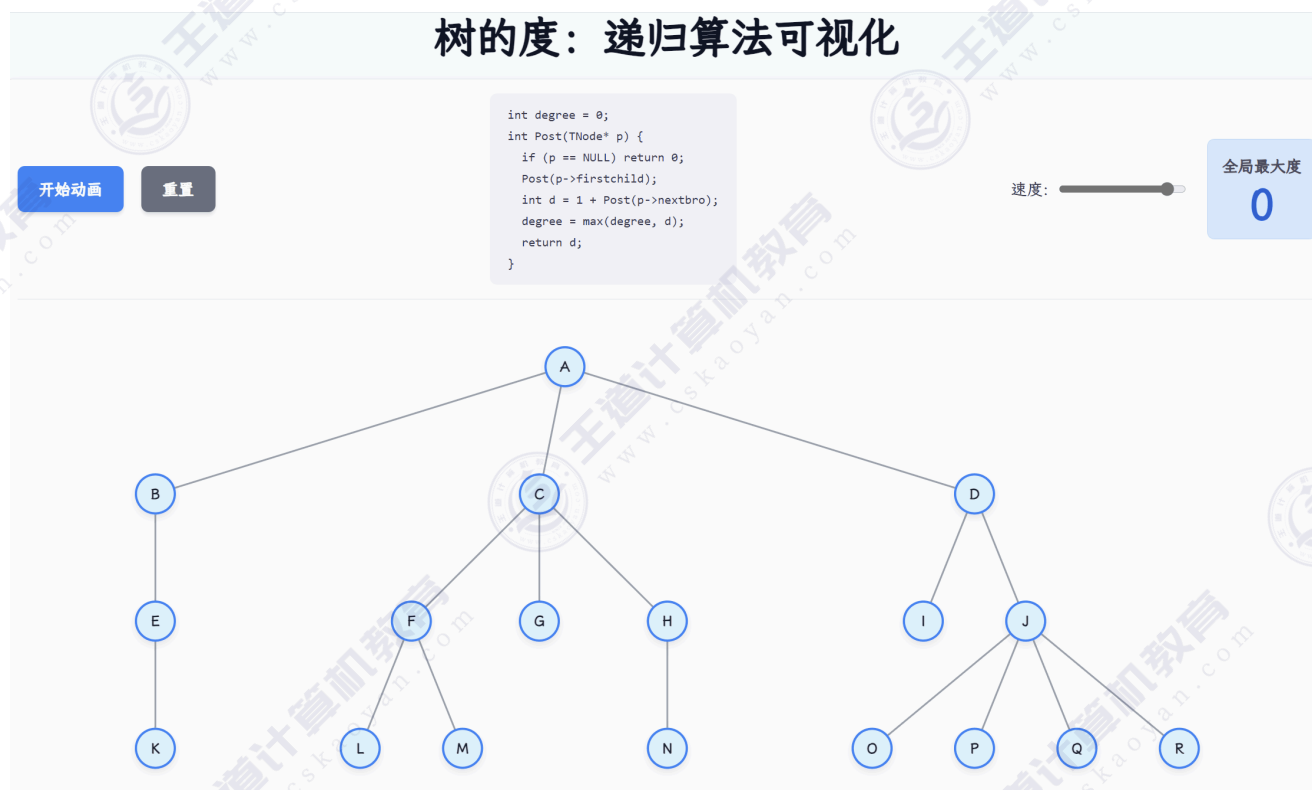
(2) 核心思想：一个父结点的度，等于其第一个孩子结点的兄弟链表的长度。递归函数返回以 p 为起点的兄弟链长度，并沿途用该长度更新全局最大度。(3) 代码如下：

```
int degree = 0; // 全局变量，最终输出 degree

int Post(TNode *p)
{
    // 后根序遍历
    if (p == NULL) return 0; // 空结点
    Post(p->firstchild); // 递归处理孩子
    int d = 1 + Post(p->nextbro); // d = 以 p 为起点的兄弟链长度
    degree = max(degree, d); // 更新全局最大度
    return d;
}
```

以根有 3 个孩子 A、B、C 为例：处理 A 时 $d = 1 + \text{Post}(B) = 1 + 2 = 3$ ，degree 更新为 3（正确）。根自身的 $d = 1$ （没有兄弟），但不影响 degree。(4) 时间复杂度： $O(n)$

树的度：递归算法可视化



习题 4-5

设二叉树以二叉链表形式存放，根结点的深度为 1，一棵二叉树的繁茂程度定义为各层结点数与该层深度的乘积之和。试设计算法，求二叉树的繁茂程度。数据类型定义如下：

```
typedef struct node{
    int data;
    struct node *lchild, *rchild;
}BTNode;
```

(1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(3) 说明所设计算法的时间复杂度和空间复杂度。

【答案1】(1) 按照二叉树的层次遍历 BFS, front 保存队头元素后一个位置下标, rear 保存下一个入队元素下标, last 保存上一层的最后一个元素下标, 当本层访问完的时候, 本层元素个数为 rear-last, 依次算出每一层的繁茂程度, 并求出最大值。(2) 代码如下:

```
int ans(BTNode *T)
{
    BTNode* q[MAX_SIZE];
    int front=0, rear=0, depth=1;
    int last=0; // 本层的最后一个结点下标
    if (T!=NULL) q[rear++]=T;
    BTNode *p; // 当前访问的结点p
    int sum=1*1; // 第一层的繁茂程度
    while(front<rear)
    {
        p=q[front++];
        if (p->lchild!=NULL)
            q[rear++]=p->lchild;
        if (p->rchild!=NULL)
            q[rear++]=p->rchild;
        if (front>last) // 该层已访问完
        {
            depth++;
            printf("%d %d %d\n", front, rear, depth);
            sum+=(rear-front)*depth;
            last=rear-1;
        }
    }
    return sum;
}
```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(n)$

【答案2】后序遍历, 传递深度

(1) 繁茂程度 = 每个结点的深度之和。后序遍历时, 每访问一个结点, 将其深度 depth 累加到返回值中。递归函数返回以 p 为根的子树中所有结点的深度之和。(2) 代码如下:

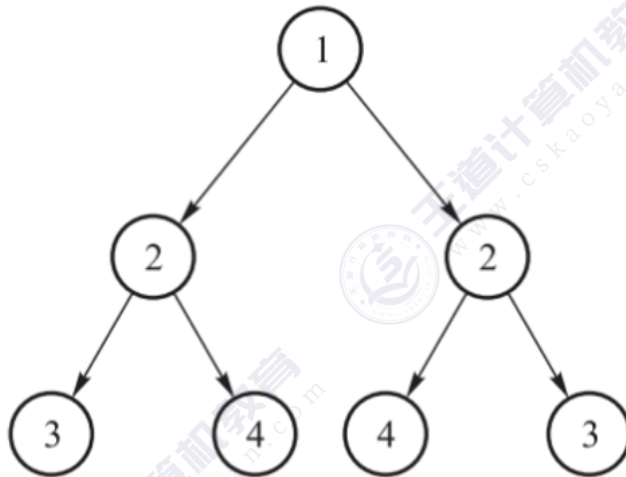
```
int PostOrder(BTNode *p, int depth)
{
    // 后序遍历, depth 为当前层数
    if (p == NULL) return 0;
    int L = PostOrder(p->lchild, depth + 1); // 遍历左子树
    int R = PostOrder(p->rchild, depth + 1); // 遍历右子树
    return L + R + depth; // 访问: 累加当前结点的深度
}
```

```
int ans(BTNode *T)
{
    return PostOrder(T, 1); // 从根开始，深度为 1
}
```

(3) 时间复杂度： $O(n)$ ，空间复杂度： $O(h)$ (递归栈)

习题 4-6

给定一棵二叉树，设计算法检查它是否是镜像对称的，如果是镜像对称则返回 1，否则返回



0。例如下图所示就是镜像对称的。

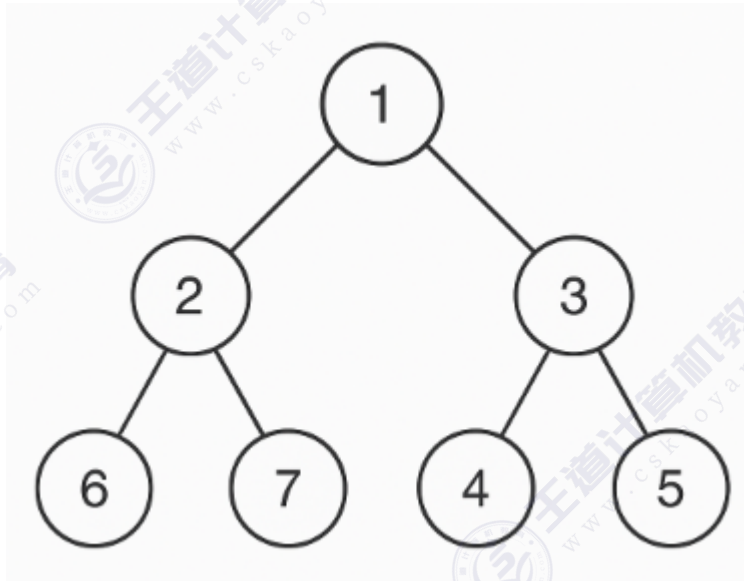
(1) 给出算法的基本设计思想。(2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。(3) 说明所设计算法的时间复杂度。【解析】本题采用先序/中序/后序遍历都可以，设置两个指针 p、q 访问即可。【答案】(1) 按照先序遍历，在遍历的时候设置两个指针 p、q，分别指向当前访问结点和它对称的位置，每次比较 p 和 q 的值是否相同。当 p 递归左（右）孩子的时候，q 递归右（左）孩子，这样可以保证 p 和 q 一直是对称位置，如果发现 p 和 q 的值不一样则返回 0。(2) 代码如下：

```
int PreOrder(BTNode *p, BTNode *q)
{
    if (p==NULL && q==NULL)
        return 1;
    if (p==NULL || q==NULL)
        return 0;
    if (p->data!=q->data)
        return 0;
    if (PreOrder(p->lchild, q->rchild)==0)
        return 0;
    if (PreOrder(p->rchild, q->lchild)==0)
        return 0;
    return 1;
}
```

(3) 时间复杂度： $O(n)$

习题 4-7

(13 分) 二叉树中两个结点 p 和 q 的最低公共祖先是二叉树中离根结点尽可能远且同时是 p 和 q 祖先的结点，结点的祖先可以是自己。给定一棵非空二叉树 T ，采用二叉链表存储。例如，对于下图所示的非空二叉树 T ，结点 3 和结点 4 的最低公共祖先是结点 3，结点 2 和结点



二叉树结

4 的最低公共祖先是结点 1。
点定义如下：

```
typedef struct node
{
    int data;
    struct node *left, *right;
} BTree;
```

$root$ 为指向 T 的根结点的指针，请设计算法求出 p 和 q 的最低公共祖先的地址，要求：1) 给出算法的基本设计思想。2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

【答案】 (1) **算法设计思想**：使用递归后序遍历二叉树，对于当前访问的结点 $root$ ：

- 若 $root == p$ 或 $root == q$ ，直接返回 $root$ ；
 - 递归分别在左子树、右子树查找 p 和 q ；
 - 若左右子树递归结果均不为 $NULL$ ，说明 p 和 q 分别在左右子树中， $root$ 即为最近公共祖先；
 - 若其中一个子树递归结果为 $NULL$ ，则返回另一侧的结果（说明两结点均在某一子树中）。
- (2) **算法代码**：

```
BTree* PostOrder(BTree* root, BTree* p, BTree* q) {
    if (root == NULL)
        return NULL;
    if (root == p || root == q)
        return root; // 若当前结点等于p或q，直接返回
    BTree* left = PostOrder(root->left, p, q); // 在左子树查找
```

```

BTree* right = PostOrder(root->right, p, q);    // 在右子树查找
if (left != NULL && right != NULL)
    return root;                                // 左右子树都找到，当前root即为最近公共祖先
// 如果只在一个子树找到则返回该子树
if (left == NULL) return right;
else return left;
}

```

(3) 时间复杂度: $O(n)$ 空间复杂度: $O(n)$

4.7.2 真题

2014 年 41 题

(13分) 二叉树的带权路径长度 (WPL) 是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T, 采用二叉链表存储, 结点结构如下:

left	weight	right
------	--------	-------

其中叶结点的weight域保存该结点的非负权值。设root为指向T的根结点的指针, 请设计求T的WPL的算法, 要求: (1) 给出算法的基本设计思想。(2) 使用 C 或 C++ 语言, 给出二叉树结点的数据类型定义。(3) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。带权路径长度 **WPL** 有两种计算方法: ① WPL的定义: **WPL** = 所有 (叶结点的权值 W_i × 该结点深度 D_i) 求和。

$$WPL = \sum_{i \text{ 对应叶结点}} D_i * W_i$$

② 推导。如果该树的所有非叶结点满足以下条件: 它的权值等于它的孩子结点权值之和。则 **WPL** = 所有非叶结点的权值和。

$$WPL = \sum_{j \text{ 对应非叶结点}} W_j$$

因此本题有两种解法, 都可以拿满分。

【解法1】前序/中序/后序遍历, 下面给出的是先序遍历

(1) 先序遍历二叉树, 设置全局变量 WPL, 每次 p 访问到叶子结点时 p 的权值与深度相乘 (根结点深度 depth=0), 再累加到 WPL 中。(2) 类型定义:

```
typedef struct BTreeNode{
    int weight;
    struct BTreeNode *left, *right;
}BTreeNode;
```

(3) 算法如下:

```
int WPL=0; // WPL是全局变量
void PreOrder(BTreeNode *p, int depth) // 当前结点p, 深度depth
{
    if (p==NULL) // p是空结点
        return;
    if (p->left==NULL && p->right==NULL) // p是叶结点
```

```

WPL+=depth*p->weight;
PreOrder(p->left, depth+1);    // 递归调用左孩子
PreOrder(p->right, depth+1);   // 递归调用右孩子
}

```

【解法2】后序遍历

(1) 后序遍历二叉树，设置全局变量 WPL，如果访问到叶结点则把权值返回给父亲，如果是非叶结点则该结点权值等于左、右孩子的权值和，累加到 WPL 中，并将该权值返回给父亲。

(2) 类型定义：

```

typedef struct BTreeNode{
    int weight;
    struct BTreeNode *left, *right;
}BTreeNode;

```

(3) 算法如下：

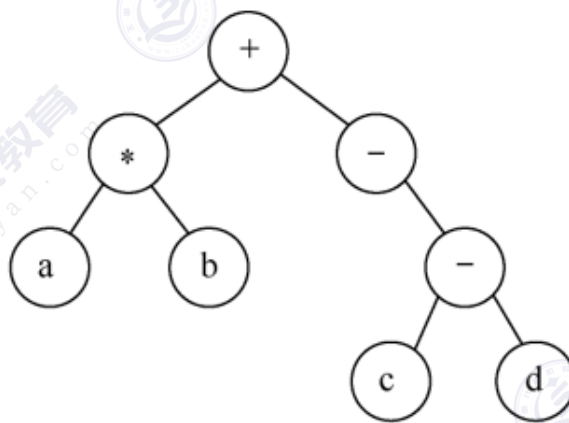
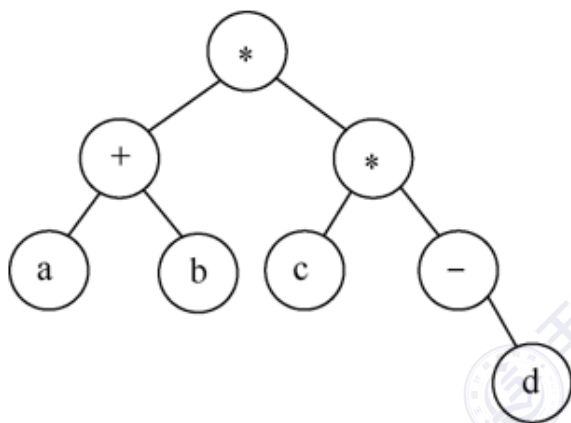
```

int WPL=0;    // WPL是全局变量
int PostOrder(BTreeNode *p)
{
    if (p==NULL)    // p是空结点
        return 0;
    int L=PostOrder(p->left);    // 递归调用左孩子
    int R=PostOrder(p->right);    // 递归调用右孩子
    if (p->left==NULL && p->right==NULL)
        return p->weight;    // p是叶结点
    WPL=WPL+L+R;    // p是非叶结点
    return L+R;
}

```

2017 年 41 题

(15分) 请设计一个算法，将给定的表达式树（二叉树）转换为等价的中缀表达式（通过括号反映操作符的计算次序）并输出。例如，当下列两棵表达式树作为算法的输入时，输出的等价中缀表达式分别为 $(a+b) \times (c \times (-d))$ 和 $(a \times b) + (-(c-d))$ 。



二叉树结点定义如下：

```
typedef struct node{
    char data[10]; // 存储操作数或操作符
    struct node *left, *right;
}BTree;
```

要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

【解法】中序遍历，递归（1）中序遍历二叉树，对于当前访问的非空结点 p ，则先输出“（”，然后递归调用左子树，输出 p 的权值，递归调用右子树，输出“）”，如果 p 是根或者叶结点则不需要输出“（”或“）”。（2）算法如下：

```
void InOrder(BTree* p, BTree* root)
{
    if (p==NULL) return; // p是空结点
    if ((p->left!=NULL || p->right!=NULL) && (p!=root))
        printf("(");
    InOrder(p->left, root); // 递归调用左子树
    printf("%s", p->data); // 输出操作数
    InOrder(p->right, root); // 递归调用右子树
    if ((p->left!=NULL || p->right!=NULL) && (p!=root))
        printf(")");
}
```

2022 年 41 题

（13分）已知非空二叉树 T 的结点值均为正整数，采用顺序存储方式保存，数据结构定义如下：

```
typedef struct {
    // MAX_SIZE为已定义常量
    int SqBiTNode[MAX_SIZE]; // 保存二叉树结点值的数组
```

```
int ElemNum;           // 实际占用的数组元素个数
} SqBiTree;
```

T 中不存在的结点在数组 SqBiTNode 中用 -1 表示。例如，对于下图所示的两棵非空二叉树 T1 和 T2，T1 的存储结果如下：

T1.SqBiTNode	40	25	60	-1	30	-1	80	-1	-1	27		
--------------	----	----	----	----	----	----	----	----	----	----	--	--

T1.ElemNum = 10

T2 的存储结果如下：

T2.SqBiTNode	40	50	60	-1	30	-1	-1	-1	-1	-1	35	
--------------	----	----	----	----	----	----	----	----	----	----	----	--

T2.ElemNum = 11

请设计一个尽可能高效的算法，判定一棵采用这种方式存储的二叉树是否为二叉搜索树，若是，则返回 true，否则，返回 false。要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

【解法】中序遍历（1）如果当前结点的数组下标为 i 则左孩子下标为 $2i+1$ ，右孩子下标为 $2i+2$ ，对该树进行递归的中序遍历，pre 保存当前访问结点的前驱结点的权值，如果 SqBiTree[i] < pre 则说明中序遍历得到的不是升序，即当前子树返回 false，如果 SqBiTree[i] >= pre 且左子树和右子树都确定是二叉排序树则返回 true。（2）算法如下：

```
int pre=0;           // 全局变量pre保存中序前驱的权值，初值0
bool InOrder(SqBiTree *T, int i)
{
    /*指针bt指向二叉树的结构体，i是当前访问的下标*/
    if (i>=T->ElemNum || T->SqBiTNode[i]==-1) // 当前结点为空
        return true;
    if (InOrder(T, 2*i+1) != true) // 递归调用左子树
        return false; // 左子树不符合定义
    if (pre>T->SqBiTNode[i]) // 访问当前结点visit(p)
        return false; // 前驱大于当前结点，不符合定义
    pre=T->SqBiTNode[i]; // 更新pre
    if (InOrder(T, 2*i+2) != true) // 递归调用右子树
        return false; // 右子树不符合定义
    return true; // 根下标为i的子树符合定义
}
```

2026 年 41 题（回忆版）

（13分）假定二叉搜索树使用二叉链表存储，存储结构如下：

```
typedef struct BSTNode{
    int data;
    struct BSTNode *left,*right;
```

```

} BSTNode;
typedef BSTNode BTNode;

```

给了一个数k，求二叉树节点值和k的差的绝对值的最小值。设计尽可能高效的算法searchX (BTtree root,int k) 输出最小值并输出所有最小值节点的关键字值。（1）给出算法的基本设计思想。（2）根据设计思想，采用 C/C++ 描述算法，关键之处给出注释。

做题思路	时间复杂度	空间复杂度	预计得分
遍历两次——中序遍历	$O(n)$	$O(n)$	10 ~ 11
遍历一次——中序遍历同时找 small 和 big	$O(n)$	$O(n)$	10 ~ 12
查找两次——分别找 small 和 big	$O(n)$	$O(1)$	12
查找一次——同时维护 small 和 big	$O(n)$	$O(1)$	13

【解法1】遍历两次——中序遍历

（1）对 BST 进行中序遍历得到升序序列。第一遍遍历计算每个结点与 k 的差值绝对值，记录全局最小差值 min_diff。第二遍遍历输出 min_diff 以及所有 $|data - k| == min_diff$ 的结点关键字。（2）算法如下：

```

int min_diff = INT_MAX; // 全局最小差值

void InOrder1(BSTNode *t, int k)
{
    if (t == NULL) return;
    InOrder1(t->left, k); // 遍历左子树
    int diff = abs(t->data - k); // 当前结点与 k 的差值
    if (diff < min_diff) min_diff = diff; // 更新最小差值
    InOrder1(t->right, k); // 遍历右子树
}

void InOrder2(BSTNode *t, int k)
{
    if (t == NULL) return;
    InOrder2(t->left, k);
    if (abs(t->data - k) == min_diff) // 差值等于最小差值的结点
        printf("%d ", t->data);
    InOrder2(t->right, k);
}

void searchX(BSTNode *root, int k)
{

```

```

min_diff = INT_MAX;
InOrder1(root, k); // 第一遍：求最小差值
printf("最小差值为 %d, 结点为: ", min_diff);
InOrder2(root, k); // 第二遍：输出匹配结点
}

```

或用一个 InOrder 函数配合全局变量区分遍历次数：

```

int min_diff = INT_MAX; // 全局最小差值
int round = 0; // 第几次遍历

void InOrder(BSTNode *t, int k)
{
    if (t == NULL) return;
    InOrder(t->left, k);
    if (round == 1) // 第一遍：求最小差值
    {
        int diff = abs(t->data - k);
        if (diff < min_diff) min_diff = diff;
    }
    else // 第二遍：输出匹配结点
    {
        if (abs(t->data - k) == min_diff)
            printf("%d ", t->data);
    }
    InOrder(t->right, k);
}

void searchX(BSTNode *root, int k)
{
    min_diff = INT_MAX;
    round = 1; InOrder(root, k); // 第一遍：求最小差值
    printf("%d: ", min_diff); // 输出最小差值
    round = 2; InOrder(root, k); // 第二遍：输出匹配结点
}

```

(3) 时间复杂度： $O(n)$ ，空间复杂度： $O(n)$

【解法2】遍历一次——中序同时找 small 和 big

(1) 中序遍历 BST 得到升序序列。一次遍历中，每当遇到值 $\leq k$ 的结点就更新 small（由于升序，后遇到的 $< k$ 的值更接近 k ）；每当遇到值 $> k$ 的结点，第一个就是 big（升序下第一个 $> k$ 的就是 $\geq k$ 的最小值）。遍历结束比较两者。若命中精确匹配则直接返回。（2）算法如下：

```

BSTNode *small = NULL, *big = NULL; // 全局变量

void InOrder(BSTNode *t, int k)
{
    if (t == NULL) return;
    InOrder(t->left, k); // 遍历左子树

```

```

    if (t->data <= k)                // 值 <= k, 更新 small
        small = t;                  // 升序下后遇到的一定更接近 k
    if (t->data > k && big == NULL)    // 第一个 > k 的就是 big
        big = t;
    InOrder(t->right, k);             // 遍历右子树
}

void searchX(BSTNode *root, int k)
{
    small = NULL; big = NULL;
    InOrder(root, k);                // 一次中序遍历
    int ans = INT_MAX;
    if (small!=NULL) ans = k - small->data;
    if (big!=NULL && big->data-k<ans) ans = big->data-k;
    printf("%d:", ans);
    if (small!=NULL && k-small->data==ans)
        printf("%d", small->data);
    if (big!=NULL && big->data-k==ans)
        printf("%d", big->data);
}

```

(3) 时间复杂度: $O(n)$, 空间复杂度: $O(n)$ (递归栈)

【解法3】查找两次——分别找 small 和 big

(1) BST 中离 k 最近的结点必为比 k 小的最大值 (small) 或比 k 大的最小值 (big)。分别沿搜索路径查找 small 和 big, 各需一次查找。比较 $|small->data - k|$ 和 $|big->data - k|$, 输出更小者及对应的关键字。(2) 算法如下:

```

void searchX(BSTNode *root, int k)
{
    BSTNode *small = NULL, *big = NULL;

    // — 查找 small: 比 k 小的最大值 —
    BSTNode *p = root;
    while (p != NULL)
    {
        if (p->data == k)                // 命中
        {
            printf("0: %d", p->data);
            return;
        }
        if (p->data < k)                  // 当前值更小, 向右走
        {
            small = p;                  // 记录 small 候选
            p = p->right;
        }
        if (p->data > k)                  // 当前值更大, 向左走
            p = p->left;
    }

    // — 查找 big: 比 k 大的最小值 —
    p = root;

```

```

while (p != NULL)
{
    if (p->data == k)                // 命中
    {
        printf("0: %d", p->data);
        return;
    }
    if (p->data > k)                  // 当前值更大，向左走
    {
        big = p;                    // 记录 big 候选
        p = p->left;
    }
    if (p->data < k)                  // 当前值更小，向右走
        p = p->right;
}

// — 比较 small 和 big, 输出结果 —
int d_small, d_big;
if (small != NULL)
    d_small = k - small->data;
else
    d_small = INT_MAX;
if (big != NULL)
    d_big = big->data - k;
else
    d_big = INT_MAX;

if (d_small < d_big)                // 左边更接近
    printf("%d: %d", d_small, small->data);
if (d_big < d_small)                // 右边更接近
    printf("%d: %d", d_big, big->data);
if (d_small == d_big)               // 两边一样近
    printf("%d: %d %d", d_small, small->data, big->data);
}

```

(3) 时间复杂度: $O(h)$ 或 $O(n)$, 空间复杂度: $O(1)$

【解法4】查找一次——同时维护 small 和 big

(1) 将两次查找合并为一次: 从根向叶查找 k , 沿途向右走时将当前结点记为 small, 向左走时记为 big。走到底后比较两者与 k 的差值即可。只需一次遍历路径。(2) 算法如下:

```

void searchX(BSTNode *root, int k)
{
    BSTNode *small = NULL, *big = NULL, *p = root;

    // — 一次查找, 同时维护 small 和 big —
    while (p != NULL)
    {
        if (p->data == k)            // 精确命中
        {
            printf("0: %d", k);
            return;
        }

```

```

    }
    if (p->data < k)                // 当前值 < k, 向右
    {
        small = p;                // 更新 small
        p = p->right;
    }
    if (p->data > k)                // 当前值 > k, 向左
    {
        big = p;                  // 更新 big
        p = p->left;
    }
}

// — 比较 small 和 big, 输出结果 —
int d_small, d_big;
if (small != NULL)
    d_small = k - small->data;
else
    d_small = INT_MAX;
if (big != NULL)
    d_big = big->data - k;
else
    d_big = INT_MAX;

if (d_small < d_big)                // 左边更接近
    printf("%d, %d", d_small, small->data);
if (d_big < d_small)                // 右边更接近
    printf("%d, %d", d_big, big->data);
if (d_small == d_big)               // 两边一样近
    printf("%d, %d %d", d_small, small->data, big->data);
}

```

(3) 时间复杂度: $O(h)$ 或 $O(n)$, 空间复杂度: $O(1)$

5 图

5.1 图概述

5.1.1 图的基本概念

想象一下，我们把图看作一张**城市地图**。地图上的每个**城市**就是一个**顶点 (Vertex)**，连接城市之间的**道路**就是一条**边 (Edge)**。

- **路径**：是从一个顶点到另一个顶点所经过的一系列边的集合。你从家里（起点）出发，经过几条街道（边），最终到达学校（终点）。这一整段路程就是一条路径。
- **环**：也叫回路，是一条特殊的路径，它的起点和终点是同一个顶点。
- **自环**：是一条从某个顶点出发，又立刻回到自身的边。
- **重边**：是指在两个相同的顶点之间，存在多于一条的边。
- **无向图**：图中的边（道路）没有方向，是双向的。
- **有向图**：图中的边（道路）有明确的方向，是单行道。
- **无权图**：所有的边都没有“权重”或“成本”的概念。我们只关心两个顶点之间有没有路，不关心路有多长。在地铁线路图上，我们通常只关心从A站到B站需要经过几站，而不关心每一站之间的具体距离。
- **有权图**：每一条边都被赋予了一个数值，称为权重。连接两个城市的道路不仅存在，还标有长度（公里），这就是边的权重。
- **连通图**：在一个无向图中，任意两个顶点之间都存在路径。
- **强连通图**：在一个有向图中，任意两个顶点之间都存在路径。
- **完全图**：在一个无向图中，任意两个顶点之间都存在边。
- **有向完全图**：在一个有向图中，任意两个顶点之间都存在边。
- **子图**：从原图中选取一部分顶点和一部分边组成的图。好比从一张完整的全国地图中，单独把“长三角地区”的地图拎出来。
- **极大连通子图**：指在保持“连通”这个性质的前提下，规模最大的子图。你不能再从原图中添加任何顶点或边到这个子图里，否则它就不再是连通的了。就是一个连通分量。
- **连通分量**：对于一个非连通的无向图，图中“抱团”的、内部任意互通的极大连通子图。
- **生成树**：在一个连通图中，选取一部分边，刚好能连接起所有的顶点，并且不形成任何环。这个由“最少的边”构成的骨架，就是一棵生成树。

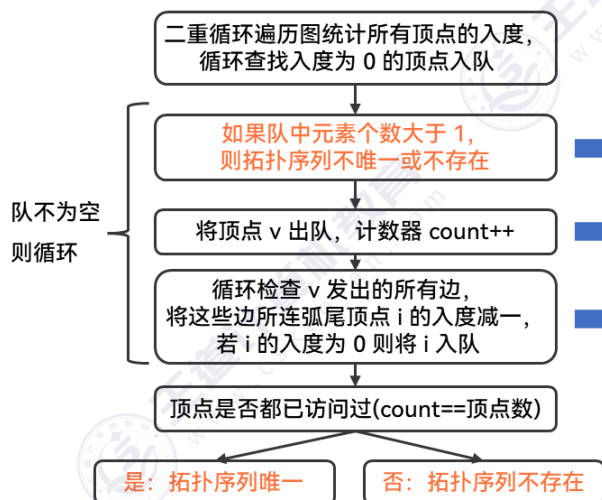
若一个图中没有自环和重边，它被称为简单图，考试中的图默认简单图。

本章默认：图的顶点集合为 V ，顶点数 $|V|$ 为 n ，边集合为 E ，边数 $|E|$ 为 e 。

5.1.2 图的做题方法

图的题是所有题目里面最简单的，重点复习遍历，其次可以复习拓扑排序，其他的图算法考察概率比较小。对于这些算法，建议以理解实现思路为主，知道对应算法由几步构成，每一步实现什么功能。在题目中如果遇到，可以先在草稿纸上列出流程，理清思路后再直接在试卷上分块作答：

草稿纸上列出流程



根据流程在试卷上分块还原代码

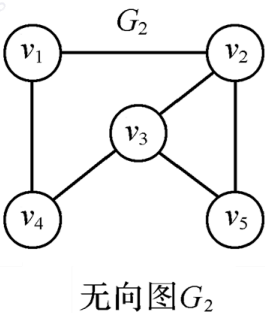
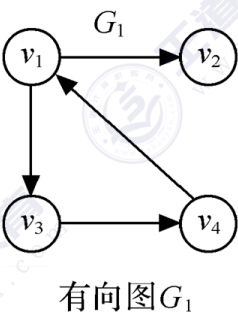
```
1 while (front < rear)
2 {
3     // 如果队列中有多个顶点，说明图的拓扑序列不唯一
4     if (rear - front > 1)
5         return 0;
6     int v = queue[front++]; // 取出队首顶点
7     count++;
8     // 更新该顶点的所有邻接顶点的入度
9     for (int i = 0; i < G.numVertices; i++)
10         if (G.Edge[v][i] != 0)
11         {
12             inDegree[i]--; // 如果入度变为0，则加入队列
13             if (inDegree[i] == 0)
14                 queue[rear++] = i;
15         }
16 }
```

5.2 图的结构和遍历

在处理图的问题时，首先要确定其存储结构。考试中主要考察两种：**邻接矩阵**和**邻接表**。邻接表可以进一步演化为适用于无向图的**邻接多重表**和适用于有向图的**十字链表**。

	邻接矩阵	邻接表	邻接多重表	十字链表
有向图	√	√	×	√
无向图	√	√	√	×

5.2.1 邻接矩阵



$$G_1 = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

$$G_2 = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

邻接矩阵结构：

```
#define MAXV 100 // 最大顶点数
typedef struct // 图的定义
{
    int numVertices, numEdges; // 图中实际的顶点数和边数
    char VerticesList[MAXV]; // 可选：每个顶点名称
    int Edge[MAXV][MAXV]; // 邻接矩阵
}MGraph;
```

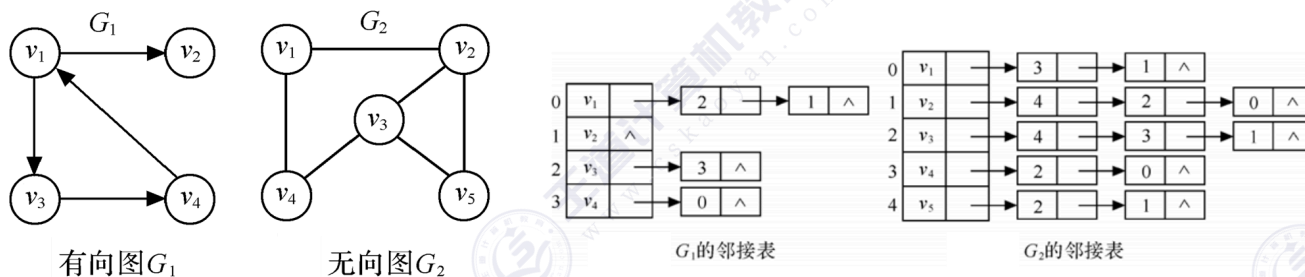
邻接矩阵通常使用 0 表示不存在边，`Edge[i][j] = 0` 说明 i 到 j 之间不存在边。

无权图使用 1 表示存在边，有权图使用非 0 的数表示存在边。

邻接矩阵的遍历：

```
void TraverseEdges_MGraph(MGraph G)
{
    for (int i = 0; i < G.numVertices; i++)
        for (int j = 0; j < G.numVertices; j++)
            if (G.Edge[i][j] != 0)
                visit(i, j); // 如果i->j存在边则处理
}
```

5.2.2 邻接表



邻接表结构:

```
#define MAXV 100 // 最大顶点数
// 边结点
typedef struct ArcNode
{
    int adjvex; // 邻接点在顶点表中的下标
    struct ArcNode *next; // 指向下一条边
    int weight; // 有权图可选: 边权值
} ArcNode;

// 顶点结点
typedef struct VNode
{
    char data; // 可选: 顶点信息
    ArcNode *first; // 指向第一条邻接边
} VNode;

// 邻接表图定义
typedef struct
{
    VNode VerticesList[MAXV]; // 顶点表
    int numVertices, numEdges; // 顶点数、边数
} ALGraph;
```

图的结构体

请大家思考一个问题: 顶点 VNode 定义中, 如果我们不需要用到顶点信息, 是不是可以省略这部分定义, 从而整个邻接表结构可以写的更简单, 该写法可能不太常用, 但大家如果看到需要认识 (习题中有出现):

```
typedef struct ArcNode
{
    int adjvex; // 该弧所指向的顶点
    struct ArcNode *next; // 指向下一条弧的指针
    int weight; // 有权图可选: 边权值
} ArcNode;

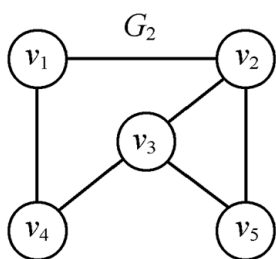
typedef struct
{
    ArcNode *VerticesList[MAXV]; // 顶点数组
```

```
int numVertices, arcnum; // 图的当前顶点数和弧数
}ALGraph;
```

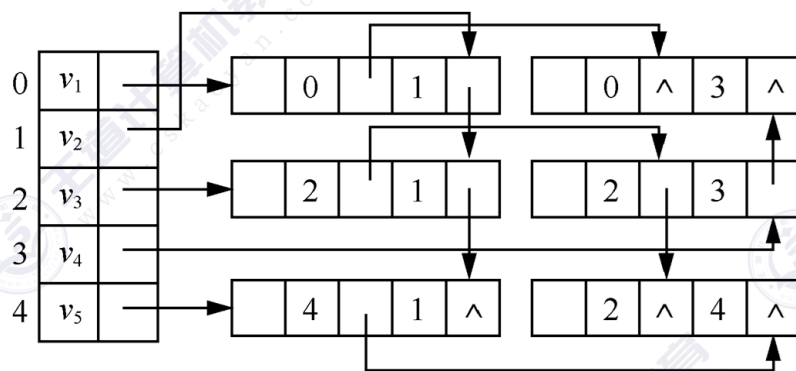
邻接表的遍历:

```
void TraverseEdges_ALGraph(ALGraph G)
{
    // 遍历所有顶点
    for (int i = 0; i < G.numVertices; i++)
    {
        // 遍历当前顶点i发出的所有边
        ArcNode *p = G.VerticesList[i].first;
        while (p != NULL)
        {
            visit(p); // 访问对应边
            p = p->next; // 处理下一条边
        }
    }
}
```

5.2.3 * 邻接多重表



无向图 G_2



邻接多重表结构:

```
#define MAXV 100

// 边结点
typedef struct EBox
{
    bool mark;           // 标记是否访问
    int ivex, jvex;      // 该边依附的两个顶点下标
    struct EBox *ilink, *jlink; // 指向 ivex 和 jvex 的下一条边
    int weight;          // 可选: 边权值
} EBox;

// 顶点结点
typedef struct VexBox
{
    char data;           // 顶点信息
    EBox *first;         // 指向第一条边
} VexBox;

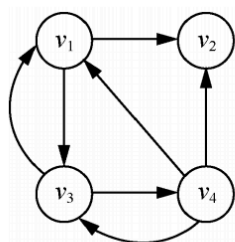
// 邻接多重表图定义
typedef struct
{
    VexBox VerticesList[MAXV]; // 顶点表
    int numVertices, numEdges; // 顶点数、边数
} AMLGraph;
```

邻接多重表的遍历:

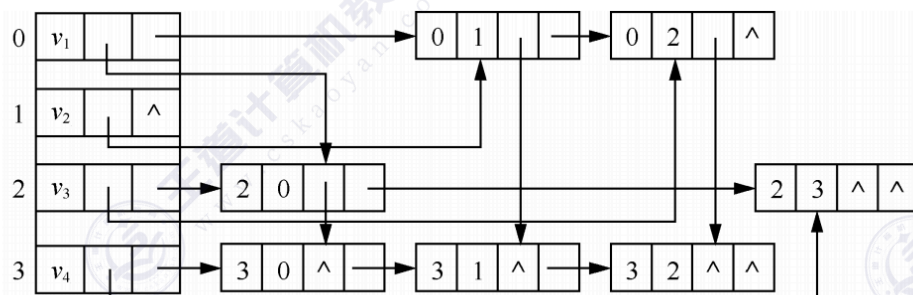
```
void TraverseEdges_AMLGraph(AMLGraph G)
{
    // 遍历所有顶点
    for (int i = 0; i < G.numVertices; i++)
    {
        // 遍历当前顶点i发出的所有边
        EBox *p = G.VerticesList[i].first;
        while (p != NULL)
        {
            if (p->mark != true) // 如果未访问过则访问且标记
            {
                // ... (traversal logic) ...
            }
            p = p->jlink;
        }
    }
}
```

```
        visit(p);
        p->mark = true;
    }
    if (i == p->ivex) p = p->ilink;
    else p = p->jlink;
}
}
```

5.2.4 * 十字链表



(a) 有向图



(b) 十字链表

十字链表结构：

```
#define MAXV 100

// 边结点
typedef struct ArcBox
{
    int tailvex, headvex;           // 弧尾（起点）和弧头（终点）在顶点表中的下标
    struct ArcBox *hlink;          // 指向同终点的下一条边（入边链）
    struct ArcBox *tlink;          // 指向同起点的下一条边（出边链）
    int weight;                     // 可选：边权值
} ArcBox;

// 顶点结点
typedef struct VexNode
{
    char data;                      // 顶点信息
    ArcBox *firstin, *firstout;      // 入边表、出边表指针
} VexNode;

// 十字链表图定义
typedef struct
{
    VexNode VerticesList[MAXV];     // 顶点表
    int numVertices, numEdges;      // 顶点数、边数
} OLGraph;
```

十字链表的遍历：

```
void TraverseEdges_OLGraph(OLGraph G)
{
    // 遍历所有顶点
    for (int i = 0; i < G.numVertices; i++)
    {
        // 遍历当前顶点i发出的所有边
        ArcBox *p = G.VerticesList[i].firstout; // 请注意是出边
        while (p != NULL)
        {
            visit(p);                          // 访问对应边
            p = p->tlink;                       // 起点相同
        }
    }
}
```

}
}



5.3 图的经典算法

图可以有多种定义，因此对于图算法，核心应该是理清：

- 功能是什么？适用于什么题目？
- 步骤有哪些？

5.3.1 深度优先遍历

基本思想：沿着图的某一路径尽可能“深入”，直到不能继续为止，再回溯到上一个结点，尝试探索其他路径。其核心是“**一条路走到黑，走不通再退回换条路**”。这非常类似于树的**先序遍历**，也像在走迷宫时，沿着一条路走到死胡同，然后原路返回到上一个岔路口，选择另一条路继续探索。

实现原理：DFS 的“后进先出”特性天然与**栈**这种数据结构相匹配。假设未访问的结点用 **蓝色**，表示平静；已加入递归栈中的用 **橙黄色**，表示正在处理中；已处理完的用 **绿色**，表示已经探索完毕。

算法步骤 (递归版)

1. 选择一个起始顶点 v ，访问它，并标记为已访问 ($visited[v] = true$)。
2. 查找 v 的所有邻接点 w 。
3. 如果邻接点 w 未被访问，则从 w 开始进行深度优先遍历（递归调用 $DFS(G, w)$ ）。
4. 如果 v 的所有邻接点都已被访问，则函数返回，实现回溯。

邻接表的 DFS 代码如下：

```
bool visited[MAXV] = {false};
void DFS_ALGraph(ALGraph G, int v)
{
    visit(v); // 访问顶点
    visited[v] = true;
    ArcNode *p = G.VerticesList[v].first;
    while (p != NULL)
    {
        if (!visited[p->adjvex])
            DFS_ALGraph(G, p->adjvex);
        p = p->next;
    }
}

void ans(ALGraph G)
{
    // 无向图调用 DFS 次数等于连通分量数
    for (i = 0; i < G.numVertices; i++)
        if (visited[i] != true) DFS_ALGraph(G, i);
}
```

复杂度分析 假设图有 n 个顶点和 e 条边。

- **时间复杂度：**
 - **邻接表：**每个顶点和每条边都只会被访问一次，所以时间复杂度为 $O(n+e)$ 。
 - **邻接矩阵：**需要遍历矩阵来查找每个顶点的邻接点，所以时间复杂度为 $O(n^2)$ 。
- **空间复杂度：**主要取决于递归深度或栈的大小，最坏情况下需要存储所有顶点，所以空间复杂度为 $O(n)$ 。

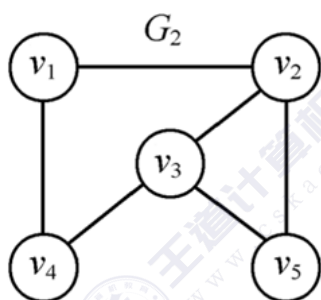
应用场景

- **寻找连通分量：**在一个非连通图中，可以确定有多少个独立的子图。
- **检测环路：**在遍历过程中，如果遇到一个已经访问过且不在当前路径上的祖先结点的顶点，则说明存在环。
- **拓扑排序：**用于有向无环图 (DAG)，可以得到一个线性的任务序列。
- **寻找路径：**寻找从一个点到另一个点的路径。

5.3.2 广度优先遍历

基本思想：相当于树与二叉树的层次遍历。从一个起始顶点开始，首先访问其所有直接相邻的邻接点，然后按距离逐层向外扩展，访问更远的顶点。其核心是“**逐层扩散，齐头并进**”。这非常类似于树的**层次遍历**，也像是往平静的水面扔一块石头，水波从中心一圈一圈向外扩散。

实现原理：BFS 的“先进先出”特性需要用**队列**来实现。假设未扫描的结点用 **蓝色**，表示平静；已加入队列的用 **橙色**，表示跃跃欲试待处理；已处理完的用 **绿色**，表示已经探索完毕。



算法步骤

1. 选择一个起始（蓝色）顶点 v ，将其加入队列，标记为已入队（蓝→橙）。
2. 当队列不为空时，循环执行以下操作：
 - a. 从队头取出一个顶点 u ，访问该结点。
 - b. 遍历 u 的所有邻接点 w ：如果 w 未入队，则访问 w ，并将其加入队列，标记为已入队（蓝→橙）。遍历完后 u 由橙→绿。

邻接表的 BFS 遍历：

```
bool visited[MAXV] = {false}; // 表示顶点是否已入队
void BFS_ALGraph(ALGraph G, int v)
{
    int queue[MAXV], front = 0, rear = 0;

    // 1. 将 v 入队，标记
    queue[rear++] = v;
    visited[v] = true;

    // 2. 如果队列不为空则循环
    while (front < rear)
    {
        // 2-a. 选择队头元素 u 访问
        int u = queue[front++];
        visit(u);

        // 2-b. 将 u 出发可达到的所有未访问顶点加入队列
        ArcNode *p = G.VerticesList[u].first;
        while (p != NULL)
        {

```

```

        if (visited[p->adjvex] != true)
        {
            queue[rear++] = p->adjvex;
            visited[p->adjvex] = true;
        }
        p = p->next;
    }
}

void ans(ALGraph G)
{
    // 无向图调用 BFS 次数等于连通分量数
    for (i=0; i < G.numVertices; i++)
        if (visited[i] != true) BFS_ALGraph(G, i);
}

```

复杂度分析：假设图有 V 个顶点和 E 条边。

- 时间复杂度：
 - 邻接表：每个顶点和每条边都只会被访问一次，所以时间复杂度为 $O(n+e)$ 。
 - 邻接矩阵：同样需要查找每个顶点的所有邻接点，时间复杂度为 $O(n^2)$ 。
- 空间复杂度：主要取决于队列的大小，最坏情况下需要存储所有顶点，所以空间复杂度为 $O(n)$ 。

应用场景

- 寻找无权图的最短路径：BFS 保证了最先找到目标顶点的路径就是最短的（边数最少）。
- 检测二分图：一个图是二分图，当且仅当它不包含奇数长度的环。我们可以使用 BFS 对图用两种颜色进行着色。

特性	深度优先遍历 (DFS)	广度优先遍历 (BFS)
数据结构	栈	队列
搜索方式	莽夫，无路可走再回溯	谨慎，一层层前进
空间复杂度	$O(n)$	$O(n)$
时间复杂度	邻接表 $O(n+e)$ ，邻接矩阵 $O(n^2)$	邻接表 $O(n+e)$ ，邻接矩阵 $O(n^2)$
应用	拓扑排序、找环、找连通分量	无权图最短路径，二分图
访问顺序	不唯一，取决于邻接点的顺序和起始点	不唯一，取决于邻接点的顺序，但层级固定

5.3.3 最短路径

无权图

对于无权图（其实也可以用于权值相同的图），最短路径是指从一个顶点到另一个顶点经过的边数最少的路径。由于每条边的“权重”可以视为 1，我们可以使用广度优先搜索（BFS）来寻找最短路径。BFS 按层级遍历图，天然地保证了最先到达某个顶点的路径就是最短路径。

思路：使用广度优先搜索（BFS）。从源点开始，逐层向外扩展，第一次访问到某个顶点时，所经过的路径就是从源点到该顶点的最短路径。

步骤：

1. 初始化距离数组 `dist`，`dist[start] = 0`，所有其他顶点的距离为无穷大 ∞ 。同时，初始化路径数组 `path` 用于记录前驱结点。
2. 创建一个队列，并将源点 `start` 入队。
3. 队列不为空则循环执行：
 - a. 从队头取出一个顶点 `u`。
 - b. 遍历 `u` 的所有邻接点 `v`。如果 `v` 尚未入队则将 `v` 入队并更新距离。

- 邻接矩阵代码如下，时间复杂度为 $O(n^2)$ ：

```
// 使用 BFS 计算无权图的单源最短路径
void BFS_ShortestPath(MGraph G, int start)
{
    int queue[MAXV];
    int visited[MAXV]={false};
    int dist[MAXV], path[MAXV];
    int front = 0, rear = 0;

    // 1. 初始化
    for (int i = 0; i < G.numVertices; i++)
    {
        dist[i] = INT_MAX;
        path[i] = -1;
    }

    // 2. 将源点入队
    dist[start] = 0;
    queue[rear++] = start;           // 源点入队
    visited[start] = true;

    // 3. 当队列不为空时，循环执行
    while (front < rear)           // 队列不为空
    {
        // 3-a. 队头元素 u 出队
        int u = queue[front++];    // 队头元素出队
```

```

// 3-b.遍历 u 的所有相邻结点 v, 如果 v 尚未入队过则将 v 入队并更新距离
for (int v = 0; v < G.numVertices; v++)
    if (visited[v] == false && G.Edge[u][v] == 1)
    {
        dist[v] = dist[u] + 1;
        path[v] = u;
        queue[rear++] = v; // 新访问的顶点入队
        visited[v] = true;
    }
}
根据 dist 数组输出长度, 根据 path 数组输出路径
}

```

如果 $\text{dist}[v]$ 为无穷大, 就说明未访问过, 因此它可以实现 `visited` 数组的功能, 后者是冗余的。所以其实可以基于上面的代码进一步改写。

邻接表代码如下, 时间复杂度为 $O(n+e)$:

```

// 使用BFS计算无权图的单源最短路径
void BFS_ShortestPath_AL(ALGraph G, int start)
{
    int queue[MAXV];
    int dist[MAXV], path[MAXV];
    int front = 0, rear = 0;

    // 1.初始化
    for (int i = 0; i < G.numVertices; i++)
    {
        dist[i] = INT_MAX;
        path[i] = -1;
    }

    // 2.将源点入队
    dist[start] = 0;
    queue[rear++] = start; // 源点入队

    // 3.当队列不为空时, 循环执行
    while (front < rear) // 队列不为空
    {
        // 3-a.队头元素 u 出队
        int u = queue[front++]; // 队头元素出队

        // 3-b.遍历 u 的所有相邻结点 v, 如果 v 尚未入队过则将 v 入队并更新距离
        p = G.VerticesList[u].first;
        while (p != NULL)
        {
            int v = p->adjvex; // 获取邻接点v的下标
            // 如果v尚未被访问过
            if (dist[v] == INT_MAX)
            {
                dist[v] = dist[u] + 1; // v的距离是u的距离+1
                path[v] = u; // v的前驱是u
                queue[rear++] = v; // 将新访问的顶点v入队
            }
            p = p->next;
        }
    }
}

```

```
}
```

根据 `dist` 数组输出长度，根据 `path` 数组输出路径

```
}
```

有权图-Dijkstra算法

在一个有权图中，我们要求从一个点出发到达其他所有点需要花费的时间。把这个过程想象成在一张地图上找从你家（起点）到所有其他地方的最短花费时间。

1. 初始化

- 在你家旁边标注“用时 0 分钟”。
- 在其他所有地点旁边标注“用时无限长”。
- 准备一个空集合“已确定最短时间的地点”。

2. 循环：重复以下三步，直到所有地点都已确定。

第一步 选择 从所有尚未确定的地点中，挑出当前记录用时最短的那个。第一次选的必然是你家（用时 0）。

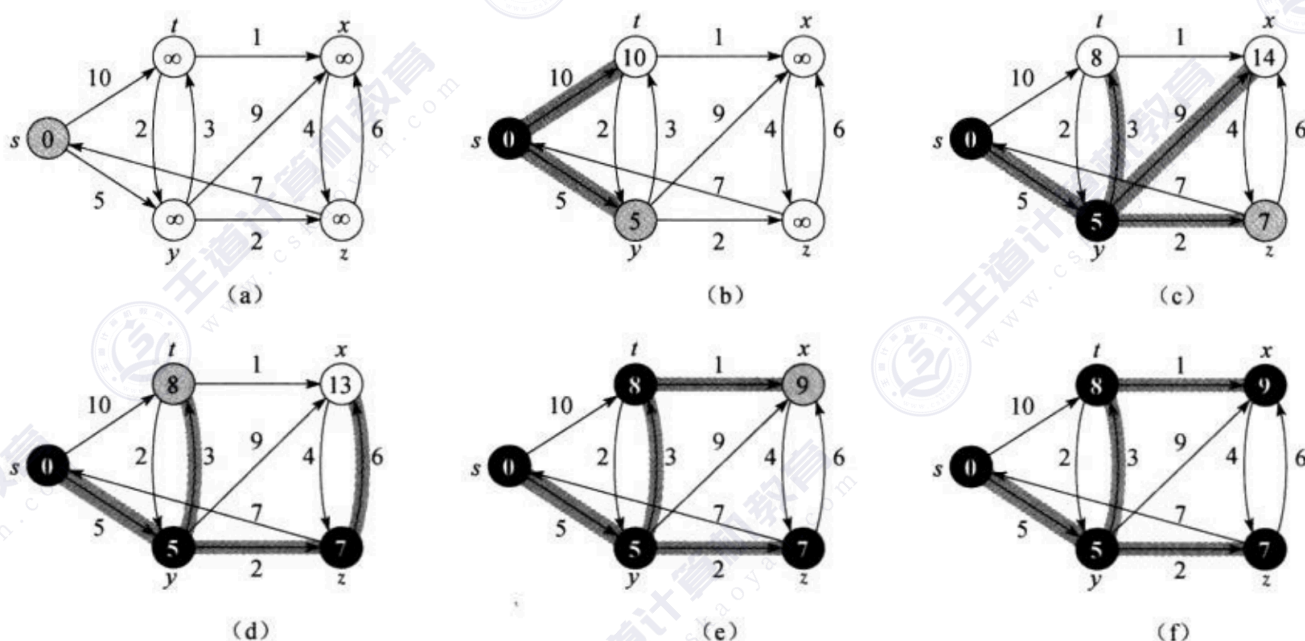
第二步 确定 将选出的地点（记为 A）加入“已确定”集合，之后不再考察。

第三步 更新 对于 A 能直接到达的、尚未确定的每个邻近地点 B：计算“从你家到 A 的时间”+“从 A 到 B 的时间”。若这个总时间比清单上 B 当前记录的用时更短，则更新。

3. 结束 当地图上所有能到达的地点都加入了“已确定”集合，算法结束。此时清单上的记录就是从你家到每个地点的最短时间。

这个算法的精髓（贪心策略）在于：**每次都选择当前看起来最近的那个点，并认定它的最短路径已经找到，然后用它来更新其他邻居点的路径。**

从已知世界 S 扩展到未知世界，每次都选择距离源点最近的：



步骤：

1. 初始化。初始化距离数组 dist, $\text{dist}[s] = 0$, 所有其他顶点的距离为无穷大 ∞ 。同时，初始化路径数组 path 用于记录前驱结点。

2. 循环直到所有顶点都已确定： a. 每次选择距离 $dist$ 最小的、未选择过已确定顶点 u 。 b. 将顶点 u 视为已确定的顶点，之后不再考虑。 c. 将 u 出发的所有邻接点 v 做松弛：

```
if (dist[u] + w(u,v) < dist[v])
    dist[v] = dist[u] + w(u,v);
```

邻接矩阵代码如下，时间复杂度为 $O(n^2)$ ：

```
int Dijkstra(MGraph G, int start)
{
    int dist[MAXV], path[MAXV];
    // 1. 初始化
    bool visited[MAXV] = {false}; // 记录顶点是否已确定
    for (int i = 0; i < G.numVertices; i++)
    {
        dist[i] = INT_MAX;
        path[i] = -1;
    }
    // 将起点到自身的距离设为 0
    dist[start] = 0;

    // 2. 循环 G.numVertices 次，每次找到一个最短路径并更新其他顶点
    for (int i = 0; i < G.numVertices; i++)
    {
        // a. 在未访问的顶点中，找到距离起点最近的顶点 u
        // 在第一次循环中，由于 dist[start] = 0, u 会被自然地赋值为 start
        int u = -1;
        int min_dist = INT_MAX;
        for (int j = 0; j < G.numVertices; j++)
        {
            if (visited[j] == false && dist[j] < min_dist)
            {
                min_dist = dist[j];
                u = j;
            }
        }

        if (u == -1) return -1; // 如果找不到下一个顶点说明是非连通图

        // b. 标记 u 为已访问
        visited[u] = true;

        // c. 松弛操作：更新从 u 出发能够到达的所有未访问顶点的距离
        for (int v = 0; v < G.numVertices; v++)
        {
            // 条件：v 未被访问过，且存在从 u 到 v 的边，
            // 并且通过 u 到达 v 的路径比当前记录的路径更短
            if (!visited[v] && G.Edge[u][v] != 0 && dist[u] + G.Edge[u][v] < dist[v])
            {
                // 更新 v 的最短距离和前驱结点
                dist[v] = dist[u] + G.Edge[u][v];
                path[v] = u;
            }
        }
    }
}
```

```

}
根据 dist 数组输出长度, 根据 path 数组输出路径
return 0;
}

```

请大家思考一个问题, Dijkstra 算法中的 visited 数组能否删去? 为什么?

邻接表代码如下, 时间复杂度为 $O(n^2)$:

```

int Dijkstra_AL(ALGraph G, int start)
{
    int dist[MAXV], path[MAXV];
    bool visited[MAXV] = {false}; // 记录顶点是否已确定

    // 1. 初始化
    for (int i = 0; i < G.numVertices; i++)
    {
        dist[i] = INT_MAX;
        path[i] = -1;
    }

    // 将起点到自身的距离设为 0
    dist[start] = 0;

    // 2. 主循环: 每次从未收集的顶点中选择一个距离最小的
    for (int i = 0; i < G.numVertices; i++)
    {
        // a. 在未访问的顶点中, 找到距离起点最近的顶点 u
        int min_dist = INT_MAX;
        int u = -1;
        for (int j = 0; j < G.numVertices; j++)
        {
            if (visited[j] == false && dist[j] < min_dist)
            {
                min_dist = dist[j];
                u = j;
            }
        }

        if (u == -1) return -1; // 如果找不到下一个顶点说明是非连通图

        // b. 标记 u 为已访问
        visited[u] = true;

        // c. 松弛操作: 更新从 u 出发能够到达的所有未访问顶点的距离
        for (ArcNode *p = G.VerticesList[u].first; p != NULL; p = p->next)
        {
            int v = p->adjvex;
            if (visited[v] == false && dist[u] + p->weight < dist[v])
            {
                dist[v] = dist[u] + p->weight;
                path[v] = u;
            }
        }
    }

    根据 dist 数组输出长度, 根据 path 数组输出路径
}

```

```
return 0;  
}
```

对于邻接表来说，时间复杂度主要瓶颈在于选择距离最短的点，这一步可以使用优先队列优化（即堆），优化后的时间复杂度为 $O(e+m\log m)$ 。

5.3.4 最小生成树

给定一个连通无向图 $G=(V, E)$ ，其生成树是：

- 一个包含图中所有 n 个顶点、 $n-1$ 条边的无环子图。
- 每个顶点都能通过生成树中的边互相连通。

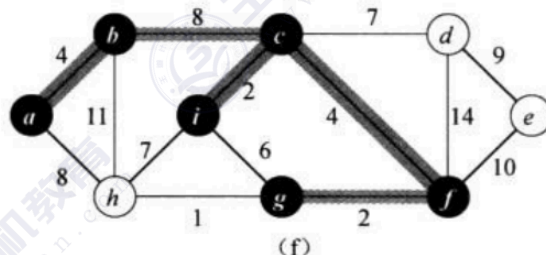
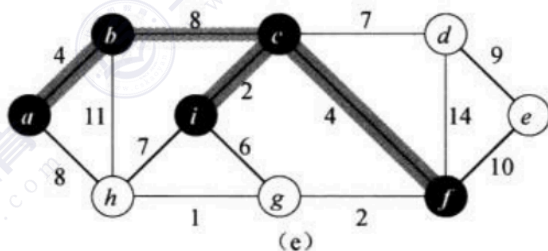
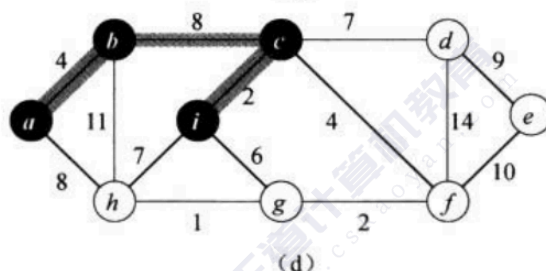
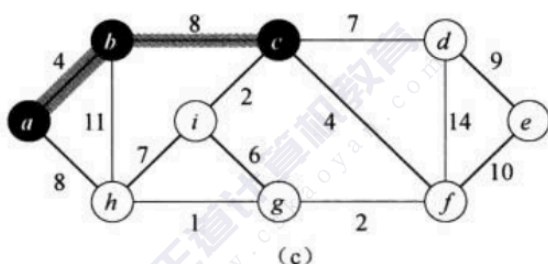
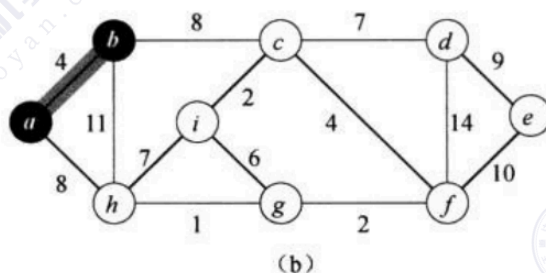
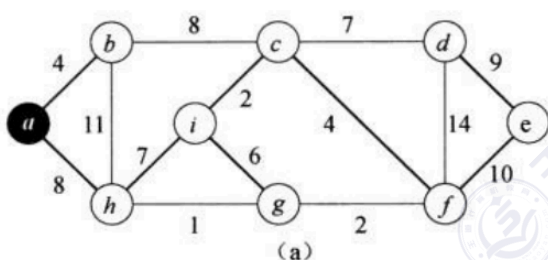
在所有可能的生成树中，边的权值之和最小的那一棵生成树，称为**最小生成树**。

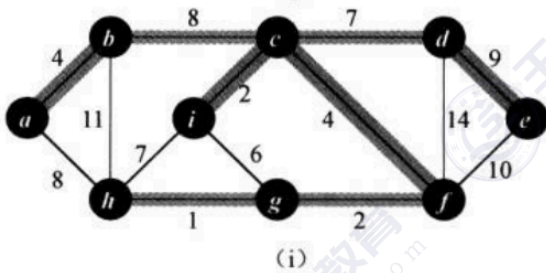
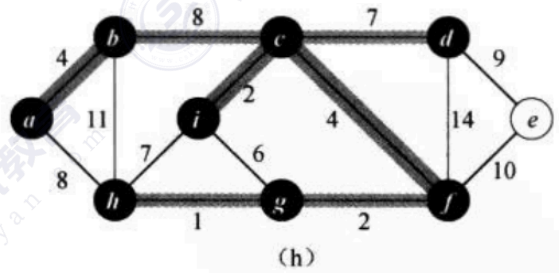
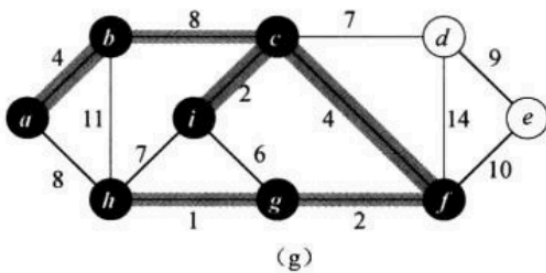
Prim算法

核心思想：从顶点的角度出发，初始时已选顶点集合只含起始顶点。每次将一个顶点纳入已选顶点集合，重复 $n-1$ 次直到该集合包含所有顶点。

步骤：

1. 初始化。从源点出发，标记该顶点已选。
2. 重复执行 $n-1$ 次：
a. 每次从**未选顶点**中找出距离**已选顶点集合**最近的顶点 k ，将该顶点标记已选。
b. 更新未选中顶点到**已选顶点集合**的距离。





邻接矩阵

伪代码如下：

```
void Prim(MGraph G)
{
    int cost[MAXV];
    bool visited[MAXV] = {false};

    // 1. 初始化。从 0 号顶点出发，加入已选集合。
    int start = 0;
    visited[start] = true;
    for (int i = 0; i < G.numVertices; i++)
        if (G.Edge[start][i] != 0) cost[i] = G.Edge[start][i];
        else cost[i] = INT_MAX; // start->i不存在边，是无穷大
    cost[start] = 0;           // 起点

    // 2. 循环选择 n-1 条边
    for (int i = 1; i < G.numVertices; i++)
    {
        int min = INT_MAX, k = -1;
        // 2-a. 从未选顶点中选择 cost 最小的顶点 k，并更新
        for (int j = 0; j < G.numVertices; j++)
        {
            if (visited[j] == false && cost[j] < min)
            {
                min = cost[j];
                k = j;
            }
        }
        if (k == -1) break; // 选不出顶点，说明不是连通图
        visited[k] = true;
        输出边 G.numVertices[k]

        // 2-b. 更新未选中顶点到已选顶点集合的距离 cost
        for (int j = 0; j < G.numVertices; j++)
        {
            if (visited[j] == false && G.Edge[k][j] < cost[j])
                cost[j] = G.Edge[k][j];
        }
    }
}
```

```
}  
cost 数组元素相加即最小生成树长度  
}
```

时间复杂度: $O(n^2)$

✎ Prim 算法和 Dijkstra 算法的比较

- Prim 每次从未选顶点中选择距离已选集合最近的点 (cost 最小), 而 Dijkstra 算法每次从未选顶点中选择距离源点最近的顶点 (dist 最小)。
- Prim 的 cost 记录的是边的长度, Dijkstra 的 dist 记录的是路径的长度。
- Prim 中 cost[k] 的更新条件是 k 距离本次选择的顶点 j 比距离已选集合中的其他顶点更近: $\text{cost}[j] > w(k, j)$ 。

Dijkstra 中 dist[v] 的更新条件是借道 u 之后源点离 v 的距离比借道 u 之前更小:
 $\text{dist}[u] + w(u, v) < \text{dist}[v]$ 。

- Prim 和 Dijkstra 都是贪心算法每次选择 xx 最小的, 邻接表的写法都可以使用小根堆来优化选择的过程。

Kruskal算法

核心思想：从边的角度出发，初始时已选边集合不包含任何边。每次将一条最小的、加入集合后不成环的边纳入已选边集合，重复 $n-1$ 次。

步骤：

1. 题目给出从小到大排序的边数组或给出优先队列（堆）。
2. 初始化并查集，初始时所有顶点属于不同集合。
3. 循环，直到选出 $n-1$ 条不成环的边：
a. 对于还未处理的最小边，使用并查集判断该边的两个端点分别属于哪个集合。
b. 如果不属于同一个集合则选择该边、合并这两个集合。

邻接表

伪代码如下（忽略快速排序）：

```
int parent[MAXV];

// 边结构体定义
typedef struct {
    int u, v;           // 边的两个顶点
    int weight;         // 边的权值
} Edge;

// 查找根结点
int Find(int x)
{
    if (parent[x] != x)
        parent[x] = Find(parent[x]); // 路径压缩
    return parent[x];
}

// 并查集：合并集合
void Union(int x, int y)
{
    parent[Find(y)] = Find(x);
}

void Kruskal(ALGraph G)
{
    Edge edges[MAXE]; // 边集数组
    int count = 0;    // 统计已纳入生成树中的边数

    // 1. 题目给出从小到大排序的边数组或给出优先队列（堆）
    // 不妨假设所有的边已经在数组 edges 中，边的数量为 m
    Qsort(edges[], 0, G.numEdges-1);

    // 2. 初始化并查集，初始时所有顶点属于不同集合。
    for (int i = 0; i < G.numVertices; i++)
        parent[i] = i;

    // 3. 循环，直到选出 n-1 条不成环的边
```

```

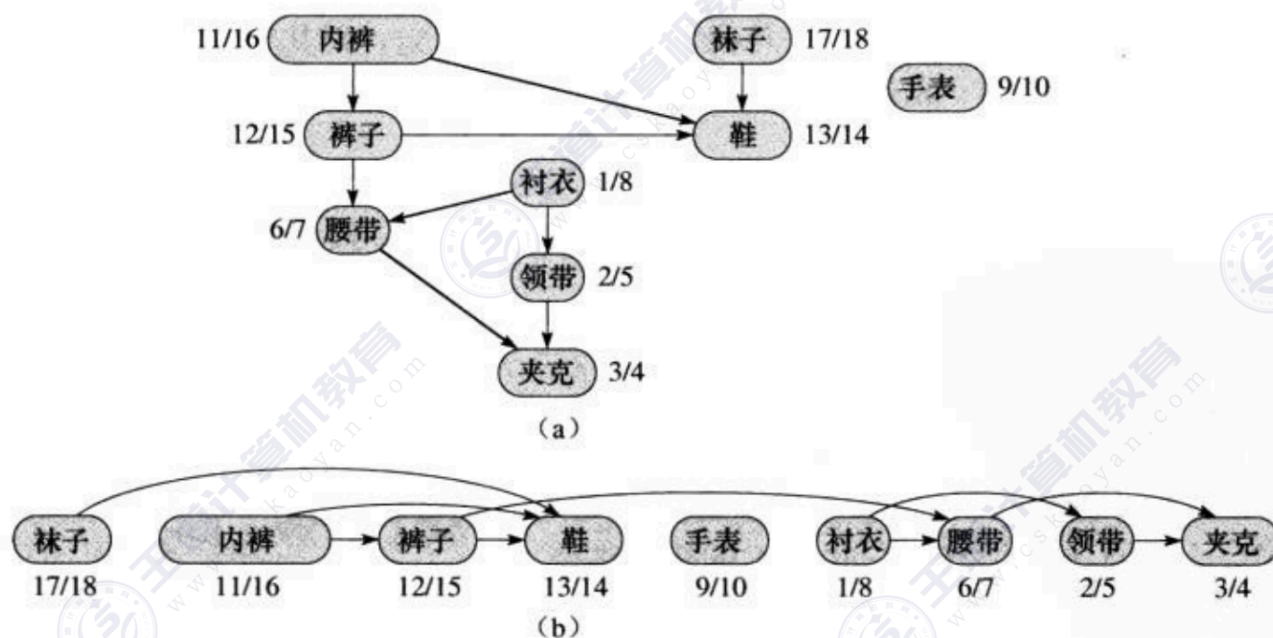
for (int i = 0; i < m; i++)
{
    // 3-a. 对于还未处理的最小边，使用并查集判断该边的两个端点分别属于哪个集合
    int rootU = Find(edges[i].u);
    int rootV = Find(edges[i].v);

    // 3-b. 如果不属于同一个集合则选择该边、合并这两个集合
    if (rootU != rootV)
    {
        Union(rootU, rootV);
        将该边加入到生成树中
        count++;
        if (count == G.numVertices-1) break;    // 已选择 n-1 条边
    }
}
输出最小生成树
}

```

第一步在考试中不太可能考察，主要考察第二、三步。

5.3.5 拓扑排序



核心目的：

- 解决有向无环图（DAG）中顶点的**先后顺序问题**，得到拓扑序列。
- 如果顶点 a 到顶点 b 存在有向边，则在拓扑序列中，顶点 a 会在顶点 b 前面。
- 如果该有向图无环，则拓扑序列存在。如果该有向图有环，则拓扑序列不存在。

拓扑序列	条件
存在且唯一	每次选择结点时都只有一个结点可选
存在且不唯一	选择结点时不止一个结点可选
不存在	图中有环，环中的顶点未被访问

步骤：

1. 统计每个顶点的入度。
2. 将入度为 0 的顶点入队。
3. 队列出队一个顶点，访问它，并将它的出边所指向顶点的入度减 1。
4. 若减到 0，再次入队。
5. 队列为空，排序完成；若有未访问顶点，存在环。

邻接表

时间复杂度： $O(n+e)$

代码如下：

```

void TopoSort(ALGraph G)
{
    int indegree[MAXV] = {0};

    // 计算每个顶点的入度
    for (int i = 0; i < G.numVertices; i++)
    {
        ArcNode *p = G.VerticesList[i].first;
        while (p != NULL)
        {
            indegree[p->adjvex]++;
            p = p->next;
        }
    }

    // 将所有入度为0的顶点入队
    int queue[MAXV], front = 0, rear = 0;
    for (int i = 0; i < G.numVertices; i++)
        if (indegree[i] == 0)
            queue[rear++] = i;

    while (front < rear)
    {
        int u = queue[front++];
        输出顶点u

        // 此处可判断是否只有一个顶点可选, 即front+1==rear

        for (ArcNode *p = G.VerticesList[u].first; p != NULL; p = p->next)
        {
            int v = p->adjvex;
            if (--indegree[v] == 0)
                queue[rear++] = v;
        }
    }

    if (rear < G.numVertices)
        拓扑序列不存在, 该有向图存在环
    }
}

```

*邻接矩阵

时间复杂度: $O(n^2)$

代码如下:

```

void TopoSort(MGraph G)
{
    int indegree[MAXV] = {0};

    // 计算每个顶点的入度
    for (int i = 0; i < G.numVertices; i++)
        for (int j = 0; j < G.numVertices; j++)
            if (G.Edge[i][j] != 0) // 存在边 i->j
                indegree[j]++;
}

```

```

// 将所有入度为0的顶点入队
int queue[MAXV], front = 0, rear = 0;
for (int i = 0; i < G.numVertices; i++)
    if (indegree[i] == 0)
        queue[rear++] = i;

while (front < rear)
{
    int u = queue[front++];
    输出顶点 u

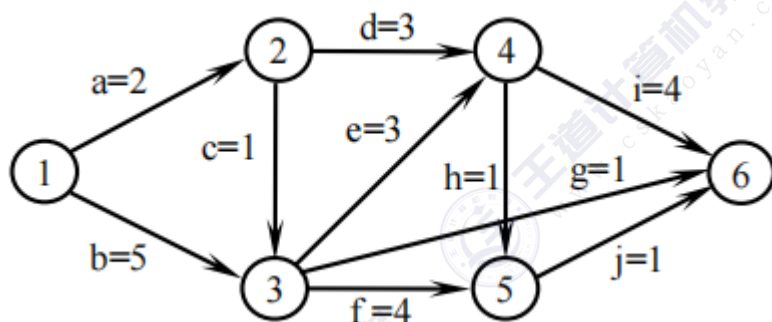
    // 此处可判断是否存在唯一拓扑序列 (front+1==rear)

    for (int v = 0; v < G.numVertices; v++)
    {
        if (G.Edge[u][v] != 0)
        {
            if (--indegree[v] == 0)
                queue[rear++] = v;
        }
    }
}

if (rear < G.numVertices)
    拓扑序列不存在, 该图存在环
}

```

5.3.6 * 关键路径



以边表示活动的网叫 AOE 网，它是**带权无环图**，顶点表示事件，弧表示活动，权表示活动持续的时间。通常，AOE-网可用来估算工程的完成时间。

在正常的情况（无环）下，网中只有一个入度为0的点，称作**源点**，也只有一个出度为0的点，称作**汇点**。在AOE-网中，一条路径各弧上的权值之和称为该路径的带权路径长度（后面简称路径长度）。要估算整项工程完成的最短时间，就是要找一条从源点到汇点的带权路径长度最长的路径，称为**关键路径**。

关键路径上的活动叫作关键活动，这些活动是影响工程进度的关键，它们的提前或延期将使整个工程提前或延期。

目的：关键路径用于工程调度、项目管理等任务中，找出一条**耗时最长**的必经路径，控制整个项目进度。

步骤：

1. 先进行拓扑排序，得到拓扑序列；
2. 根据拓扑序列，正向计算每个顶点的最早发生时间 ve ；
3. 根据拓扑序列反过来逆向计算每个顶点的最迟发生时间 vl ；
4. 依次判断每条边 $u \rightarrow v$ ：若某活动的最早时间 $ve(u)$ 等于最迟时间 $vl(v) - w(u, v)$ ，则为关键活动。

伪代码：

仅供参考，重在理解过程，代码考察概率不大，万一考察也只是考察部分步骤（如已知拓扑序列求关键路径）。

```
void CriticalPath(ALGraph G)
{
    int indegree[MAXV] = {0};           // 存储每个顶点的入度
    int ve[MAXV] = {0};                 // 各顶点最早开始时间
    int vl[MAXV];                       // 各顶点最迟开始时间
    int topo[MAXV];                     // 存储拓扑排序序列
    int front = 0, rear = 0;

    // 第一步：统计入度
```

```

for (int i = 0; i < G.numVertices; i++)
{
    ArcNode *p = G.VerticesList[i].first;
    while (p != NULL)
    {
        indegree[p->adjvex]++;
        p = p->next;
    }
}

// 第二步：拓扑排序，并计算每个顶点的最早发生时间 ve
for (int i = 0; i < G.numVertices; i++)
    if (indegree[i] == 0)
        topo[rear++] = i;

while (front < rear)
{
    int u = topo[front++];
    ArcNode *p = G.VerticesList[u].first;
    while (p != NULL)
    {
        int v = p->adjvex;
        if (--indegree[v] == 0)
            topo[rear++] = v;

        // 更新 ve 值
        if (ve[u] + p->weight > ve[v])
            ve[v] = ve[u] + p->weight;

        p = p->next;
    }
}

// 如果 rear < G.numVertices，图中有环，不存在关键路径
if (rear < G.numVertices)
{
    输出：图中存在环
    return;
}

// 第三步：初始化 vl 数组
for (int i = 0; i < G.numVertices; i++)
    vl[i] = ve[G.numVertices - 1]; // 默认项目总工期

// 第四步：逆拓扑序更新 vl 值
for (int i = G.numVertices - 1; i >= 0; i--)
{
    int u = topo[i];
    ArcNode *p = G.VerticesList[u].first;
    while (p != NULL)
    {
        int v = p->adjvex;
        if (vl[v] - p->weight < vl[u])
            vl[u] = vl[v] - p->weight;

        p = p->next;
    }
}

```

```

// 第五步：输出关键活动
for (int u = 0; u < G.numVertices; u++)
{
    ArcNode *p = G.VerticesList[u].first;
    while (p != NULL)
    {
        int v = p->adjvex;
        int e = ve[u];
        int l = vl[v] - p->weight;
        if (e == l)
            输出活动<u,v>是关键活动
        p = p->next;
    }
}
}

```

时间复杂度： $O(n+e)$

5.4 章节练习

5.4.1 习题

【习题 5-1】 给定一个用邻接表保存的有向图 G，设计算法 int OddInDegree(ALGraph G)，返回出度为奇数的顶点个数。邻接表定义如下：

```
typedef struct ArcNode
{
    int adjvex;           // 该弧所指向的顶点
    struct ArcNode *nextarc; // 指向下一条弧的指针
}ArcNode;                // 边结点

typedef struct
{
    int data;             // 顶点信息
    ArcNode *firstarc;    // 指向第一条依附该顶点的弧的指针
}VNode;                  // 顶点结点

typedef struct
{
    VNode VerticesList[MAXV]; // 顶点数组
    int numVertices, arcnum;  // 图的当前顶点数和弧数
}ALGraph;
```

- (1) 给出算法的基本设计思想。
- (2) 采用 C 或 C++ 语言描述算法，关键之处给出注释。
- (3) 说明所设计算法的时间复杂度。

【习题 5-2】 给定一个用邻接表保存的无向图 G，设计算法 int IsConnected(ALGraph G) 判断该图是否连通，若连通则返回 1，否则返回 0。邻接表定义如下：

```
typedef struct ArcNode
{
    int adjvex;           // 该弧所指向的顶点
    struct ArcNode *nextarc; // 指向下一条弧的指针
}ArcNode;                // 边结点
typedef struct
{
    int data;             // 顶点信息
    ArcNode *firstarc;    // 指向第一条依附该顶点的弧的指针
}VNode;                  // 顶点结点
typedef struct
{
    VNode VerticesList[MAXV]; // 顶点数组
    int numVertices, arcnum;  // 图的当前顶点数和弧数
}ALGraph;
```

- (1) 给出算法的基本设计思想。
- (2) 采用 C 或 C++ 语言描述算法，关键之处给出注释。
- (3) 说明所设计算法的时间复杂度。

【习题 5-3】 给定一个用邻接表保存的无向图 G，设计算法 int HasCycle(ALGraph * G) 判断该图是否含有环，若含有环则返回 1，否则返回 0。邻接表定义如下：

```
typedef struct ArcNode
{
    int adjvex;           // 该弧所指向的顶点
    struct ArcNode *nextarc; // 指向下一条弧的指针
}ArcNode;                // 边结点
typedef struct
{
    ArcNode *VerticesList[MAXV]; // 顶点数组
    int numVertices, arcnum;      // 图的当前顶点数和弧数
}ALGraph;
```

- (1) 给出算法的基本设计思想。
- (2) 采用 C 或 C++ 语言描述算法，关键之处给出注释。
- (3) 说明所设计算法的时间复杂度。

【习题 5-4】 给定一个用邻接表保存的带权无向图 G，你可以直接调用该函数 ArcNode* FindEdge() 得到一个指针，指向当前未处理过的边中**权值最小**的那条边（内部实现细节不需要考虑）。设计算法 int MSTWeight(ALGraph G) 判断返回最小生成树的权值之和。邻接表定义如下：

```
typedef struct ArcNode
{
    int ivex;           // 该弧起始的顶点
    int jvex;           // 该弧所指向的顶点
    struct ArcNode *nextarc; // 指向下一条弧的指针
    int weight;         // 边权值
}ArcNode;              // 边结点

typedef struct
{
    int data;           // 顶点信息
    ArcNode *firstarc;  // 指向第一条依附该顶点的弧的指针
}VNode;                // 顶点结点

typedef struct
{
    VNode VerticesList[MAXV]; // 顶点数组
    int numVertices, arcnum;  // 图的当前顶点数和弧数
}ALGraph;
```

(1) FindEdge() 函数每次会从某个数据结构中取出最小边，应该使用什么数据结构？为什么？ (2) 给出算法的基本设计思想。 (3) 采用 C 或 C++ 语言描述算法，关键之处给出注释。

【* 习题 5-5】 设无向图 $G=(V, E)$ ，如果顶点可被分割为两个互不相交的子集，并且图中每条边的两个顶点 i 和 j 分别属于这两个不同的顶点集，则称图 G 为一个二分图。无向图 G 为二分图的充要条件是： G 中至少包含两个顶点且 G 中任意一个简单回路所包含的边数为偶数。图 G 采用邻接矩阵存储，类型定义如下：

```
typedef struct {  
    int numV, numE;           // 顶点数和边数  
    char VerticesList[MAXV];  // 顶点信息  
    int Edge[MAXV][MAXV];     // 权值矩阵  
}MGraph;
```

请设计算法 `int IsBipartite (MGraph G)`，判断无向图 G 是否为二分图，是则返回 1，否则返回 0。（1）给出算法的基本设计思想。（2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

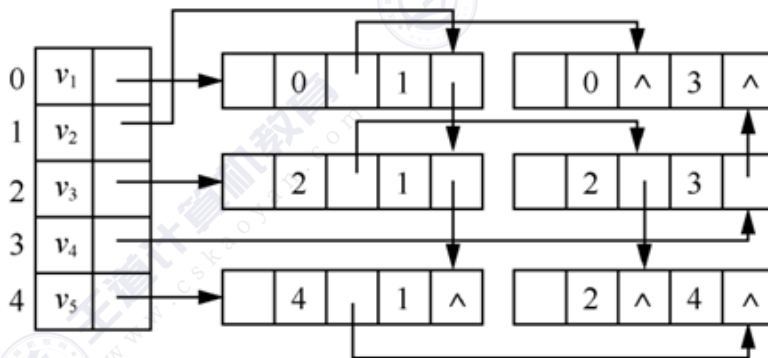
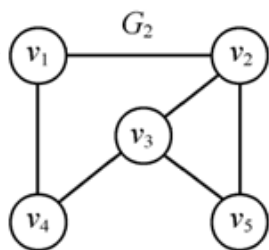
【习题 5-6】 请设计算法：int components(ALGraph G)，对给定的无向图 G，返回其中连通分量的个数。邻接表定义如下：

```
typedef struct ArcNode
{
    int adjvex;           // 该弧所指向的顶点
    struct ArcNode *nextarc; // 指向下一条弧的指针
}ArcNode;                // 边结点
typedef struct
{
    int data;             // 顶点信息
    ArcNode *firstarc;    // 指向第一条依附该顶点的弧的指针
}VNode;                  // 顶点结点
typedef struct
{
    VNode VerticesList[MAXV]; // 顶点数组
    int numVertices, arcnum;  // 图的当前顶点数和弧数
}ALGraph;
```

- (1) 给出算法的基本设计思想。
- (2) 采用 C 或 C++ 语言描述算法，关键之处给出注释。
- (3) 说明所设计算法的时间复杂度。

【* 习题 5-7】 已知无向图 G 类型定义如下：

```
typedef struct EBox
{
    int mark;           // 用于遍历标记
    int ivex, jvex;     // 边依附的两个顶点位置
    struct EBox *ilink, *jlink; // 分别指向依附于ivex、jvex的下一条边
} EBox;
typedef struct VexBox
{
    char data;          // 顶点信息
    EBox *firstEdge;    // 指向第一条依附该顶点的边
} VexBox;
typedef struct
{
    VexBox adjMulList[MAXV]; // 顶点表
    int numVertices, numEdges; // 顶点数和边数
} AMLGraph;
```



请设计算法：int KDegree(AMLGraph G, int k)，对给定的无向图 G，返回图 G 中顶点下标为 k 的顶点的度。例如图 G_2 ，k=1 时，对应顶点 v_2 ，度为 3，返回 3。要求：（1）请问图的这种存储结构叫什么？（2）给出算法的基本设计思想。（3）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

【* 习题 5-8】有向完全图的定义是：在有向图中，任意两个顶点 u 和 v 之间，必须同时存在 $u \rightarrow v$ 和 $v \rightarrow u$ 两条弧。对于有向图 G ，请设计算法：int IsComplete(OLGraph G) 判断它是不是有向完全图，如果是则返回 1，否则返回 0。请回答：（1）应该采用以下哪种数据结构保存：邻接多重表还是十字链表？（2）写出该数据结构定义。（3）采用 C 或 C++ 语言描述算法，关键之处给出注释。

5.4.2 真题

【2021 年 41 题】（15 分）已知无向连通图 G 由顶点集 V 和边集 E 组成， $|E| > 0$ ，当 G 中度为奇数的顶点个数为不大于 2 的偶数时， G 存在包含所有边且长度为 $|E|$ 的路径（称为 EL 路径）。设图 G 采用邻接矩阵存储，类型定义如下：

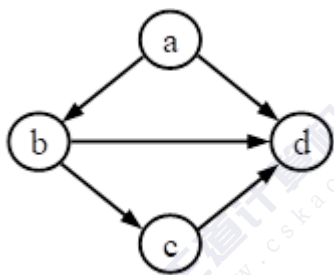
```
typedef struct{           // 图的定义
    int numVertices, numEdges; // 图中实际的顶点数和边数
    char VerticesList[MAXV]; // 顶点表。MAXV为已定义常量
    int Edge[MAXV][MAXV]; // 邻接矩阵
}MGraph;
```

请设计算法 `int IsExistEL(MGraph G)`，判断 G 是否存在 EL 路径，若存在，则返回 1，否则返回 0。要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。

【2023 年 41 题】 (13 分) 已知有向图 G 采用邻接矩阵存储，类型定义如下：

```
typedef struct                // 图的类型定义
{
    int    numVertices, numEdges; // 图的顶点数和有向边数
    char    VerticesList[ MAXV ]; // 顶点表，MAXV为已定义常量
    int    Edge[ MAXV ][ MAXV ]; // 邻接矩阵
} MGraph;
```

将图中出度大于入度的顶点称为 K 顶点。例如在下图中，顶点 a 和 b 为 K 顶点。



请设计算法：int printVertices(MGraph G)，对给定的任意非空有向图 G，输出 G 中所有的 K 顶点，并返回 K 顶点的个数。要求：1) 给出算法的基本设计思想。
2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

【2024 年 41 题】 (13分) 2023 年 10 月 26 日, 神舟十七号载人飞船发射取得圆满成功, 再次彰显了中国航天事业辉煌成就。载人航天工程是包含众多子工程的复杂系统工程, 为了保证工程的有序开展, 需要明确各子工程的前导子工程, 以协调各子工程的实施。该问题可以简化、抽象为有向图的拓扑序列问题。已知有向图 G 采用邻接矩阵存储, 类型定义如下。

```
typedef struct          //图的类型定义
{
    int numVertices, numEdges; //图的顶点数和有向边数
    char VerticesList[MAXV]; //顶点表, MAXV为已定义常量
    int Edge[MAXV][MAXV]; //邻接矩阵
} MGraph;
```

请设计算法: int uniquely(MGraph G), 判定 G 是否存在唯一的拓扑序列, 若是, 则返回 1, 否则返回 0。要求如下。1) 给出算法的基本设计思想。(4分) 2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(9分)

5.5 章节答案

5.5.1 习题

习题 5-1

给定一个用邻接表保存的有向图 G ，设计算法 `int OddInDegree(ALGraph G)`，返回出度为奇数的顶点个数。邻接表定义如下：

```
typedef struct ArcNode
{
    int adjvex;           // 该弧所指向的顶点
    struct ArcNode *nextarc; // 指向下一条弧的指针
} ArcNode;               // 边结点
typedef struct
{
    int data;             // 顶点信息
    ArcNode *firstarc;    // 指向第一条依附该顶点的弧的指针
} VNode;                 // 顶点结点
typedef struct
{
    VNode VerticesList[MAXV]; // 顶点数组
    int numVertices, arcnum;   // 图的当前顶点数和弧数
} ALGraph;
```

- (1) 给出算法的基本设计思想。 (2) 采用 C 或 C++ 语言描述算法，关键之处给出注释。
(3) 说明所设计算法的时间复杂度。

【答案】 (1)

1. 设置出度数组 `outdegree`，初始为 0。
2. 遍历所有顶点和边，统计每个顶点的出度。
3. 扫描每个顶点，统计出度为奇数的顶点数。 (2)

```
int OddInDegree(ALGraph G)
{
    // 1. 设置出度数组
    int outdegree[MAXV] = {0};

    // 2. 遍历所有顶点和边，统计每个顶点的出度
    for (int i = 0; i < G.numVertices; i++)
    {
        // 遍历当前顶点 i 发出的所有边
        ArcNode *p = G.VerticesList[i].firstarc;
        while (p != NULL)
        {
```

```

        outdegree[i]++;
        p = p->nextarc; // 处理下一条边
    }
}

// 3.扫描每个顶点，统计出度的顶点数
int count = 0;
for (int i = 0; i < G.numVertices; i++)
    if (outdegree[i] % 2 == 1) count++;
return count;
}

```

(3) $O(n+e)$

习题 5-2

给定一个用邻接表保存的无向图 G ，设计算法 `int IsConnected(ALGraph G)` 判断该图是否连通，若连通则返回 1，否则返回 0。邻接表定义如下：

```

typedef struct ArcNode
{
    int adjvex;           // 该弧所指向的顶点
    struct ArcNode *nextarc; // 指向下一条弧的指针
}ArcNode;               // 边结点

typedef struct
{
    int data;             // 顶点信息
    ArcNode *firstarc;    // 指向第一条依附该顶点的弧的指针
}VNode;                 // 顶点结点

typedef struct
{
    VNode VerticesList[MAXV]; // 顶点数组
    int numVertices, arcnum;  // 图的当前顶点数和弧数
}ALGraph;

```

- (1) 给出算法的基本设计思想。
- (2) 采用 C 或 C++ 语言描述算法，关键之处给出注释。
- (3) 说明所设计算法的时间复杂度。

【解析】判断图是否连通只需要从任意结点出发遍历即可，遍历完成后如果存在结点未访问则说明该图不连通，否则该图连通。遍历可以选择广度优先遍历 BFS 或深度优先遍历 DFS 中的一种，本方法用 BFS 完成。 【答案】 (1)

1. 队列 q 保存待访问结点，将 0 号元素入队。
2. 如果队中还有元素则循环：选择队头元素 u 出队访问并记录，然后将 u 相邻的未入队过的顶点依次入队，循环执行直到队列访问完，BFS 完成。

3. 扫描每个顶点是否都已被访问，如果存在顶点未被访问，则说明不连通，返回 0；否则连通，返回 1。

(2) 代码如下：

```
int isConnected(ALGraph G)
{
    bool visited[MAXV] = {false}; // 记录顶点是否被访问过

    // 1. 队列 q 保存待访问结点，将 0 号元素入队。
    int q[MAXV]; // 队列q存储待访问顶点
    int front = 0, rear = 0; // 队头front和队尾rear
    int u = 0; // 选择0号顶点作为起始顶点
    visited[u] = true;
    q[rear++] = 0;

    // 2. 如果队中还有元素则循环处理
    while (front < rear)
    {
        u = q[front++];
        ArcNode* p = G.VerticesList[u].firstarc;
        while (p != NULL)
        {
            int v = p->adjvex; // v是u的相邻顶点
            if (visited[v] != true) // 如果v未被访问则入队
            {
                visited[v] = true;
                q[rear++] = v;
            }
            p = p->nextarc;
        }
    }

    // 3. 扫描每个顶点是否都已被访问
    for (int i = 0; i < G.numVertices; i++) // 检查所有顶点是否访问过
        if (visited[i] != true)
            return 0; // 图不连通
    return 1; // 图连通
}
```

(3) 时间复杂度： $O(n + e)$

本题 DFS 和 BFS 都可以

习题 5-3

给定一个用邻接表保存的无向图 G，设计算法 int HasCycle(ALGraph * G) 判断该图是否含有环，若含有环则返回 1，否则返回 0。邻接表定义如下：

```
typedef struct ArcNode
{
```

```

int adjvex;           // 该弧所指向的顶点
struct ArcNode *nextarc; // 指向下一条弧的指针
}ArcNode;             // 边结点
typedef struct
{
    ArcNode *VerticesList[MAXV]; // 顶点数组
    int numVertices, arcnum;      // 图的当前顶点数和弧数
}ALGraph;

```

- (1) 给出算法的基本设计思想。(2) 采用 C 或 C++ 语言描述算法，关键之处给出注释。
(3) 说明所设计算法的时间复杂度。

【解析】判断图是否有环可以用 DFS 遍历，遍历完成后如果存在下一个要访问的结点已被访问过则说明存在环，否则不存在环。【答案】(1) 使用深度优先遍历 DFS 算法：从 0 号顶点开始，遍历其所有相邻顶点。如果访问过程中遇到已经访问过的顶点（且不是他的父结点），则说明存在环。一轮 DFS 完成，继续判断是否还有顶点未访问，下一轮 DFS 从未访问的顶点开始。而如果访问完所有结点都未发现环，则说明不存在环。

(2) 代码如下：

```

bool visited[MAXV] = {false}; // 记录顶点是否被访问过
bool DFS(ALGraph *G, int v, int parent)
{
    visited[v] = true;
    ArcNode *p = G->VerticesList[v];
    while (p != NULL)
    {
        int u = p->adjvex;
        if (visited[u] != true)
            if (DFS(G, u, v) == true) return true;
        else;
        else
            if (u != parent) return true;
        p = p->nextarc;
    }
    return false;
}

int HasCycle(ALGraph *G)
{
    for (int i = 0; i < G->numVertices; i++)
        visited[i] = false;
    for (int v = 0; v < G->numVertices; v++)
        if (visited[v] != true)
            if (DFS(G, v, -1) == true) return 1;
    return 0;
}

```

(3) 时间复杂度： $O(n + e)$

题目中给的是 ALGraph G, DFS 中也是 ALGraph G, 所以 G 后面应该写 "->" 而不是 ".", 请思考这样写有何好处。

* 习题 5-4

给定一个用邻接表保存的带权无向图 G, 你可以直接调用该函数 ArcNode* FindEdge() 得到一个指针, 指向当前未处理过的边中**权值最小**的那条边 (内部实现细节不需要考虑)。设计算法 int MSTWeight(ALGraph G) 判断返回最小生成树的权值之和。邻接表定义如下:

```
typedef struct ArcNode
{
    int ivex;           // 该弧起始的顶点
    int jvex;           // 该弧所指向的顶点
    struct ArcNode *nextarc; // 指向下一条弧的指针
    int weight;         // 边权值
}ArcNode;              // 边结点

typedef struct
{
    int data;           // 顶点信息
    ArcNode *firstarc;  // 指向第一条依附该顶点的弧的指针
}VNode;                // 顶点结点

typedef struct
{
    VNode VerticesList[MAXV]; // 顶点数组
    int numVertices, arcnum;  // 图的当前顶点数和弧数
}ALGraph;
```

(1) FindEdge() 函数每次会从某个数据结构中取出最小边, 应该使用什么数据结构? 为什么? (2) 给出算法的基本设计思想。 (3) 采用 C 或 C++ 语言描述算法, 关键之处给出注释。

【答案】 (1) FindEdge() 的内部数据结构 为了每次都能快速找到权值最小的边, 最适合的数据结构是最小堆 (Min-Heap), 也叫优先队列。原因: 把所有边放进最小堆里, 堆顶永远是当前权值最小的边。每次获取最小边的操作 (出队) 时间复杂度是 $O(\log e)$, 这比每次都遍历所有边来找最小值的 $O(e)$ 要快得多。 (2)

1. 准备一个并查集, 一开始每个顶点都是一个单独的集合。
2. 循环, 直到已经选中 $n-1$ 条不成环的边: a. 每次通过 FindEdge() 得到一条边 (u, v) 。 b. 用并查集检查 u 和 v 是不是已经在同一个集合里了。如果在, 说明加上这条边会形成环, 就跳过边 (u, v) 。 c. 如果不在, 选择该边、合并这两个集合, 把边的权重加到总和里)。
3. 重复这个过程, 直到选出了 $n-1$ 条边, 最小生成树就找到了。 (3)

```
int parent[MAXV];
```

```
// 边结构体定义
```

```

typedef struct {
    int u, v; // 边的两个顶点
    int weight; // 边的权值
} Edge;

// 查找根结点
int Find(int x)
{
    if (parent[x] != x)
        parent[x] = Find(parent[x]); // 路径压缩
    return parent[x];
}

// 并查集：合并集合
void Union(int x, int y)
{
    parent[Find(y)] = Find(x);
}

int MSTWeight(ALGraph G)
{
    Edge edges[MAXE]; // 边集数组
    int count = 0; // 统计已纳入生成树中的边数
    int totalWeight = 0;

    // 1. 初始化并查集，初始时所有顶点属于不同集合。
    for (int i = 0; i < G.numVertices; i++)
        parent[i] = i;

    // 2. 循环，直到选出 n-1 条不成环的边
    while (count < G.numVertices-1)
    {
        // 3-a. 通过 FindEdge() 得到最小的边
        ArcNode *p = FindEdge();

        // 3-b. 对于还未处理的最小边，使用并查集判断该边的两个端点分别属于哪个集合
        int rootU = Find(p->ivex);
        int rootV = Find(p->jvex);

        // 3-c. 如果不属于同一个集合则选择该边、合并这两个集合
        if (rootU != rootV)
        {
            Union(rootU, rootV);
            totalWeight += p->weight;
            count++;
        }
    }
    return totalWeight;
}

```

* 习题 5-5

设无向图 $G=(V, E)$ ，如果顶点可被分割为两个互不相交的子集，并且图中每条边的两个顶点 i 和 j 分别属于这两个不同的顶点集，则称图 G 为一个二分图。无向图 G 为二分图的充要条件是： G 中至少包含两个顶点且 G 中任意一个简单回路所包含的边数为偶数。图 G 采用邻接矩阵存储，类型定义如下：

```
typedef struct {
    int numV, numE;           // 顶点数和边数
    char VerticesList[MAXV];  // 顶点信息
    int Edge[MAXV][MAXV];     // 权值矩阵
}MGraph;
```

请设计算法 `int IsBipartite (MGraph G)`，判断无向图 G 是否为二分图，是则返回 1，否则返回 0。（1）给出算法的基本设计思想。（2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

【解析】本题是要判断是否存在含有偶数个元素的环，此处可以用 DFS 也可以用 BFS。【答案】（1）使用广度优先遍历 BFS 算法：从任意未染色的顶点开始，先将起始 0 号顶点染色为 1，并将其入队。依次处理队列中的顶点，出队一个顶点 v ，遍历其所有相邻顶点 u 。若 u 未被染色，则将其染色为与 v 不同的颜色（1 或 2），并将其入队。若 u 已被染色且颜色与 v 相同，则说明图 G 不是二分图，返回 0。重复上述步骤，直到队列为空。若所有顶点都被染色且没有相邻顶点颜色相同的情况，则说明图 G 是二分图，返回 1。（2）代码如下：

```
int IsBipartite(MGraph G)
{
    int color[MAXV] = {0};    // 记录顶点的颜色，0表示未染色
    int q[MAXV];
    int front = 0, rear = 0;
    for (int v = 0; v < G.numV; v++)
    {
        if (color[v] != 0)
            continue;
        color[v] = 1;
        q[rear++] = v;
        while (front < rear)
        {
            int u = q[front++];
            for (int w = 0; w < G.numV; w++)
            {
                if (G.Edge[u][w] != 1)
                    continue;
                if (color[w] == color[u])
                    return 0;    // 发现边两端顶点颜色相同
                if (color[w] == 0)
                {
                    if (color[u] == 1)
                        color[w] = 2;
                    else color[w] = 1;
                    q[rear++] = w;
                }
            }
        }
    }
}
```

```

    }
}
return 1;
}

```

习题 5-6

请设计算法：int components(ALGraph G)，对给定的无向图 G，返回其中连通分量的个数。
邻接表定义如下：

```

typedef struct ArcNode
{
    int adjvex;           // 该弧所指向的顶点
    struct ArcNode *nextarc; // 指向下一条弧的指针
}ArcNode;               // 边结点
typedef struct
{
    int data;             // 顶点信息
    ArcNode *firstarc;    // 指向第一条依附该顶点的弧的指针
}VNode;                 // 顶点结点
typedef struct
{
    VNode VerticesList[MAXV]; // 顶点数组
    int numVertices, arcnum;  // 图的当前顶点数和弧数
}ALGraph;

```

- (1) 给出算法的基本设计思想。
- (2) 采用 C 或 C++ 语言描述算法，关键之处给出注释。
- (3) 说明所设计算法的时间复杂度。

【答案1】DFS (1)

1. 设置计数器 count，初始为 0。准备一个 visited 数组，记录哪些点走过了，初始都为 false。
2. 循环检查每个顶点：如果该顶点还未访问过，说明找到了一个新的连通分量的起点，就把 count 加 1，递归调用深度优先遍历 DFS。(2) 代码如下：

```

bool visited[MAXV] = {false};

// DFS函数，用于遍历一个连通分量
void DFS(ALGraph G, int v)
{
    visited[v] = true;
    ArcNode *p = G.VerticesList[v].firstarc;
    while (p != NULL)
    {
        if (visited[p->adjvex] != true)

```

```

        DFS(G, p->adjvex);
        p = p->nextarc;
    }
}

```

```

int components(ALGraph G)
{
    int count = 0;
    for (int i = 0; i < G.numVertices; i++)
        if (visited[i] != true)
        {
            count++;
            DFS(G, i);
        }
    return count;
}

```

(3) $O(n+e)$ 。

【答案2】并查集 (1)

1. 设置并查集 father，初始时每个结点的 father[i] = i 自身。
2. 遍历所有边 $\langle u, v \rangle$ ，合并 u 和 v 所在的并查集。
3. 统计这些顶点属于几个集合。

```

int father[MAXV]; // 并查集数组
// 并查集查找（带路径压缩）
int find(int x)
{
    if (father[x] != x)
        father[x] = find(father[x]);
    return father[x];
}
// 并查集合并
void Union(int x, int y)
{
    int fx = find(x);
    int fy = find(y);
    if (fx != fy)
        father[fy] = fx;
}
// 主函数：统计连通分量个数
int components(LGraph G)
{
    // 1. 初始化并查集
    for (int i = 0; i < G.numVertices; i++)
        father[i] = i;
    // 2. 遍历所有顶点
    for (int i = 0; i < G.numVertices; i++)
    {
        ArcNode *p = G.VerticesList[i].firstarc;
        // 遍历当前顶点的所有边
    }
}

```

```

while (p != NULL)
{
    // 合并两个顶点所在集合
    Union(i, p->adjvex);
    // 处理下一条边
    p = p->nextarc;
}
}
// 3.统计不同集合个数
int count = 0;
for (int i = 0; i < G.numVertices; i++)
    if (find(i) == i) count++;
return count;    // 输出连通分量个数
}

```

* 习题 5-7

已知无向图 G 类型定义如下：

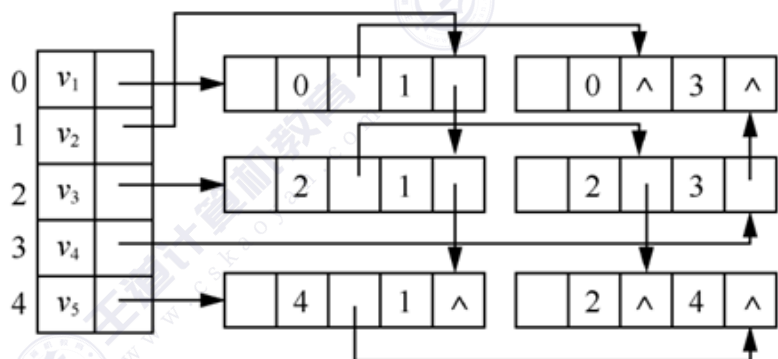
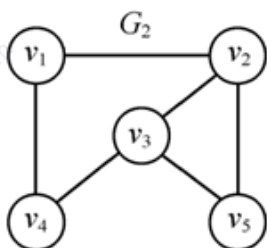
```

typedef struct EBox
{
    int mark;           // 用于遍历标记
    int ivex, jvex;     // 边依附的两个顶点位置
    struct EBox *ilink, *jlink; // 分别指向依附于ivex、jvex的下一条边
} EBox;

typedef struct VBox
{
    char data;          // 顶点信息
    EBox *firstEdge;    // 指向第一条依附该顶点的边
} VBox;

typedef struct
{
    VBox adjMulList[MAXV]; // 顶点表
    int numVertices, numEdges; // 顶点数和边数
} AMLGraph;

```



请设计算法：int KDegree(AMLGraph G, int k)，对给定的无向图 G，返回图 G 中顶点下标为 k 的顶点的度。例如图 G2，k=1 时，对应顶点 V2，度为 3，返回 3。要求：（1）请问图的

这种存储结构叫什么？（2）给出算法的基本设计思想。（3）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

【解析】（1）邻接多重表（2）

1. 从 `G.adjMulList[k].firstEdge` 开始，拿到第一条边。
2. 循环执行，直到查找完链表（NULL）：
a. 计数器加一。
b. 根据当前顶点是 `ivex` 还是 `ivex`，选择正确的下一条边指针。（3）

```
int KDegree(AMLGraph G, int k)
{
    int degree = 0;
    // 1. 从第一条边开始
    EBox *p = G.adjMulList[k].firstEdge;
    // 2. 循环执行，直到查找完链表
    while (p != NULL)
    {
        // 2-a. 计数器加一
        degree++;
        // 2-b. 根据当前顶点是 ivex 还是 jvex，选择正确的下一条边指针
        if (p->ivex == k)
            p = p->ilink;
        else
            p = p->jlink;
    }
    return degree;
}
```

* 习题 5-8

有向完全图的定义是：在有向图中，任意两个顶点 u 和 v 之间，必须同时存在 $u \rightarrow v$ 和 $v \rightarrow u$ 两条弧。对于有向图 G ，请设计算法：`int IsComplete(OLGraph G)` 判断它是不是有向完全图，如果是则返回 1，否则返回 0。请回答：（1）应该采用以下哪种数据结构保存：邻接多重表还是十字链表？（2）写出该数据结构定义。（3）采用 C 或 C++ 语言描述算法，关键之处给出注释。

【解析】对于简单图来说，有向图是有向完全图等价于： $e = n \times (n-1)$ 。或者可以遍历统计所有顶点，看每个顶点是否都可以到达其他顶点。【答案】（1）十字链表（2）定义：

```
#define MAXV 100

// 边结点
typedef struct ArcBox
```

```

{
    int tailvex, headvex;        // 弧尾和弧头在顶点表中的下标
    struct ArcBox *hlink;        // 指向同终点的下一条边（入边链）
    struct ArcBox *tlink;        // 指向同起点的下一条边（出边链）
    int weight;                  // 可选：边权值
} ArcBox;

// 顶点结点
typedef struct VexNode
{
    char data;                  // 顶点信息
    ArcBox *firstin, *firstout; // 入边表、出边表指针
} VexNode;

// 十字链表图定义
typedef struct
{
    VexNode VerticesList[MAXV]; // 顶点表
    int numVertices, numEdges;   // 顶点数、边数
} OLGraph;

```

(3) 代码如下：

```

int IsComplete(OLGraph G)
{
    if (G.numEdges == G.numVertices*(G.numVertices-1))
        return 1;    // e = n*(n-1)
    else 0;
}

```

或者统计所有结点的出度是否为 $n-1$ ：

```

int IsComplete(OLGraph G)
{
    // 遍历所有顶点
    for (int i = 0; i < G.numVertices; i++)
    {
        int count = 0;                // 统计到达几个顶点

        // 遍历当前顶点i发出的所有边
        ArcNode *p = G.VerticesList[i].firstout; // 请注意是出边
        while (p != NULL)
        {
            count++;                // 访问对应边
            p = p->tlink;            // 起点相同
        }

        if (count != G.numVertices-1) return 0; // 该顶点不满足要求
    }
    return 1;                        // 所有顶点都满足要求
}

```

5.5.2 真题

2021 年 41 题

(15 分) 已知无向连通图 G 由顶点集 V 和边集 E 组成, $|E| > 0$, 当 G 中度为奇数的顶点个数为不大于 2 的偶数时, G 存在包含所有边且长度为 $|E|$ 的路径 (称为 EL 路径)。设图 G 采用邻接矩阵存储, 类型定义如下:

```
typedef struct{           // 图的定义
    int numVertices, numEdges; // 图中实际的顶点数和边数
    char VerticesList[MAXV]; // 顶点表。MAXV为已定义常量
    int Edge[MAXV][MAXV]; // 邻接矩阵
}MGraph;
```

请设计算法 `int IsExistEL(MGraph G)`, 判断 G 是否存在 EL 路径, 若存在, 则返回 1, 否则返回 0。要求: (1) 给出算法的基本设计思想。(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(3) 说明你所设计算法的时间复杂度和空间复杂度。

【解析】分析可知本题需要访问所有顶点, 统计所有顶点的度, 然后判断有多少个度为奇数的顶点, 只需要对邻接矩阵一次遍历即可实现。【答案】(1) 访问所有的顶点, 两层循环判断每两个顶点间是否有边相连、统计所有顶点的度, 进而求出有多少个度为奇数的顶点, 对邻接矩阵一次遍历即可。(2) 算法如下:

```
int IsExistEL(MGraph G)
{
    int count = 0;
    for (int i = 0; i < G.numVertices; i++)
    {
        int degree = 0;
        for (int j = 0; j < G.numVertices; j++)
            if (G.Edge[i][j] == 1)
                degree++;
        if (degree % 2 == 1)
            count++;
    }
    if (count == 0 || count == 2)
        return 1;
    else
        return 0;
}
```

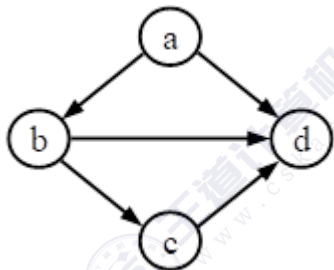
(3) 时间复杂度: $O(n^2)$ 空间复杂度: $O(1)$

2023 年 41 题

(13 分) 已知有向图 G 采用邻接矩阵存储，类型定义如下：

```
typedef struct          // 图的类型定义
{
    int    numVertices, numEdges; // 图的顶点数和有向边数
    char    VerticesList[ MAXV ]; // 顶点表, MAXV为已定义常量
    int     Edge[ MAXV ][ MAXV ]; // 邻接矩阵
} MGraph;
```

将图中出度大于入度的顶点称为 K 顶点。例如在下图中，顶点 a 和 b 为 K 顶点。



请设计算法：int printVertices(MGraph G)，对给定的任意非空有向图 G，输出 G 中所有的 K 顶点，并返回 K 顶点的个数。要求：1) 给出算法的基本设计思想。2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

【解析】出度大于入度的顶点称为 K 顶点，本题关键是遍历图求出所有顶点的入度和出度，才能判断出顶点是否为 K 顶点。依次判断所有顶点是否为 K 顶点，并统计数量即可。

【答案】注：以下两种答案本质一样，仅供参考

(1) 图的邻接矩阵 $G.Edge[i][j]>0$ 表示 i 到 j 有边，二重循环遍历邻接矩阵，算出每个顶点的入度 indegree 和出度 outdegree，然后判断每个顶点是否为 K 顶点，如果是则输出并且计数器 count+1，最后返回 count 即可。(2) 写法 1：

```
int printVertices(MGraph G)
{
    int indegree, outdegree;          // 入度和出度
    int count = 0;                    // 计数器count初值为0
    for (int i = 0; i < G.numVertices; i++)
    {
        indegree = outdegree = 0;      // 更换i之后要清零，重新统计入度和出度
        for (int j = 0; j < G.numVertices; j++)
            if (G.Edge[i][j] > 0) outdegree++; // i到j有边，i的出度+1
        for (int j = 0; j < G.numVertices; j++)
            if (G.Edge[j][i] > 0) indegree++; // j到i有边，i的入度+1
        if (outdegree > indegree)          // 如果i的出度大于入度则说明i是K顶点
        {
            printf("%c", G.VerticesList[i]);
            count++;
        }
    }
}
```

```
    return count;
}
```

写法 2:

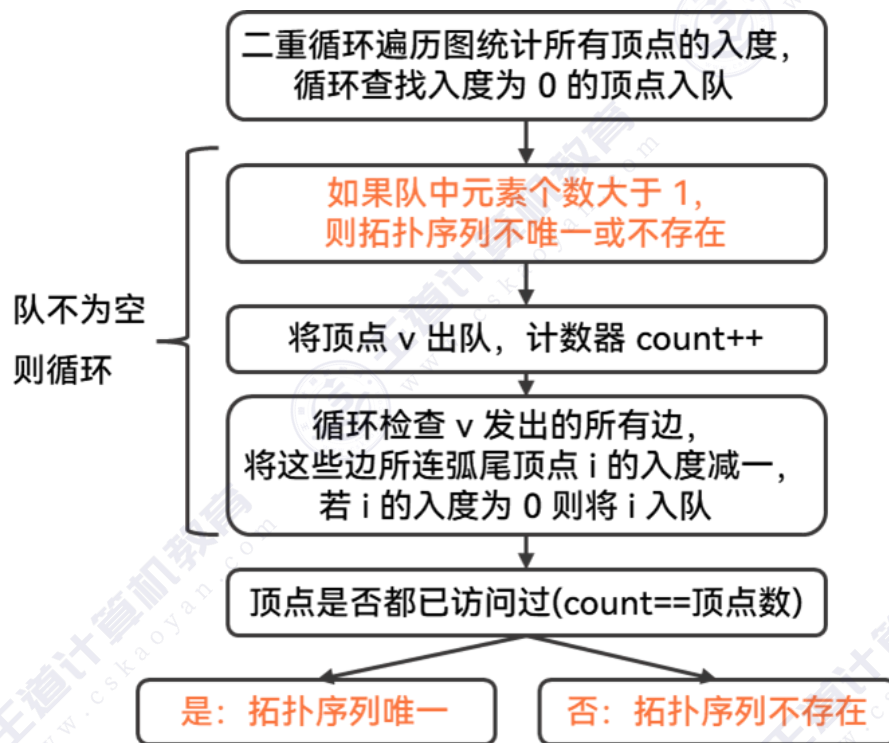
```
int printVertices(MGraph G)
{
    int indegree, outdegree;           // 入度和出度
    int count = 0;                     // 计数器count初值为0
    for (int i = 0; i < G.numVertices; i++)
    {
        indegree = outdegree = 0;      // 更换i之后要清零，重新统计入度和出度
        for (int j = 0; j < G.numVertices; j++)
        {
            if (G.Edge[i][j] > 0) outdegree++; // i到j有边，i的出度+1
            if (G.Edge[j][i] > 0) indegree++;  // j到i有边，i的入度+1
        }
        if (outdegree > indegree)           // 如果i的出度大于入度则说明i是K顶点
        {
            输出G.VerticesList[i]
            count++;
        }
    }
    return count;
}
```

2024 年 41 题

(13分) 2023 年 10 月 26 日，神舟十七号载人飞船发射取得圆满成功，再次彰显了中国航天事业辉煌成就。载人航天工程是包含众多子工程的复杂系统工程，为了保证工程的有序开展，需要明确各子工程的前导子工程，以协调各子工程的实施。该问题可以简化、抽象为有向图的拓扑序列问题。已知有向图 G 采用邻接矩阵存储，类型定义如下。

```
typedef struct           //图的类型定义
{
    int numVertices, numEdges; //图的顶点数和有向边数
    char VerticesList[MAXV];   //顶点表，MAXV为已定义常量
    int Edge[MAXV][MAXV];      //邻接矩阵
} MGraph;
```

请设计算法：int uniquely(MGraph G)，判定 G 是否存在唯一的拓扑序列，若是，则返回 1，否则返回 0。要求如下。1) 给出算法的基本设计思想。(4分) 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。(9分)



【解析 1】

(1)

【答案 1】

1. 遍历图统计所有顶点的入度，将入度为 0 的顶点都入队。
2. 循环执行：① 如果队中元素个数不为 1，说明拓扑序列不唯一或不存在，return 0。② 出队一个顶点记为 v，count++。③ 更新所有与顶点 v 相连的顶点 i 的入度，若 i 的入度为 0，则将 i 入队。
3. 如果 count 等于顶点数则拓扑序列唯一，return 1；否则拓扑序列不存在，return 0。

(2) 代码如下：

```
int uniquely1(MGraph G)
{
    int inDegree[MAXV] = {0}; // 存储每个顶点的入度
    int queue[MAXV]; // 用于存储入度为0的顶点
    int front = 0, rear = 0; // 队列的前后指针
    int count = 0; // 拓扑序列中的顶点计数
    // 计算每个顶点的入度
    for (int i = 0; i < G.numVertices; i++)
        for (int j = 0; j < G.numVertices; j++)
            if (G.Edge[i][j] != 0)
                inDegree[j]++;
    // 将所有入度为0的顶点入队
    for (int i = 0; i < G.numVertices; i++)
        if (inDegree[i] == 0)
            queue[rear++] = i;
    // 进行拓扑排序
    while (front < rear)
    {
        // 如果队列中有多个顶点，说明图的拓扑序列不唯一
        if (rear - front > 1)
            return 0;
        int v = queue[front++]; // 取出队首顶点
        count++;
        // 更新该顶点的所有邻接顶点的入度
        for (int i = 0; i < G.numVertices; i++)
```

```

if (G.Edge[v][i] != 0)
{
    inDegree[i]--; // 如果入度变为0, 则加入队列
    if (inDegree[i] == 0)
        queue[rear++] = i;
}
}
if (count == G.numVertices)
    return 1; // 拓扑序列包含所有顶点, 图的拓扑序列唯一
else
    return 0; // 拓扑序列不包含所有顶点, 图的拓扑序列不存在
}

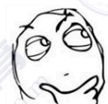
```

【解析 2】

拓扑序列	条件
存在且唯一	每次选择结点时都只有一个结点可选
存在且不唯一	选择结点时不止一个结点可选
不存在	图中有环, 环中的顶点未被访问

拓扑排序的步骤:

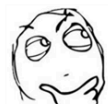
1. 选择入度为 0 的顶点并输出它。
2. 从图中删除该顶点和所有以它为弧头的边。
3. 重复步骤 1、2, 直到不存在入度为 0 的顶点。



拓扑排序为什么使用队列?



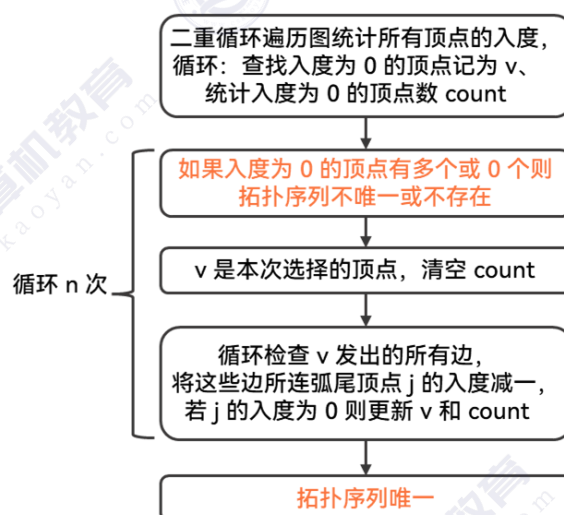
可能有多个入度为 0 的顶点。



有多个选择时就会 return 0, 是不是只需要考虑选择唯一时的情况?



选择唯一就可以不使用队列!



【答案 2】 (1)

1. 遍历图并统计所有顶点的入度, count 是入度为 0 的顶点的数量, 入度为 0 的顶点记为 v。
2. 循环执行: ① 如果 count 不为 1, 说明拓扑序列不唯一或不存在, return 0。② v 是本次选择的顶点, 将 count 清零重新计数。③ 更新所有与顶点 v 相连的顶点 j 的入度, 若 j 的入度为 0, 则计数器 count++。
3. 拓扑序列唯一, return 1。 (2) 代码如下:

```

int uniquely(MGraph G)
{
    int v, inDegree[MAXV] = {0};

    // 计算每个顶点的入度
    for (int i = 0; i < G.numVertices; i++)
        for (int j = 0; j < G.numVertices; j++)
            if (G.Edge[i][j] > 0)

```

```

        inDegree[j]++;
int count = 0;
for (int j = 0; j < G.numVertices; j++)
    if (inDegree[j] == 0)
    {
        v = j;
        count++;          // 统计入度为0的顶点数
    }

// 循环执行次数等于顶点数，若循环执行完则拓扑序列唯一
for (int i = 0; i < G.numVertices; i++)
{
    if (count != 1)
        return 0;          // 若count不等于1，拓扑序列不存在或不唯一
    count = 0;              // 下一次选择时入度为0的顶点数先设为0
    int temp = v;
    for (int j = 0; j < G.numVertices; ++j)
        if (G.Edge[temp][j] != 0)
        {
            inDegree[j]--;    // 将v所连弧尾顶点的入度-1
            if (inDegree[j] == 0)
            {
                v = j;
                count++;        // 如果j的入度为0则count++
            }
        }
}
return 1;                  // 循环执行完，拓扑序列唯一
}

```

6 补充知识

6.1 注释与函数

考试中的注释也是必不可少的，因为重要的是让老师理解我们的思路，所以在关键位置处写注释有助于老师阅读。当然如果你的代码足够清晰、明确，也可以不写注释。

- 在变量的定义处告知该变量是做什么的：

```
bool visited[MAXV] = {false}; // 表示顶点是否已入队
```

- 在使用伪代码处和函数调用处告知这部分实现哪些功能：

```
输出 xxx  
  
swap(A[L], A[i]); // 交换 A[L] 与 A[i]
```

- 代码逻辑复杂的地方告知其功能：

```
// 计算每个顶点的入度  
for (int i = 0; i < G.numVertices; i++)  
    for (int j = 0; j < G.numVertices; j++)  
        if (G.Edge[i][j] > 0)  
            inDegree[j]++;  
  
int count = 0;  
for (int j = 0; j < G.numVertices; j++)  
    if (inDegree[j] == 0)  
    {  
        v = j;  
        count++; // 统计入度为 0 的顶点数  
    }
```

很多简单的代码、库函数写起来浪费时间，考试时建议只写调用而不写具体内部实现，只要让老师知道你想干嘛即可。但重要的代码一定不能这样，否则是 0 分。比如说本题就是考察快速排序，如果直接调用 Qsort() 而不写快排具体过程，当然拿不到分。

那如何判断？只要这部分代码有可能超过你的所有代码 1/4 或者超过 5 行（表示他对于你的代码很重要），就一定要写。

以下是给大家总结的最常见可以偷懒节省时间的地方，考试时简写，使用的时候写清楚功能而不需要给出具体实现细节。

6.1.1 最大最小值 INT_MAX、INT_MIN

从名字可以看出来，INT_MAX 是最大的整数值，INT_MIN 是最小的整数值（C 语言自带函数，在头文件 limits.h 中），他常用于比较选择最大最小值时，比如初值设为 INT_MAX，那遇到任意值都会比他小或者相等，这样只要遇到比他小的就更新为更小的值。

```
int D_min = INT_MAX;           // D_min 初始设为最大的整数
for (int i = 0; i < n; i++)    // 遍历 D 循环选出最小的 D
    if (D[i] < D_min) D_min = D[i]; // D[i]比 D_min 小, D_min 更新
```

6.1.2 比大小函数 max(a, b)、min(a, b)

函数定义如下（不用掌握）：

```
int min(int a, int b)
{
    if (a < b) return a;
    else return b;
}

int max(int a, int b)
{
    if (a > b) return a;
    else return b;
}
```

使用这两个函数能让我们的代码变得简洁、清晰：

```
int D_min = INT_MAX;    // D_min 初始设为最大的整数

// 遍历 D 循环选出最小的 D
for (int i = 0; i < n; i++)
    D_min = min(D_min, D[i]);
```

如果我们是要选出最小值，则初值应该赋最大可能值。如果我们是要选出最大值，则初值应该赋最小可能值。

6.1.3 输入输出函数 scanf、printf

C 语言中的输入、输出函数分别是 scanf 和 printf 函数，因为和变量的类型有关，写起来比较麻烦，在考试中可以用 C++，那我们可以使用 C++ 的 cin、cout 函数来输入、输出，而不用管变量的类型，会简洁很多。

```
int a;
float b;
char c;
char d[10];
scanf("%d%f%c%s", &a, &b, &c, &d); // 输入整数a、浮点数b、字符c、字符串d
printf("%d%f%c%s", a, b, c, d);    // 输出整数a、浮点数b、字符c、字符串d
```

考试中需要记住如下格式：

整型 int	浮点型 float	字符型 char	字符串 char[]
%d	%f	%c	%s

如果怕自己记错，也可以直接写中文，如：输入 a、b、c、d；输出 a、b、c、d。写中文还有一个好处，需要循环输入输出的时候，中文描述更简便，如循环输出 int 数组 a 中的值：

```
for (int i = 0; i < n; i++)
    printf("%d ", a[i]);
```

用中文描述：

输出数组 a

除以上两种写法外，还可以用 C++ 的写法，输出是 `cout << a[i]`，输入是 `cin >> a[i]`。

6.1.4 a[i++] 和 a[++i]

这两个写法是不一样的，`a[i++]` 中 `i` 在前表示先使用 `a[i]`，再 `i+1`；`a[++i]` 中先 `i+1`，再使用 `a[i]`（此时 `i` 是 `+1` 后的结果）。这两种写法相当于两步缩写成了一步。

假设 `a[0] = 0`, `a[1] = 1`, `i = 0`。

- `x = a[i++]` 这条指令，相当于先用再加：

```
x = a[i];
i++;
```

`x` 的值为 `a[0] = 0`。

- `x = a[++i]` 这条指令，相当于先加再用：

```
i++;
x = a[i];
```

`x` 的值为 `a[1] = 1`。

6.1.5 交换函数 swap(a, b)

函数定义如下（不用掌握）：

```
void swap(int &a, int &b)
{
    int temp = a;
    a = b;
```

```
b = temp;
}
```

让我们看看这两种不同的写法

① 不使用 swap 函数：

```
// 冒泡排序
for(int i = n-1; i > 1; i--)
    for (int j = 0; j < i; j++)
        if (A[i] < A[j])
        {
            temp = A[i];
            A[i] = A[j];
            A[j] = temp;
        }
```

② 使用 swap 函数：

```
// 冒泡排序
for(int i = n-1; i > 1; i--)
    for (int j = 0; j < i; j++)
        if (A[i] < A[j]) swap(A[i], A[j]);
```

6.2 复杂度的估算

考试中经常要求时间和空间复杂度，如无特殊说明，数据结构大题中间的时间和空间复杂度都是最坏情况下的复杂度，只有一个例外：快速排序（考虑平均复杂度，或者给快速排序增加一种优化）。

在计算机学科中， $\log_2 n$ 常常写作 $\log n$ ，他们都是以 2 为底的对数运算。

6.2.1 时间复杂度

时间复杂度是对时间增长速度的一个估计。时间复杂度为 $O(f(n))$ 表示该算法执行的指令数的上界是 $O(f(n))$ 级别的，可以简单的用极限理解，若所有指令总执行次数 T ，时间复杂度为 $O(f(n))$ 的算法表示 $T = K$ (K 为常数)。

例如 $T1 = n^2 + n$, $T2 = 2n^2 - n$ ，通过极限的思想得到这两个算法的 $f(n) = n^2$ 即时间复杂度都是 $O(n^2)$ ，常数 $K1=1$, $K2=2$, $K1 \neq K2$ ，但复杂度是一样的，所以常数系数不影响复杂度；本例中时间复杂度只有最高次决定，所以较低次项不影响复杂度。总之我们就是需要查看算法的指令执行次数 T 的最高次项（而不考虑他的系数）。

常见时间复杂度： $O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3)$ 。

$O(\log_2 n)$ 和 $O(\log_3 n)$ 其实是相等的，因为 $\log_2 n = \log_2 3 \times \log_3 n$ ，而 $\log_2 3$ 是常数，所以 $O(\log_2 n) = O(\log_3 n)$ ，考虑复杂度时，所有的 $\log$ 都尽量写成 $\log_2 n$ 或者 $\log n$ 的形式。

对于考试时写的代码，时间复杂度只需要考虑**循环和递归次数**。

循环次数

所有循环都可以写成 for 循环。

以链表和数组为例，如果我们知道访问次数，尽量用 for 循环

```
for (i = 0; i < n; i++)
    cout << A[i];
```

如果不知道访问次数而是满足某个条件时终止，用 while 循环

```
while (p != NULL)
{
    cout << p->data;
    p = p->next;
}
```

① 对于多层 for 循环，如果每一层的变量都是自增 1，则直接把每层执行次数相乘

```

for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        for (k = 0; k < n; k++)
            if (A[i][j] > A[i][k] + A[k][j])
                A[i][j] = A[i][k] + A[k][j];

```

他的每一层都是变量自增，且变量都是从 0~n-1 共 n 次，所以总次数是 n^3 ，时间复杂度是 $O(n^3)$ 。

② 对于多层 for 循环，如果每一层的变量都是自增 1，但第二三层变量取值范围和第一层变量有关

```

for (i = 0; i < n; i++)
    for (j = 0; j < i; j++)
        for (k = 0; k < j; k++)
            if (A[i][j] > A[i][k] + A[k][j])
                A[i][j] = A[i][k] + A[k][j];

```

时间复杂度和第 ① 种一样，但不用考虑具体次数，时间复杂度是 $O(n^3)$ 。

③ 对于两层循环，变量不是自增 1（这种情况主要针对小题）

```

// 22 考研真题
for (i = 1; i < n; i*=2)
    for (j = 0; j < i; j++)
        sum++;

```

需要讨论第一层变量 i 取不同值时，j 的取值有 x 种，然后对 x 求和。比如本题， $i = 1, 2, 4, 8 \dots 2^k (2^k < n \leq 2^{k+1})$ 时，第二层 j 有 i 个取值，所以总次数是对 i 求和，即 $T = 1+2+4+2^k = 2^{k+1}-1$ ， $n < T < 2n$ ，时间复杂度就是 $O(n)$ 。

递归次数

递归总次数和时间复杂度有关，而最大递归层数和空间复杂度有关。

对于树中序遍历的递归写法，每个结点都会调用一次所以总共调用次数是 $O(n)$ 级别的，时间复杂度是 $O(n)$ 。

```
void InOrder(BTNode *p)
{
    if (p == NULL) return;
    InOrder(p->lchild);
    visit(p); // 对 p 结点的访问等
    InOrder(p->rchild);
}
```

6.2.2 空间复杂度

只算额外空间开销，不考虑题目中已经给出的数据所占的空间，比如题目中给出了一个数组，这个数组是不计入空间复杂度的。

定义的数组、链表

单个定义的变量都不需要考虑，但定义的数组中元素个数以及链表中结点个数都要计入空间复杂度中。

```
// 单个变量不考虑
int i, j, k;

// 数组考虑元素个数，空间复杂度 O(n)
int a[n];

// 链表考虑结点个数，空间复杂度 O(n)
Node* L = (Node*)malloc(sizeof(Node));
Node* p = L;
for (int i = 0; i < n; i++)
{
    p->next = (Node*)malloc(sizeof(Node));
    p = p->next;
}
```

递归层数

是在递归栈中最多出现多少个过程函数，层数和总次数比较见时间复杂度的递归次数。

对于树中序遍历的递归写法，每个结点都会调用一次所以总共调用次数是 $O(n)$ 级别的，时间复杂度是 $O(n)$ ，而递归使用的层数最多是树高 h ，空间复杂度是 $O(h)$ ，但 h 是不确定的，而我们又知道 $h \leq n$ ，最坏情况下高度 $h = n$ ，因此空间复杂度为 $O(n)$ 。

```
void InOrder(BTNode *p)
{
    if (p == NULL) return;
    InOrder(p->lchild);
    visit(p); // 对 p 结点的访问等
    InOrder(p->rchild);
}
```

6.3 书写

6.3.1 题目要求

考试的代码题一般分为三小问：

第一小问：设计思想

主要介绍逻辑，告诉老师你用了什么，比如快速排序，重要的是让老师知道你用的方法是什么。

第二小问：描述算法

注释要详实，用来介绍变量、过程的作用，提示老师。对于快速排序和折半查找（注意是否查找成功）可以直接默写，过程函数不需要改动，注意调用的参数即可。

第三小问：求时间和空间复杂度

直接写复杂度，不用过程，要检验是否符合代码。

6.3.2 作答建议

本节指引

本节不仅会教你如何解题，更重要的是：如果你不敢面对算法题、总想着数据结构还没学好，该方法旨在帮助你克服对代码题的**心理恐惧**和**无力感**。

1. **化解“无从下手”的绝望感**：把一个大问题拆解成多个小任务，让你不再觉得面对的是一座无法翻越的高山，而是一级级可以攀登的阶梯。每完成一步，你都会获得一份正向反馈，这份小小的成就感会驱动你继续前进。
2. **降低“害怕犯错”的恐惧**：先用草稿纸思考，再用文字描述，最后才动手写代码。这个过程允许你在**低成本**的环境中犯错和调整，很多时候你写不出来代码，不是你完全不会，可能只是某些模块没掌握。你可以准确定位问题，并针对性地学习和掌握，无需再害怕犯错。

这套方法不是让你立刻变得完美，而是让你变得比昨天更好，早晚有一天能够彻底拿下算法题。请今天开始，勇敢地面对它！

我建议同学们先思考清楚整个算法需要做哪些事情已经对应流程，分析完复杂度再开始写。

对于能想出思路但不会下笔的同学我有如下建议，可以练习模块化写法，分为如下步骤：

1. 想好思路之后在**草稿纸上**写简易流程图，把一道题分为不同的步骤，估算复杂度并确保方法和复杂度都正确。
2. 根据流程图在答题卡上回答第一小问设计思想，适当扩充描述。
3. 根据流程图在答题卡上回答第二小问描述算法，按照步骤尝试写出每一块的代码，这样可以减少代码的涂改。
4. 写上时间和空间复杂度。

在练习时使用此方法有助于找到自己的问题到底是理不清过程还是具体哪一块代码不会写（比如说折半查找），这有助于我们针对性练习对应模块。

以真题 2011.47 为例：

（15 分）一个长度为 L ($L \geq 1$) 的升序序列 S ，处在第 $\lfloor L/2 \rfloor$ 个位置的数称为 S 的中位数。例如，若序列 $S_1 = (11, 13, 15, 17, 19)$ ，则 S_1 的中位数是 15，两个序列的中位数是含它们所有元素的升序序列的中位数。例如，若 $S_2 = (2, 4, 6, 8, 20)$ ，则 S_1 和 S_2 的中位数是 11。现在有两个等长升序序列 A 和 B ，试设计一个在时间和空间两方面都尽可能高效的算法，找出两个序列 A 和 B 的中位数。要求：（1）给出算法的基本设计思想。（2）根据设计思想，采用 C、C++ 或 Java 语言描述算法，关键之处给出注释。（3）说明你所设计算法的时间复杂度和空间复杂度。

	做题思路	时间复杂度	空间复杂度	预计得分
暴力解	快速排序	$O(n\log n)$	$O(n)$	11
优化解	多指针法+新数组	$O(n)$	$O(n)$	13
优化解	多指针法	$O(n)$	$O(1)$	14
优化解	折半查找	$O(\log n)$	$O(1)$	15

步骤 1：思考、在草稿纸上写流程图

想好思路之后在草稿纸上写简易流程图，把一道题分为不同的步骤，估算复杂度并确保方法和复杂度都正确。

分析：题目中要我们求出所有元素的中位数，那如果所有元素已经排好序，中间的元素就是中位数。因此归并两个有序数组得到合并数组，然后输出中位数。

【解法2】优化解：多指针法，归并两个数组

流程：

1. 合并两个数组。
2. 输出中位数。

详细流程：

1. 合并两个数组。 a. 设置 i、j、k 分别指向各数组最小元素。 b. 循环将最小值存入新数组。
c. 将剩余元素依次存入数组 C 中。
2. 输出中位数。 时间复杂度为 $O(n)$ ，空间复杂度为 $O(n)$ 。

步骤 2：在答题卡上写第 1 小问

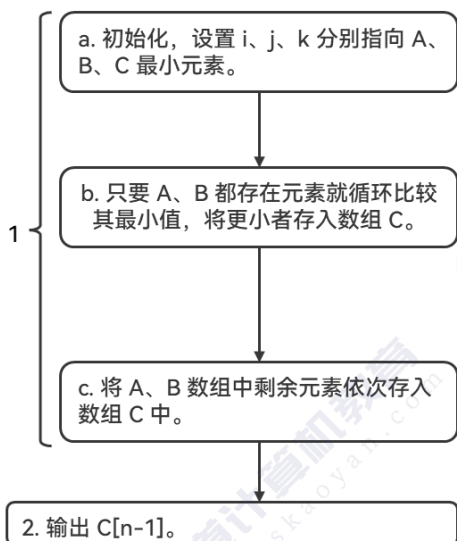
根据草稿纸的简易流程图扩写变成第 1 小问，在写的时候需要慢慢考虑实现问题。

1. 合并两个数组，得到新的升序数组 C。
 - a. 设置 i、j、k 分别指向 A、B、C 最小元素。
 - b. 只要 A、B 都存在元素就循环比较其最小值，将更小者存入数组 C。
 - c. 将 A、B 数组中剩余元素依次存入数组 C 中。
2. 输出 C[n-1]。

步骤 3：在答题卡上写第 2 小问

根据简易流程图和第 1 小问，一个模块一个模块将过程变为代码。

1. 合并两个数组，得到新的升序数组 C。



```
int ans(int A[], int B[], int n)
{
    // 1-a. 设置 i、j、k 分别指向 A、B、C 最小元素
    int C[2 * n];
    int i = 0, j = 0, k = 0;

    // 1-b. 只要 A、B 都存在元素就循环比较其最小值, 将更小者存入 C
    while (i < n && j < n)
        if (A[i] < B[j])
            C[k++] = A[i++];
        else
            C[k++] = B[j++];

    // 1-c. 将 A、B 数组中剩余元素依次存入数组 C 中
    while (i < n)
        C[k++] = A[i++];
    while (j < n)
        C[k++] = B[j++];

    // 2. 输出中间的元素
    return C[n - 1]; // 输出C[n-1]
}
```

步骤 4：在答题卡上写第 3 小问

书写时间和空间复杂度

(3) 时间复杂度： $O(n)$ 空间复杂度： $O(n)$

* 步骤 5：复习时复盘

基础不太扎实的同学，如果做不出来算法题，可以分析到底是哪里出了问题：

- 不能很好的理清楚步骤、流程：

这一类同学思路有问题，需要优先练习思路，可以先尽可能多做题（真题+习题），只写步骤，不写代码，当你大部分题目都可以写出流程时再做下一步。

- 能写出详细的流程，但无法将对应流程转化为代码：

这一类同学思路没问题但代码写太少了，要把所有真题练习完，如果某一道题过程没问题而只是具体模块不会写代码，则可以针对性的练习这一块代码，如：快速排序、折半查找，只要所有的模块都能写出代码，完整的代码肯定也没问题。

大部分畏惧代码题的同学都是因为自身在很多地方都有问题，自己只知道写不出来，但是哪里有问题不知道，这样的话也不能够有的放矢的分析，而总是想着我重新学一遍数据结构说不定就会了，但其实我们更需要的是针对性练习，首先知道问题在哪：是过程不会写还是具体某个模块有问题。在此基础上再针对性复习就能事半功倍。

6.4 复习思维导图

算法题

